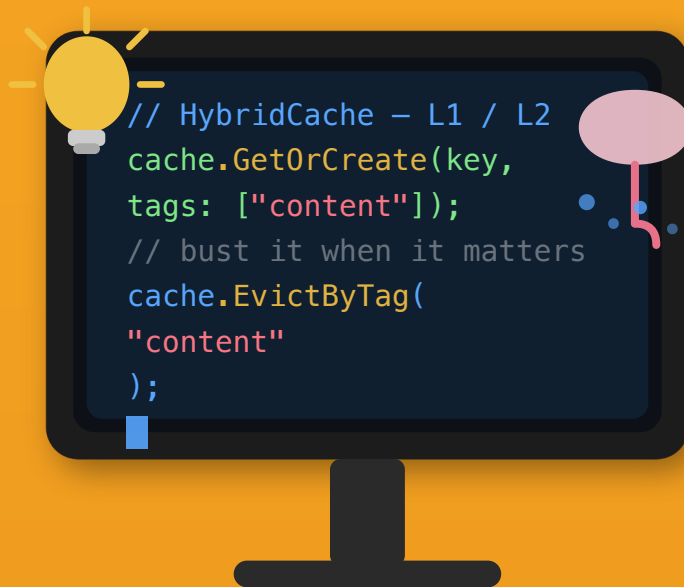




Umbraco Cache for the **Intelligent Dummy**

-
- HybridCache, NuCache & output caching explained
 - Cache busting & invalidation without the guesswork
 - Load balancing, tags, seeding & stale-data risk

01. The Big Picture

Start here. This chapter is the map for everything that follows. By the end you will know that Umbraco has not one cache but a small family of them, what each one is for, and the single idea the rest of the book keeps circling back to: *deciding when to throw cached data away is harder than storing it*. No setup required — just the lay of the land.

If you only remember one thing from this chapter, make it this:

"Umbraco cache" is not one single cache. It is a small family of caches with different jobs.

A lot of confusion around Umbraco caching comes from treating it as one thing. Once you see it as a family, the individual members start to make sense.

Analogy — the restaurant kitchen. Picture a busy restaurant on a Friday night. There is a walk-in fridge holding every raw ingredient — that is your *database*. There are prep stations with ingredients already chopped and ready to assemble — that is the *published content cache*. There are finished plates under the heat lamp, ready to carry straight out — that is the *output cache*. And there is a shelf by the front door where takeaway bags wait for customers to grab without ever talking to staff — that is the *browser and proxy cache*. Almost every cache in this book is one of those surfaces, and we will keep coming back to this kitchen.

The three most important cache stories

1. Published content cache — the prep stations

This is the kitchen's *mise en place*: content prepped ahead of time so nobody has to sprint to the walk-in fridge in the middle of an order. Every time Umbraco needs to serve a published document, a media item, a domain, or a related published model, it goes through this cache first.

- Used when Umbraco loads published documents, media, domains, and related published models
- In Umbraco 17 it is built on Microsoft's `HybridCache` [^01-hybrid]
- It also keeps a small in-process "fast lane" cache for already materialised published objects
- This is not the same thing as website output caching

The fast-lane cache is worth knowing about: if the same published object was already resolved earlier in the request, it is handed back immediately from in-process memory without touching `HybridCache` at all.

Key term — materialised object. When Umbraco turns raw cached data into a ready-to-use `IPublishedContent` object, it has *materialised* it. The "fast lane" stores those already-materialised objects, so the next request can skip the assembly step entirely.

2. Content Delivery API output cache — plates under the heat lamp

This is the finished plate kept warm and ready to serve: rather than re-cook a response from scratch, Umbraco hands out the JSON it already prepared. If you are running Umbraco as a headless CMS — feeding JSON to a Next.js frontend, a mobile app, or anything else that is not a Razor view — this is the output cache that matters to you.

- Stores ready-made JSON responses on the server
- Skips re-fetching from the published content cache on repeated requests to the same URL
- Keyed on URL, query string, and culture
- Invalidated automatically when content is published or unpublished
- Opt-in, disabled by default

Going deeper — skim on a first read. The next two sections crack open the machinery behind "invalidated automatically" with real code, and they are some of the most satisfying detail in the book. If you only want the big-picture map for now, skip ahead to the third cache (browser and proxy) and circle back once the lay of the land has settled.

How invalidation actually works: tag-based eviction

The bullet above says "invalidated when content is published." That is true, but the *how* is worth understanding — it explains why only the right responses get dropped instead of the whole cache being wiped.

When the CDA stores a response, it stamps it with tags derived from the content it contains. Think of a tag as a sticky label on a cached entry. Every tag is a string constant defined in `Constants.DeliveryApi.OutputCache` :

```
// src/Umbraco.Core/Constants-DeliveryApi.cs
public const string ContentTagPrefix    = "umb-dapi-content-";          // + content GUID
public const string MediaTagPrefix      = "umb-dapi-media-";            // + media GUID
public const string AncestorTagPrefix  = "umb-dapi-content-ancestor-"; // + ancestor GUID
public const string ContentTypeTagPrefix = "umb-dapi-content-type-";    // + content type alias
public const string AllContentTag       = "umb-dapi-content-all";
public const string AllMediaTag         = "umb-dapi-media-all";
public const string AllTag              = "umb-dapi-all";
```

So a cached response for a page with GUID `a1b2...` gets the tag `umb-dapi-content-a1b2...` stamped on it at store time. This happens in `DeliveryApiOutputCachePolicyBase.ServeResponseAsync` , which runs once per unique response before it is written to the cache:

```
// src/Umbraco.Cms.Api.Delivery/Caching/DeliveryApiOutputCachePolicyBase.cs
foreach (IPublishedContent item in items)
{
    // Tag with specific item key for targeted eviction.
    context.Tags.Add(ItemTagPrefix + item.Key);
    // ... plus ancestor tags, content-type tags, and custom provider tags
}
```

When an editor publishes content, Umbraco fires a `ContentCacheRefresherNotification` . The `DeliveryApiDocumentOutputCacheEvictionHandler` picks that up and evicts by tag — not by URL:

```
// src/Umbraco.Cms.Api.Delivery/Caching/DeliveryApiDocumentOutputCacheEvictionHandler.cs
await OutputCacheStore.EvictByTagAsync(
    Constants.DeliveryApi.OutputCache.ContentTagPrefix + contentKey,
    cancellationTokens);
```

The result is surgical: only the responses that actually contain the changed content are dropped. A homepage that has nothing to do with the updated article keeps its cached response perfectly intact.

Branch publishes need a wider sweep. If a whole branch of the content tree is refreshed (`TreeChangeTypes.RefreshBranch`), the handler also evicts by the ancestor tag — so any response that listed the changed node as an ancestor is cleared too:

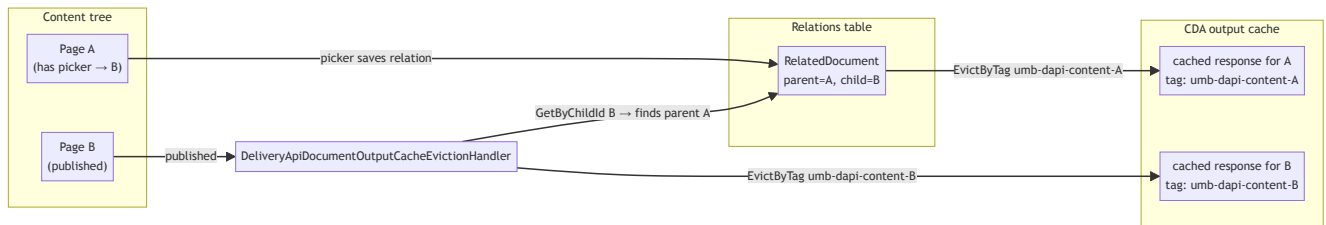
```
await OutputCacheStore.EvictByTagAsync(
    Constants.DeliveryApi.OutputCache.AncestorTagPrefix + contentKey,
    cancellationTokens);
```

Full-tree refreshes fall back to evicting `AllTag` , which clears every CDA cache entry at once. This is the sledgehammer, used only when precision is not possible.

Relation-based eviction

There is a subtler case worth knowing about. Imagine page A has a picker property that points at page B, and the CDA response for A embeds B's data inline. If an editor publishes B, A's cached response is now stale — but A's GUID was never in the change payload, so the tag for A would not normally be touched.

Umbraco handles this through its relation system. When content is saved with a picker property, Umbraco automatically records a `RelatedDocument` relation from A (the referencing page, treated as parent) to B (the picked item, treated as child). After every content change, the eviction handler queries those relations and evicts the referencing pages too:



The code in `RelationOutputCacheEvictionHandlerBase.EvictRelatedContentAsync` looks up all `RelatedDocument` relations where the changed content is the child, then evicts the cache tag for each parent it finds. This is why publishing a referenced item correctly clears the pages that embed it — without needing to know their URLs in advance.

Using Razor views instead? If your project renders HTML server-side through Razor views, Umbraco 17 also ships a website output cache built on ASP.NET Core output caching[^01-output]. The idea is the same — store ready-made responses on the server — but it stores HTML rather than JSON. The rest of this book assumes headless-first, so that path is not covered in depth.

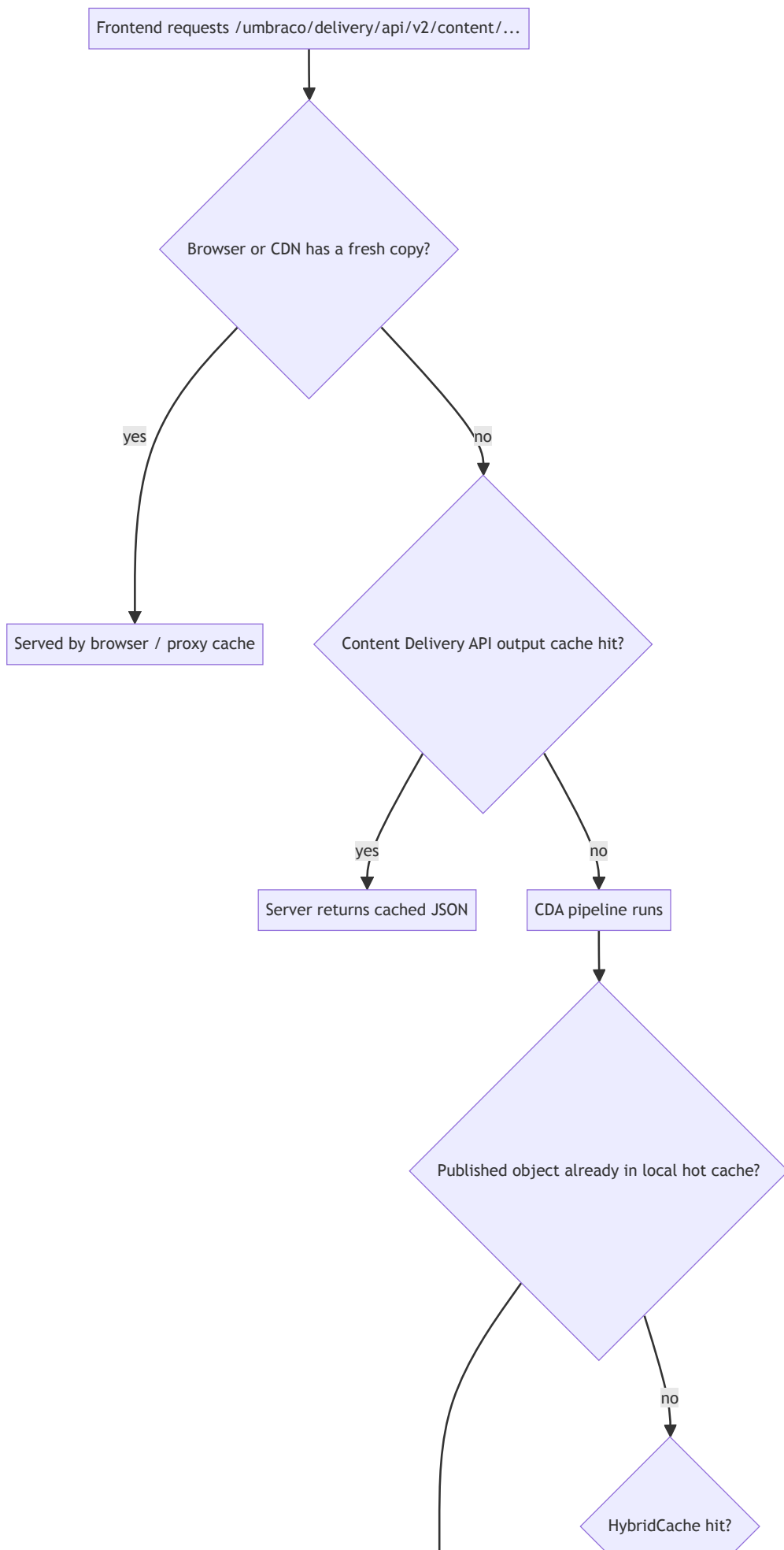
3. Browser and proxy cache — the takeaway shelf

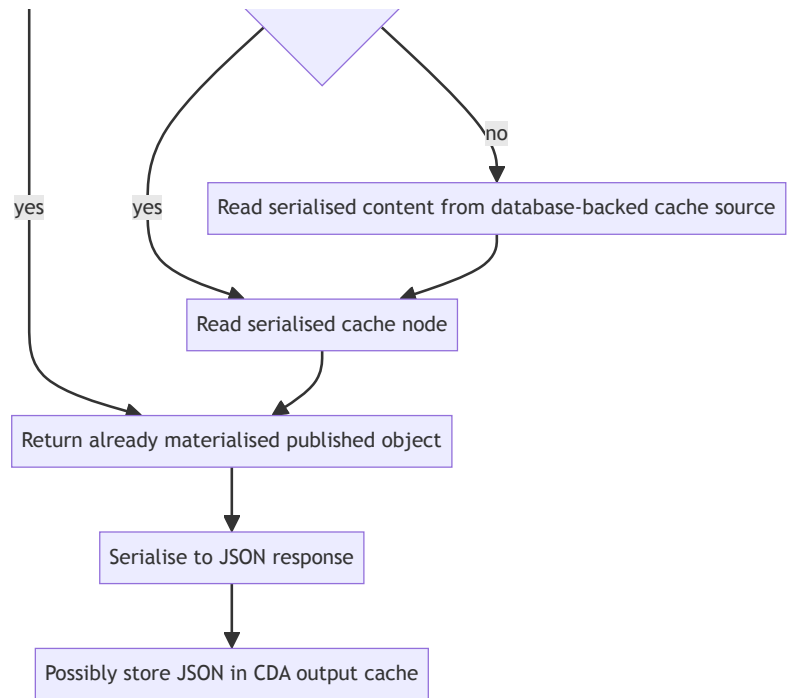
This is the bag waiting on the shelf by the door: ordinary HTTP caching, nothing Umbraco-specific, just the web platform doing its job. The customer grabs it and leaves without the kitchen lifting a finger.

- Controlled with `Cache-Control` headers
- Described in the Umbraco docs as "Response Caching"[^01-response]
- Great for static assets, and sometimes useful for API responses with stable content
- Worth remembering: a cached response at this layer does not stop the server from doing work if the request still makes it through

Mental model

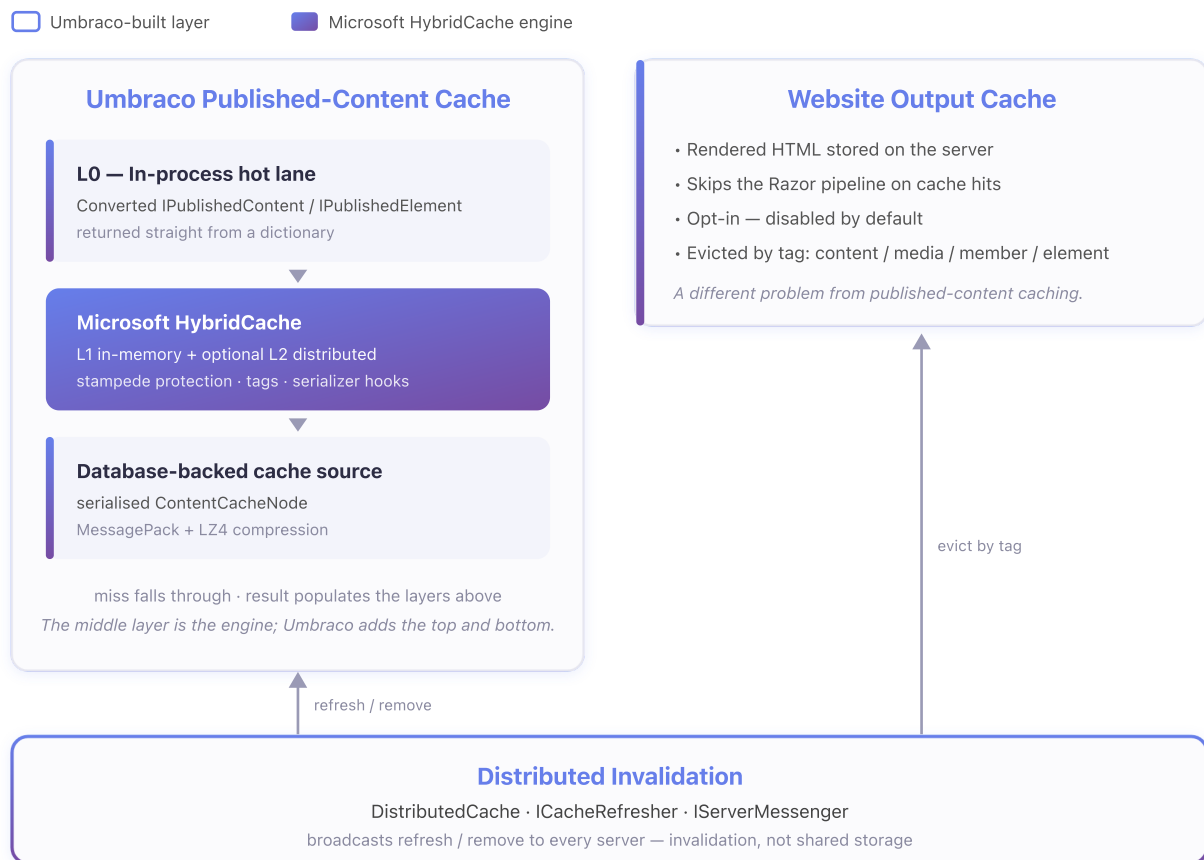
Put all three layers together and a headless request looks like this:





How the cache families relate

The diagram above follows a single request from top to bottom. This next diagram steps back and shows how the main cache families sit *alongside* each other — what Microsoft provides, what Umbraco builds on top, and where invalidation crosses the boundaries.



What changed in Umbraco 17, and why it matters

In older versions, people often talked about "NuCache" as the main published-content cache. That name has mostly faded, but it still appears in configuration keys and settings — which can be confusing if you upgrade an existing project.

In Umbraco 17:

- the published cache is now built on Microsoft's `HybridCache`
- old configuration names still appear under `Umbraco:CMS:NuCache` for backward compatibility[^01-nucache]
- output caching for API and website responses became a first-class documented feature

One thing worth clearing up straight away: those `NuCache` config names are *just names*. The NuCache engine itself was retired in Umbraco 15, so v17 does **not** ship two cache engines for you to choose between. Anything still labelled `NuCache` — settings, serialiser options, SQL templates — is quietly feeding Hybrid Cache under the hood. [Chapter 12](#) walks through this in detail if you want the full story.

Looking ahead beyond v17, the documentation also reveals where the platform is heading next:

- element cache settings are becoming first-class
- cache seeding is described for documents, media, and elements
- cache entry settings are documented for documents, media, and elements

A useful way to frame the whole topic for a headless project:

1. "How do we cache Content Delivery API responses?"
2. "How does Umbraco cache published content internally?"
3. "How does invalidation stay correct across multiple servers?"

Each of those questions has its own answer, and they are mostly independent of each other.

Quick glossary

These terms come up constantly. Getting them straight early saves a lot of confusion later.

Microsoft cache stack

Umbraco is built on top of .NET's own caching primitives. The three you will encounter most often are:

- `IMemoryCache` — plain in-process memory caching
- `IDistributedCache` — an abstraction for distributed cache storage (Redis, SQL, etc.)
- `HybridCache` — a higher-level API that combines a fast local memory layer with an optional distributed second level[^01-msstack]

Why does this matter? Umbraco's published-content cache is built on `HybridCache`. Once you know that `HybridCache` is already sitting on top of those lower primitives, the rest of Umbraco's cache architecture becomes much easier to picture.

`AppCaches`

Umbraco's own helper object for general application caching. You will reach for this when you need to cache custom data in your own code.

It exposes three caches:

- `RuntimeCache` — lives for the lifetime of the application
- `RequestCache` — lives for a single HTTP request
- `IsolatedCaches` — per-entity caches, one per entity type

`DistributedCache`

This one trips people up, so it is worth being explicit: Umbraco's `DistributedCache` is **not** about storing content in Redis. It is an invalidation messenger.

What it actually does:

- Umbraco broadcasts refresh and remove instructions across all servers
- every server receives those instructions
- each server clears or refreshes its own local caches in response

This is related to, but meaningfully different from, Microsoft's `IDistributedCache`.

- Microsoft `IDistributedCache` is a **storage** abstraction — it holds data
- Umbraco `DistributedCache` is an **invalidation** abstraction — it coordinates what to forget[^01-distributed]

That distinction is one of the most common sources of confusion for developers new to Umbraco. When someone says "set up distributed cache," they could mean either thing, and they are very different tasks.

`ICacheRefresher`

The standard Umbraco contract for telling all servers:

"This thing changed — clear or refresh the matching cache entries."

In kitchen terms, this is the head chef calling "86 the salmon!" — telling every station to stop serving a dish the instant it is no longer good. You will see implementations of this throughout the codebase, one per entity type (content, media, members, and so on).

Things to keep in mind

A few rules of thumb that will save you from the most common misunderstandings:

- To speed up Content Delivery API responses, enable the CDA output cache — that is the right lever.
- To cache your own custom app or service data, reach for `AppCaches`.
- If your data can change from backoffice actions and you run more than one server, learn about `ICacheRefresher` and `DistributedCache` — you will need them.
- "Response caching" and "output caching" sound similar but are different things. Do not conflate them.
- "Distributed cache" in Umbraco usually means distributed *invalidation*, not distributed *storage*. Check which one the conversation is about.
- The HTML output cache (for Razor pages) and the CDA output cache are separate features. Enabling one does nothing to the other.

What is coming in Umbraco 18

The biggest cache-related addition in the Umbraco 18 codebase is first-class element cache handling:

- `ElementCacheRefresherNotification`
- `ElementCacheService`
- `WebsiteElementOutputCacheEvictionHandler`

The practical difference this makes:

- in v17, publishing content clears element caches broadly because individual elements are hard to target precisely
- in v18, element caching and element-driven output-cache eviction become explicit and surgical — the same tag-based precision that the CDA cache already uses

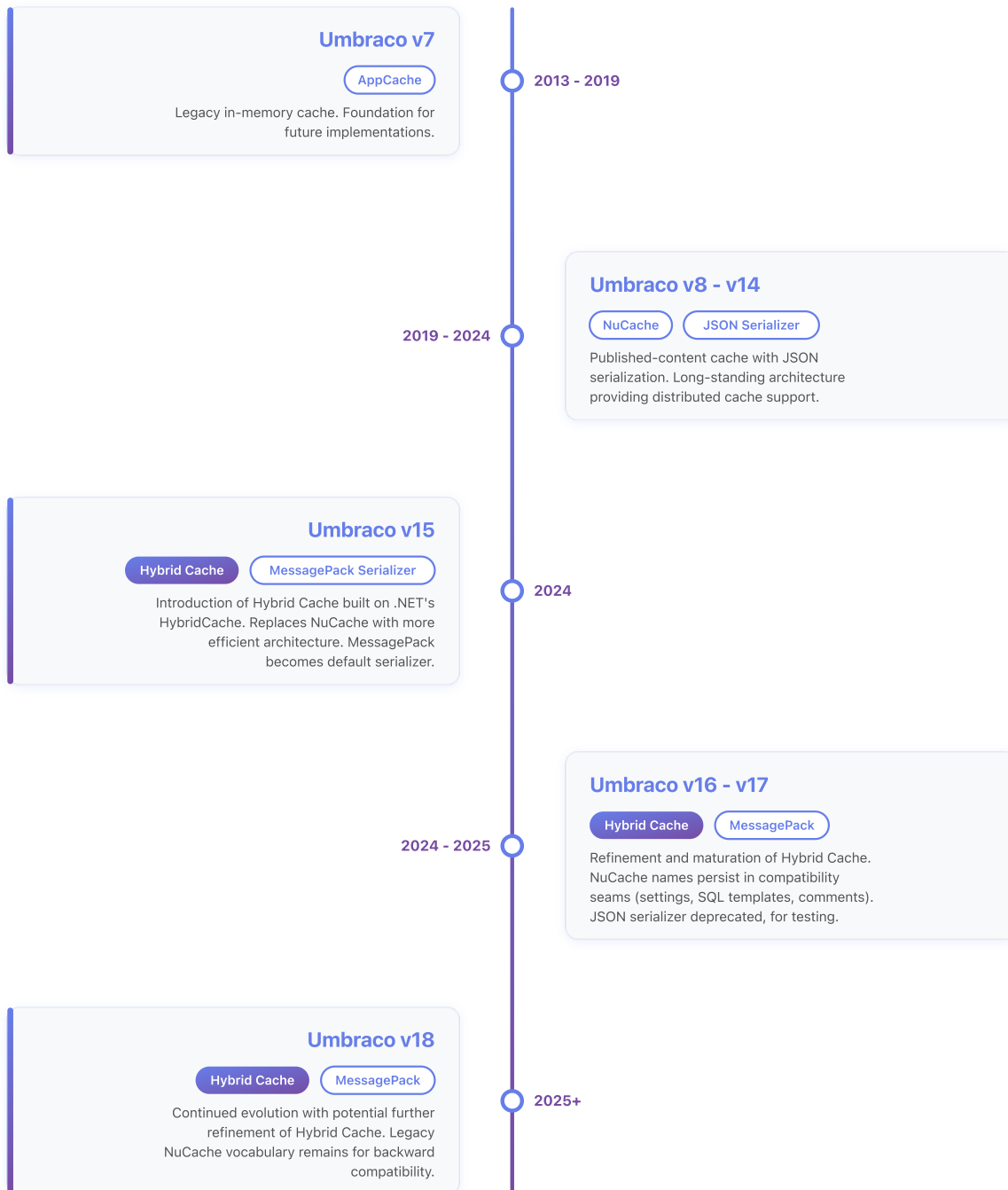
This should make the cache story considerably easier to reason about for projects that use block-based content heavily.

A brief history of Umbraco caching

If a timeline helps, here it is in one line per era:

- **Umbraco 8–14:** Published content cache was "NuCache" — load almost everything, think NuCache.
- **Umbraco 15–17:** The NuCache *engine* is gone. HybridCache takes over, with a local hot lane, smart seeding, and careful invalidation.
- **Umbraco 18 onwards:** Elements become first-class in both cache storage and cache busting.

Cache Architecture Evolution (v7 - v18)



For more detail on the differences between NuCache and Hybrid Cache, see [Chapter 12 - NuCache vs Hybrid Cache](#).

[^01-response]: See [U2 in the appendix](#). [^01-output]: See [U3](#) and [C6](#) in the appendix. [^01-hybrid]: See [M2](#), [C1](#), and [C4](#) in the appendix. [^01-msstack]: See [M1](#) and [M2](#) in the appendix. [^01-distributed]: See [C7](#) and [M1](#) in the appendix. [^01-nucache]: See [12 - NuCache vs Hybrid Cache](#) and [C1](#) in the appendix.

Sources

- Docs:
 - [Caching overview \(v17\)](#)
 - [Server-side cache extensions \(v17\)](#)
 - [Application cache \(v17\)](#)
 - [Cache settings \(v17\)](#)
 - [Cache settings \(latest\)](#)
- Code:
 - `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/DependencyInjection/UmbracoBuilderExtensions.cs`

- `umbraco-v17/src/Umbraco.Core/Cache/AppCaches.cs`
- `umbraco-v17/src/Umbraco.Core/Cache/DistributedCache.cs`
- `umbraco-v17/src/Umbraco.Cms.Api.Delivery/Caching/DeliveryApiOutputCachePolicyBase.cs`
- `umbraco-v17/src/Umbraco.Cms.Api.Delivery/Caching/DeliveryApiDocumentOutputCacheEvictionHandler.cs`
- `umbraco-v17/src/Umbraco.Web.Common/Caching/RelationOutputCacheEvictionHandlerBase.cs`
- `umbraco-v17/src/Umbraco.Core/Constants-DeliveryApi.cs`
- `umbraco-v18/src/Umbraco.PublishedCache.HybridCache/Services/ElementCacheService.cs`
- `umbraco-v18/src/Umbraco.Web.Website/Caching/WebsiteElementOutputCacheEvictionHandler.cs`

02. Website Output Caching

This is the cache most people mean when they say:

"Can Umbraco avoid rerendering the whole page every time?"

In Umbraco 17, the answer is yes. [^02-output]

Plain-English definition

Website output caching stores the final rendered HTML on the server.

So on the next request:

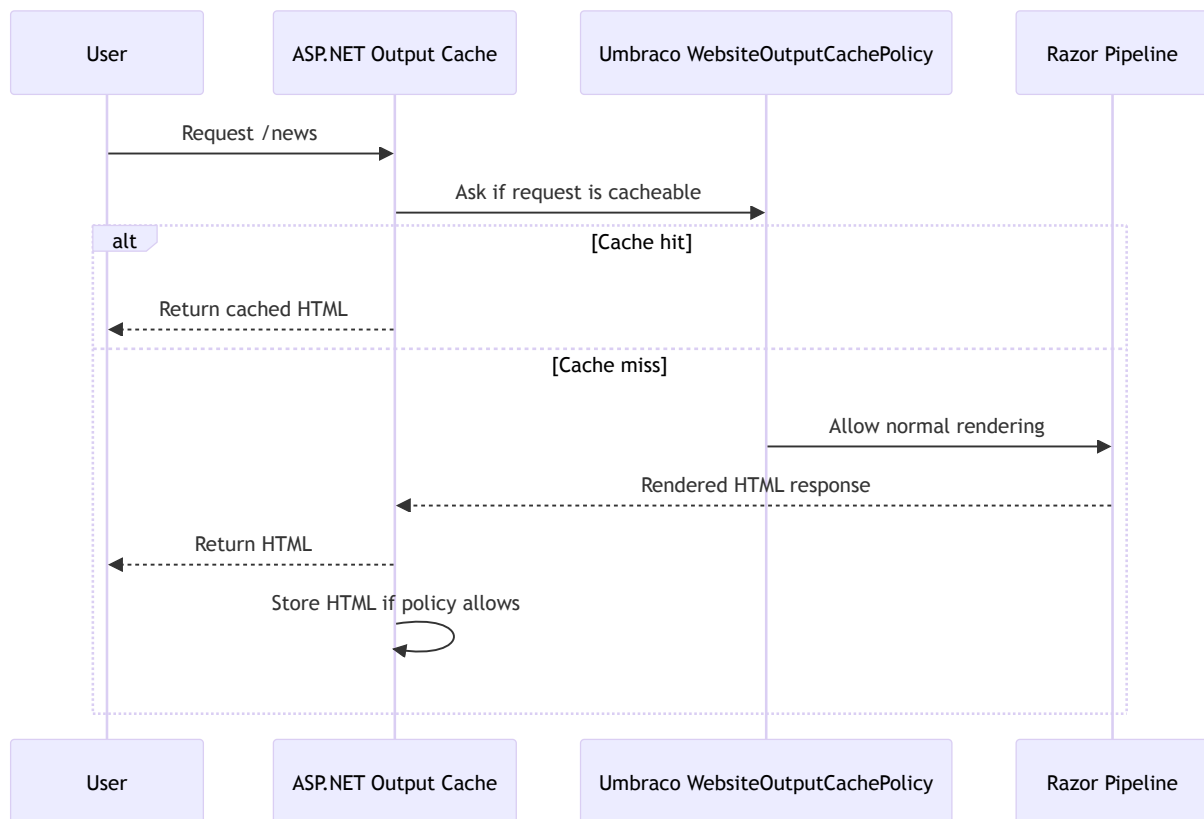
- Umbraco does not run the full Razor rendering pipeline again
- your controllers, views, and value converters do less work
- the user gets the response faster

It is not enabled by default

You must turn it on in `appsettings.json` :

```
{
  "Umbraco": {
    "CMS": {
      "Website": {
        "OutputCache": {
          "Enabled": true,
          "ContentDuration": "00:00:30"
        }
      }
    }
  }
}
```

How the request flows



What makes a request cacheable in v17

The main decision logic lives in `WebsiteOutputCachePolicy` plus `DefaultWebsiteOutputCacheRequestFilter`.

By default, Umbraco refuses to output-cache: [^02-rules]

- preview requests
- authenticated requests
- requests where `PublishedRequest.SetNoCacheHeader` is set
- responses with `Set-Cookie`
- responses marked `Cache-Control: no-store`

That last point matters a lot:

- pages using `@Html.AntiForgeryToken()` usually become non-cacheable
- forms often need special care

Default tags Umbraco adds

When Umbraco stores a page in the website output cache, it tags the cached entry with:

- the page's own content key
- the keys of all ancestors
- the "all content" tag
- custom tags from registered `IWebsiteOutputCacheTagProvider` implementations

The docs also explain the intended model in broader terms:

- content key
- ancestor keys
- content type alias
- relation-driven tags for referenced items

In the code, content type tagging is provided by a registered tag provider, not hardcoded directly into the policy.

Why tags matter

Tags are the trick that makes eviction precise.

Example:

- cache `/products/shoes/nike-air`
- tag it with its own key
- tag it with ancestors like `products` and `shoes`
- later, if `shoes` moves or refreshes as a branch, Umbraco evicts descendants by ancestor tag

Eviction after content changes

This is the most important part.

Creating a cache is easy.

Creating one that disappears at the right moment is the hard part, and this is where Umbraco is doing most of its real work.

The eviction handler for website pages in v17 is `WebsiteDocumentOutputCacheEvictionHandler`.

It listens for `ContentCacheRefresherNotification`.

Then it can:

- evict one page by content key
- evict descendants by ancestor tag on branch refresh
- evict everything on `RefreshAll`
- evict related pages through Umbraco relations
- run custom `IWebsiteOutputCacheEvictionProvider` logic

So the website output cache is not mainly a "storage feature".

It is really a combination of:

- ASP.NET Core output-cache storage
- Umbraco cacheability rules
- Umbraco tagging
- Umbraco invalidation handlers

The relations part is easy to miss

This is one of the nicest features.

If page A references media item B through a picker relation, and B changes, Umbraco can evict page A's cached HTML too.

In other words:

- the page itself did not change
- but something it depends on changed
- so Umbraco still invalidates the page cache



Load balancing story

By default the output cache store is in-process memory on each server.

That means:

- each server has its own HTML cache
- a content publish triggers notifications
- each server evicts matching entries locally

This is distributed invalidation, not shared storage.

If you want shared storage, the docs recommend replacing the default store with a Redis-backed `IOutputCacheStore`.

The non-obvious busting triggers

Besides content publish/unpublish/move/delete, website output cache can also be busted when:

- media changes and pages reference that media
- members change and pages reference those members
- custom eviction providers return extra tags to evict
- a global content refresh happens

That means the busted page is often not the thing that changed directly.

It might be a page that depends on the changed thing.

Extension points you should know

`IWebsiteOutputCacheRequestFilter`

Decides whether a request should be cacheable.

Use this when you want to skip cache for a specific content type or loosen a default rule.

`IWebsiteOutputCacheDurationProvider`

Lets you vary cache duration per content item.

`IWebsiteOutputCacheTagProvider`

Adds extra tags to entries.

`IWebsiteOutputCacheEvictionProvider`

Lets you define extra tags to evict when content changes.

`IWebsiteOutputCacheVaryByProvider`

Controls what creates separate cache entries, for example:

- query keys
- custom cookies
- culture markers

Debugging tips

The docs recommend enabling debug logging for:

```
"Umbraco.Cms.Web.Website.Caching": "Debug"
```

This helps answer:

- why was this request skipped?
- which content key got cached?
- how many tags were attached?

Also check the `Age` response header on cached responses.

Good beginner use cases

- content-heavy public pages
- landing pages
- article pages

- catalogue pages without per-user server-side personalisation

Bad beginner use cases

- pages with member-specific HTML rendered on the server
- pages that must show publish changes instantly with zero delay
- form-heavy pages that set anti-forgery or cookies

What changes in 18

In Umbraco 18, website output caching gains explicit element-based eviction support via `WebsiteElementOutputCacheEvictionHandler`.

That is a clue that block and element dependencies are becoming more first-class in the cache invalidation model.

Sources

- Docs:
 - [Website output caching \(v17\)](#)
 - [Caching overview \(v17\)](#)
- Code:
 - `umbraco-v17/src/Umbraco.Web.Website/DependencyInjection/UmbracoBuilderExtensions.cs`
 - `umbraco-v17/src/Umbraco.Web.Website/Caching/WebsiteOutputCachePolicy.cs`
 - `umbraco-v17/src/Umbraco.Web.Website/Caching/DefaultWebsiteOutputCacheRequestFilter.cs`
 - `umbraco-v17/src/Umbraco.Web.Website/Caching/WebsiteDocumentOutputCacheEvictionHandler.cs`
 - `umbraco-v17/src/Umbraco.Web.Website/Caching/RelationOutputCacheEvictionHandlerBase.cs`
 - `umbraco-v18/src/Umbraco.Web.Website/Caching/WebsiteElementOutputCacheEvictionHandler.cs`

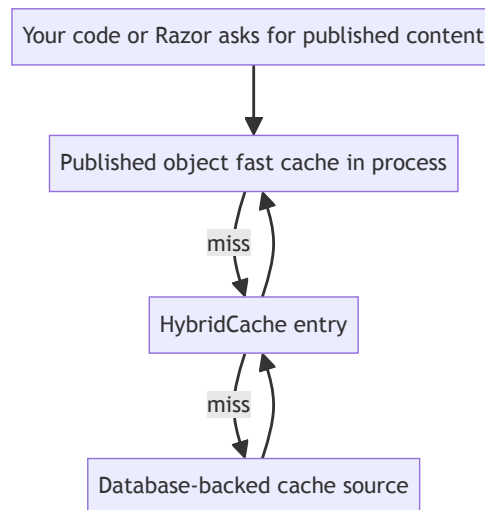
[^02-output]: See [U3 in the appendix](#) and [C6](#). [^02-rules]: See [U3](#) and [C6](#).

03. Published Content Cache, AppCaches, and Load Balancing

This chapter is about the part people usually find fuzzy:

"When Umbraco loads content, where is it really cached, and how do multiple servers stay in sync?"

First: separate three layers in your head



Layer 1. Materialized published objects

In `DocumentCacheService`, Umbraco keeps a local concurrent dictionary of already built `IPublishedContent` objects.^[^03-10]

That is the fastest path.

In the newer `main` branch architecture, this pattern also exists explicitly for elements, not only documents.

Layer 2. `HybridCache`

If the materialised object is not already there, Umbraco tries Microsoft's `HybridCache`.^[^03-hybrid]

This can use:

- local memory
- and optionally a second-level remote cache

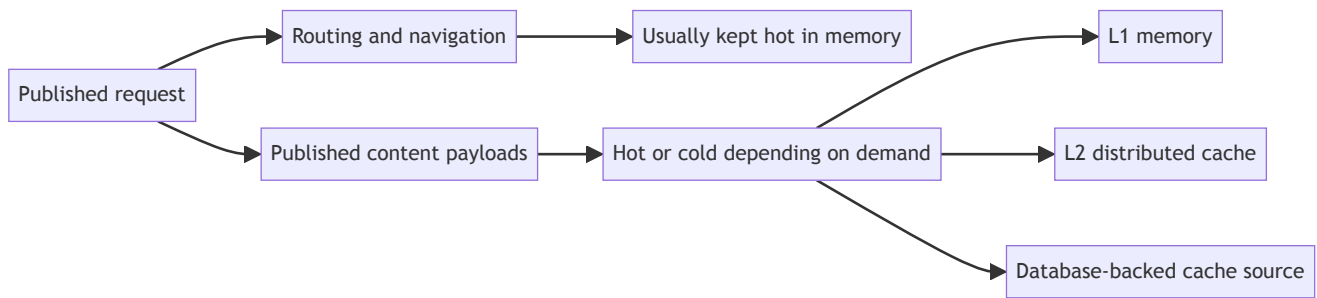
Layer 3. Database-backed cache source

If the `HybridCache` misses, Umbraco falls back to its database-backed cache repository and then repopulates the cache.^[^03-db]

So even though old blog posts still say "NuCache", in v17 the living implementation is now centred around `HybridCache`.

The "split the problem" model

One of the most helpful ideas from the `HybridCache` presentation is that Umbraco no longer treats all published concerns the same way.



What Umbraco registers in v17

From `AddUmbracoHybridCache()` in the source, Umbraco registers:

- `IPublishedContentCache`
- `IPublishedMediaCache`
- `IPublishedMemberCache`
- `IDomainCache`
- `IElementsCache`
- services like `DocumentCacheService`, `MediaCacheService`, `MemberCacheService`

It also sets `HybridCacheOptions.MaximumPayloadBytes` to 100 MB by default because the Microsoft default of 1 MB is too easy to hit for real Umbraco content.^[^03-payload]

That detail is important for multilingual and block-heavy sites.

It also lines up with the HybridCache talk and slide material we looked at:

- Microsoft defaults are too small for real Umbraco payloads
- block-heavy and multilingual sites are exactly where cache payload size starts to matter

Why there is still `AppCaches`

`AppCaches` is still the standard helper for your own application caching.

It gives you:

- `RuntimeCache`
- `RequestCache`
- `IsolatedCaches`

Use `AppCaches` when you want to cache your own service/repository results.

Do not confuse that with the internal published-content cache machinery.

Friendly rule

- Use website output caching for rendered HTML
- use published cache APIs for published content
- use `AppCaches` for your own app-level cache entries

Cache seeding

Umbraco 17 uses lazy loading by default:

- first request for content may pay the cache-fill cost
- later requests are faster

Cache seeding prewarms selected keys on startup.

The built-in seed providers include:

- content types configured in settings

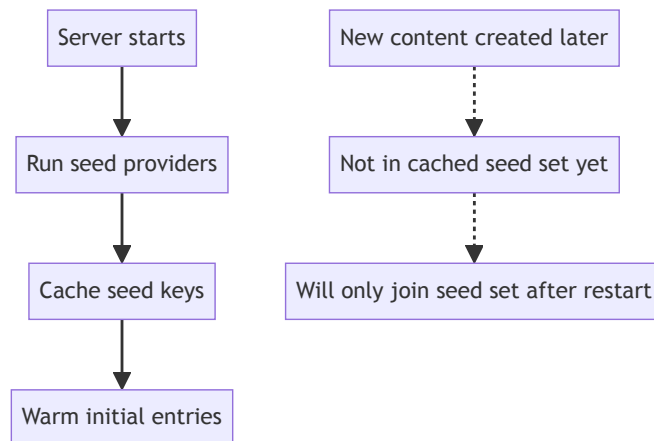
- breadth-first traversal of document tree
- breadth-first traversal of media tree

In newer docs and code direction, element seeding joins that list too.

The union of seed keys is cached at startup.

Important consequence:

- if your seed logic is dynamic, new matching content is not considered seeded until restart



How a document lookup works in v17

The `DocumentCacheService` logic is roughly:

1. decide preview or published mode
2. check the local in-process published-content dictionary
3. check `HybridCache`
4. if missing, load from database cache repository
5. if safe, store result back in `HybridCache`
6. materialise `IPublishedContent`
7. store that materialised object in the local fast cache

In the future Hybrid Cache code on `main`, the same layered idea is even clearer:

- local converted-object cache first
- `HybridCache` second
- database-backed cache source third

That is one reason the future architecture deserves its own chapter.

One especially careful bit:

- if a content item exists but its published ancestor path currently fails validation, Umbraco avoids caching that null result in the distributed cache because it may be a transient rebuild race
- in newer code, there is also a cache-generation guard to stop stale in-flight reads from writing old data back over freshly refreshed entries

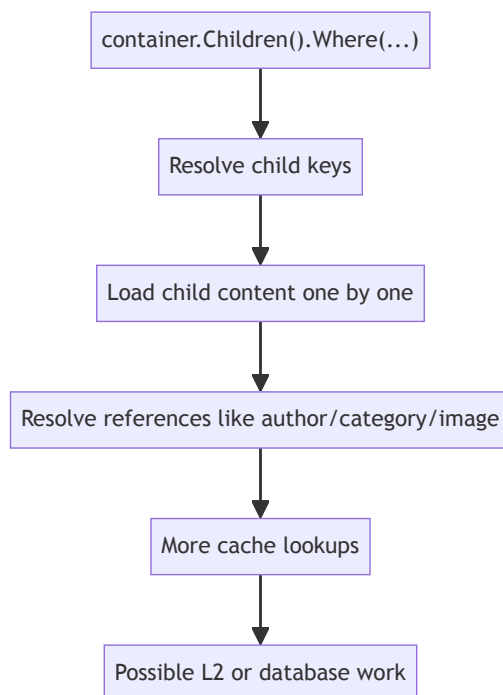
That is the kind of detail that explains why the implementation is more subtle than "just cache by key".

Why traversal feels different now

The deck makes this change very concrete: traversal and filtering patterns that felt cheap in the older world can become much more expensive in the `HybridCache` world.

That does not always mean "add another cache".

Sometimes it means "use an index instead of traversal", which is exactly where Examine fits. See [13 - Examine, Indexes, and Cache-Adjacent Querying](#) for the full comparison.



Invalidation after content changes

The main content refresher in v17 is `ContentCacheRefresher`.

It coordinates more than one thing:

- memory cache cleanup
- document cache updates
- routing/url updates
- navigation updates
- domain cache handling
- partial view cache clearing when appropriate
- distributed notification fan-out

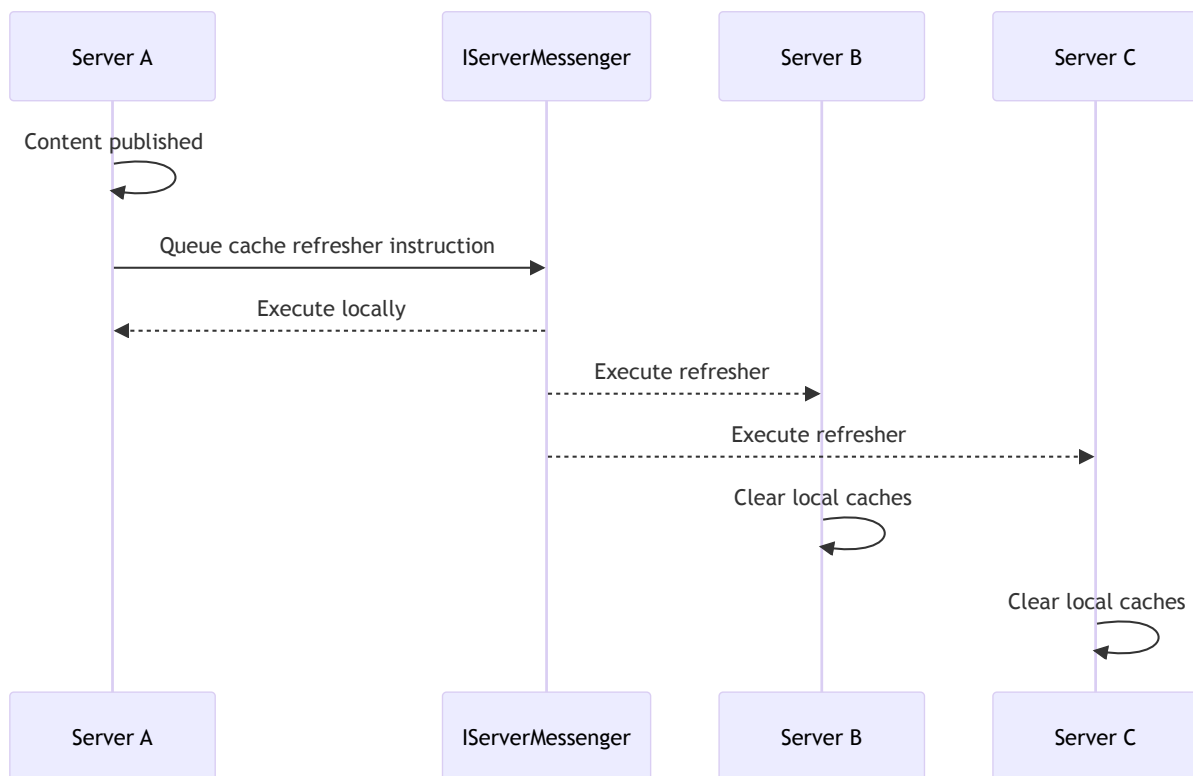
Load balancing: what actually happens

When cache refresh is triggered through `DistributedCache`, Umbraco tells every server:

"run this refresher"

That means:

- server A publishes content
- Umbraco sends refresh instructions
- servers B, C, and D also run the matching cache refresher
- each server clears or refreshes its own local caches



This is why the docs say:

- if you cache business data affected by backoffice actions
- and you are not using `ICacheRefresher`
- your load-balanced setup may become inconsistent

Why `ICacheRefresher` matters for custom code

If you build a package or custom service that caches database-backed business data, a single-server solution may look fine even with naive caching.

But in multi-server production:

- one server may see the edit
- the other servers may keep stale memory

`ICacheRefresher` plus `DistributedCache` fixes that by standardizing invalidation.

Partial view cache note

`ContentCacheRefresher` also clears the partial view cache in specific published-change scenarios.

That is a useful reminder:

- not every cache invalidation in Umbraco is about published-content keys only
- some refreshes are there to keep rendered fragments honest too

What changes in 18

Umbraco 18 makes element caching much more explicit:

- `ElementCacheService`
- `ElementCacheRefresherNotification`
- output-cache eviction for changed elements

Why that matters:

- in v17, element cache invalidation is broader because published elements are hard to address precisely
- in v18, the platform is moving towards more explicit element-level cache management

That is especially relevant for modern Umbraco builds using blocks heavily.

For the fuller future direction, see [10 - Future Hybrid Cache Architecture](#).

The practical performance trade-off

The HybridCache presentation material gives a useful warning for beginners:

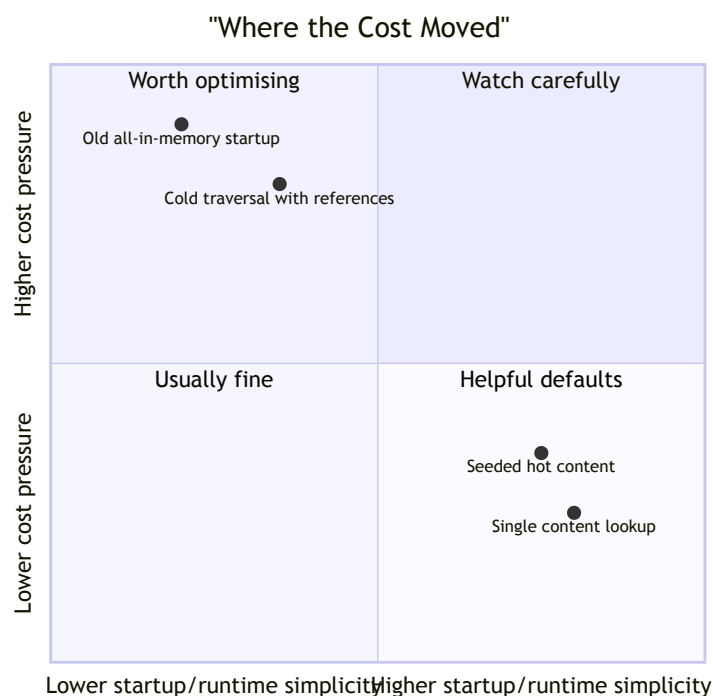
- lower memory usage and faster startup are wins
- but tree traversal and broad in-memory filtering patterns can become more expensive

So code like:

- `.Children().Where(...)`

deserves more suspicion in the new model than it did in the old "keep everything hot" mental model.

Cost moved, not vanished



Practical checklist

- For whole-page speedups, start with website output caching.
- For custom app/service caching, use `AppCaches`.
- For multi-server-safe custom invalidation, implement `ICacheRefresher`.
- For hot content, consider cache seeding.
- For huge multilingual or block-heavy payloads, remember the `HybridCache` payload size story.

Sources

- Docs:
 - [Server-side cache extensions \(v17\)](#)
 - [Application cache \(v17\)](#)
 - [Cache seeding \(v17\)](#)
 - [Cache settings \(v17\)](#)
 - [Cache settings \(latest\)](#)

- Supporting material:
 - [Hybrid Cache förändrar allt — Umbraco Kalaset session \(YouTube\)](#)
 - [Hybrid Cache förändrar allt — Umbraco Kalaset slides \(PDF\)](#)
- Code:
 - `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/DependencyInjection/UmbracoBuilderExtensions.cs`
 - `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/Services/DocumentCacheService.cs`
 - `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/NotificationHandlers/CacheRefreshingNotificationHandler.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/AppCaches.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/DistributedCache.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/Refreshers/Implement/ContentCacheRefresher.cs`
 - `umbraco-v18/src/Umbraco.PublishedCache.HybridCache/Services/ElementCacheService.cs`

[^03-l0]: See [C1](#) and [C4](#) in the appendix. [^03-hybrid]: See [M2](#) and [C1](#) in the appendix. [^03-db]: See [C4](#) and [C5](#) in the appendix. [^03-payload]: See [M6](#), [U4](#), and [T1](#) in the appendix.

04. Cache Busting and Invalidation

If cache creation is the easy half, invalidation is the half that decides whether your site is trustworthy.

This chapter is the real centre of the Umbraco caching story.

The main idea

When something changes in Umbraco, the system usually does not ask:

"How do I build cache?"

It asks:

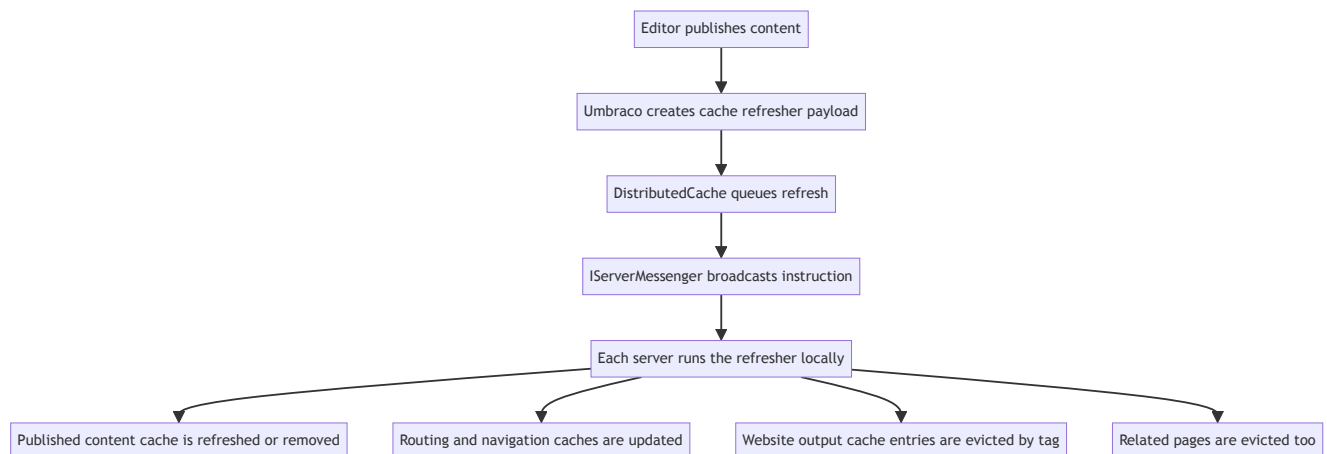
"Which layers are now stale, and how do I tell every server to stop trusting them?"

The short version

In Umbraco 17, cache busting is mostly driven by: [^04-core]

- `ContentCacheRefresher`
- `MediaCacheRefresher`
- `MemberCacheRefresher`
- `DistributedCache`
- `IServerMessenger`
- output-cache eviction handlers listening for refresher notifications

The core mental model



What `ContentCacheRefresher` actually busts

This class is much broader than its name first suggests. [^04-refresher]

For content changes, it coordinates all of these:

- document memory cache refresh/removal
- branch refresh/removal for descendants
- full document cache clear on `RefreshAll`
- URL cache updates
- URL alias cache updates
- navigation tree updates

- publish status updates
- ID/key map cleanup on delete
- partial view cache clearing in relevant published-change scenarios
- domain cache refresh when deleted content had assigned domains

So if someone says "publishing busts the cache", the honest version is:

publishing busts several related caches, not just one

Refresh types matter

Umbraco uses tree-oriented change types.

The important ones are:

- RefreshNode
- RefreshBranch
- RefreshAll
- Remove

These do not mean the same thing at all.

RefreshNode

Think:

- one item changed
- refresh that item's cache entry
- update routing for that item

RefreshBranch

Think:

- one item and descendants are affected
- maybe a move
- maybe branch publish/unpublish
- maybe restore/trash flow

This is where Umbraco often refreshes or removes a whole branch of document cache entries.

Remove

Think:

- item should no longer exist as a cacheable published thing
- remove it from caches
- clear routing before navigation loses the tree shape

RefreshAll

Think:

- stop trying to be precise
- rebuild or clear broad layers

How document cache is busted

Inside `ContentCacheRefresher.HandleMemoryCache()` :

- `RefreshNode` calls `RefreshMemoryCacheAsync(key)`
- `RefreshBranch` refreshes or removes the node plus descendants
- `RefreshAll` calls `ClearMemoryCacheAsync()`

- `Remove` calls `RemoveFromMemoryCacheAsync(key)`

The important branch rule is this:

- if a branch is in the bin, or fully unpublished, Umbraco removes those entries
- otherwise it refreshes them

That is smarter than just nuking everything.

What refresh vs remove means in published cache terms

In `DocumentCacheService` :

- refresh means re-read current serialised state and write it back into `HybridCache`
- remove means delete the relevant cache entry and local materialised object

That distinction matters because:

- some changes still leave the content as a valid published thing
- other changes mean it should disappear from published resolution

Output cache busting is tag-based

Website output cache invalidation is separate from published object cache invalidation.

It works through tags.

Document changes

`WebsiteDocumentOutputCacheEvictionHandler` can evict:

- one page by content-key tag
- descendants by ancestor tag during branch refresh
- all pages on `RefreshAll`
- related pages through relations
- extra custom tags from `IWebsiteOutputCacheEvictionProvider`

Media changes

`WebsiteMediaOutputCacheEvictionHandler` evicts pages that reference the changed media.

Member changes

`WebsiteMemberOutputCacheEvictionHandler` evicts pages that reference the changed member.

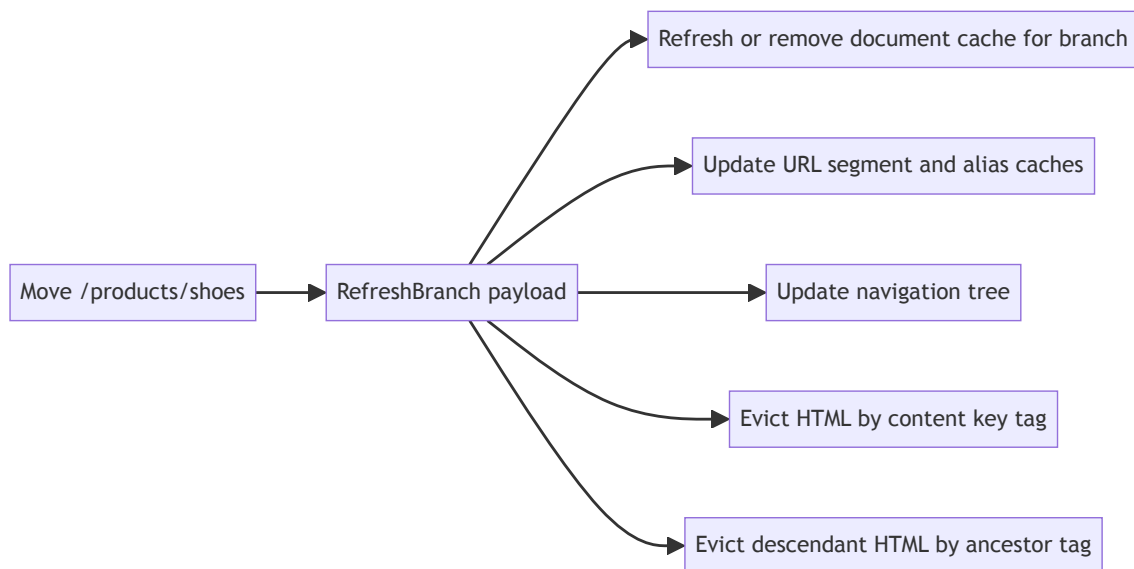
This is a big deal because many stale-page bugs are dependency bugs, not page-key bugs.

Example: one publish, many busts

Imagine `/products/shoes` is moved under `/catalogue/shoes` .

What may need busting?

- the page's own published content entry
- descendants' published content entries
- URL caches for the moved branch
- navigation tree entries
- cached HTML for the moved page
- cached HTML for descendants
- cached HTML for other pages that reference moved content



Why distributed invalidation matters more than distributed storage

This is the key production lesson.

On a multi-server setup, each server can have:

- its own in-memory output cache
- its own local materialised published objects
- its own local runtime cache entries

So the important question is:

"How do all servers learn that content changed?"

That is why `DistributedCache` and `IServerMessenger` are so central.

They ensure:

- the refresh instruction is broadcast
- each server runs the refresher locally
- each server clears its own stale state

This is why cache busting is more important than cache creation.

Why partial view cache clearing appears here

`ContentCacheRefresher` also clears partial view cache for relevant published changes.

That is Umbraco being conservative in the right places.

If a partial depends on changed published content, leaving its fragment cache behind would produce confusing stale output even if the underlying content cache is fresh.

When precision is hard

One revealing comment in the v17 code explains that element cache invalidation is difficult because published elements live inside JSON property blobs.

That means Umbraco cannot always target element-level impact perfectly in v17.

So in some areas it clears broadly instead.

This is one reason the v18 element-cache work matters.

What changes in 18

In Umbraco 18, the cache-busting model becomes more explicit for elements:

- `ElementCacheRefresherNotification`
- `ElementCacheService`
- `WebsiteElementOutputCacheEvictionHandler`

Practical takeaway:

- v17 sometimes has to invalidate broadly around element changes
- v18 is clearly moving towards more direct element-aware invalidation

Publish vs Deploy vs output-cache eviction

These three busting paths share one spine — the cache-refresher notification — but they are triggered and shaped differently. It helps to see them side by side.

Path	What triggers it	What carries the signal	What is distinctive
Normal publish	An editor publishes, saves, moves, or deletes content	<code>ContentCacheRefresher</code> → <code>DistributedCache</code> → <code>IServerMessenger</code>	Refreshes or removes published entries, routing, and output tags according to the change type (<code>RefreshNode</code> / <code>RefreshBranch</code> / <code>Remove</code> / <code>RefreshAll</code>)
Deploy	Schema or content transfer, import, or restore	The same cache-refresher notifications, emitted after the operation	Can suppress the normal flood of notifications during bulk work (<code>SuppressCacheRefresherNotifications</code>), then signal once; some disk-read operations need a broader memory-cache reload afterwards
Output-cache eviction	Any of the above, observed downstream	<code>WebsiteDocumentOutputCacheEvictionHandler</code> and friends listening for refresher notifications	Tag-based eviction of rendered HTML; dependency-aware (media, members, relations), not just page-key based

The key insight: a normal publish drives the refresher pipeline directly, Deploy deliberately batches and suppresses notifications before signalling once, and output-cache eviction is a listener sitting on the same notifications rather than a separate mechanism.

For the Deploy specifics, see [Chapter 5](#) and [Chapter 8](#). For output caching itself, see [Chapter 2](#).

Beginner checklist for custom code

- If you cache custom business data, think first about how it is busted.
- If backoffice actions can change that data, do not rely on local memory only.
- In load-balanced setups, prefer `ICacheRefresher`-driven invalidation.
- If a page depends on external or related data, think in dependency graphs, not only in single keys.
- If you use output caching, design your eviction path before celebrating cache hits.

Best one-sentence summary

Umbraco caching works because it is aggressive about invalidation choreography, not because it simply stores things quickly.

Sources

- Docs:
 - [Server-side cache extensions \(v17\)](#)
 - [Website output caching \(v17\)](#)
- Code:
 - `umbraco-v17/src/Umbraco.Core/Cache/Refreshers/Implement/ContentCacheRefresher.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/DistributedCache.cs`

- `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/Services/DocumentCacheService.cs`
- `umbraco-v17/src/Umbraco.Web.Website/Caching/WebsiteDocumentOutputCacheEvictionHandler.cs`
- `umbraco-v17/src/Umbraco.Web.Website/Caching/WebsiteMediaOutputCacheEvictionHandler.cs`
- `umbraco-v17/src/Umbraco.Web.Website/Caching/WebsiteMemberOutputCacheEvictionHandler.cs`
- `umbraco-v18/src/Umbraco.Web.Website/Caching/WebsiteElementOutputCacheEvictionHandler.cs`

[^04-core]: See [C7 in the appendix](#) and [U6](#). [^04-refresher]: See [C7](#) and [C6](#).

05. Extensions and Deploy Cache Busting

If you build Umbraco extensions, packages, or custom event handlers, this chapter matters a lot.

Deploy changes the normal cache-busting story in a very deliberate way.

The main surprise

During a normal CMS operation, your code may rely on notifications like:

- `ContentPublishedNotification`
- `ContentSavedNotification`
- other per-item save or publish hooks

But during a Deploy operation, many of those normal CMS notifications are suppressed.^[^05-deploy]

That means:

- the content still changes
- caches still need updating
- but your usual extension hooks may not run

What the decompiled OnPrem package proves

Looking at the decompiled `Umbraco.Deploy.OnPrem` package adds one very useful detail:^[^05-decompiled]

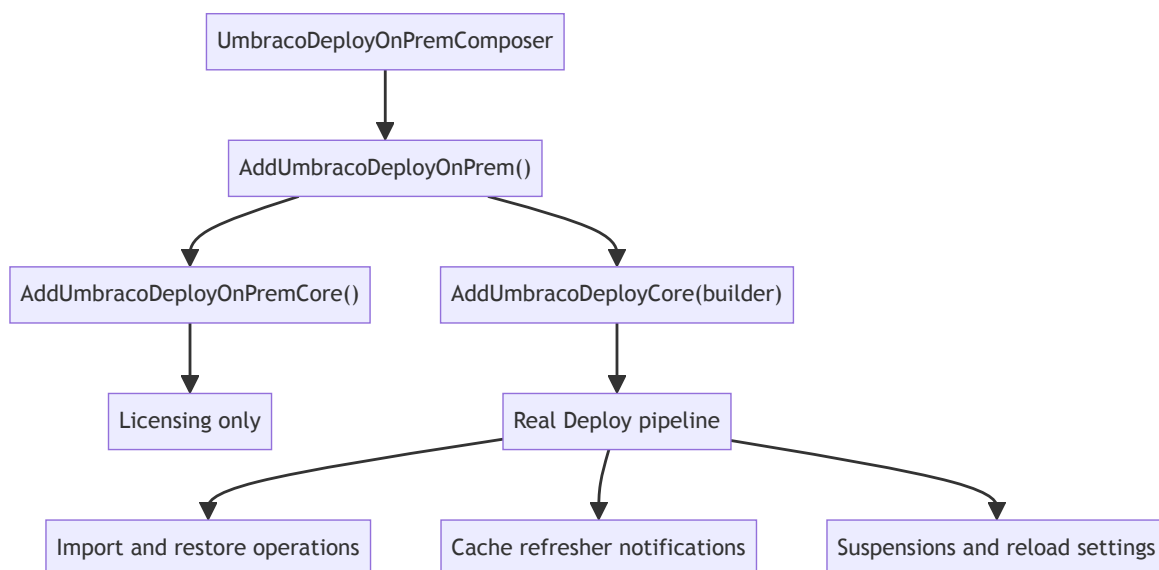
- `UmbracoDeployOnPremComposer.Compose()` only calls `builder.AddUmbracoDeployOnPrem()`
- `AddUmbracoDeployOnPrem()` checks whether Deploy core is already registered
- if not, it calls `Umbraco.Deploy.Infrastructure.Extensions.UmbracoBuilderExtensions.AddUmbracoDeployCore(builder)`
- after that, it adds a small OnPrem-specific layer for licensing

That means the OnPrem package itself is not where the interesting cache behaviour lives.

The important cache work is deeper down in:

- `Umbraco.Deploy.Infrastructure`
- the CMS cache refresher pipeline
- the Deploy settings that decide whether refreshers are fired, suspended, or followed by a forced reload

So if you only decompile `Umbraco.Deploy.OnPrem.dll`, you mostly see the front door, not the machinery room.



Where Deploy uses cache

Deploy interacts with cache in a few different ways, and these are easy to mix up.

1. It relies on Umbraco's normal cache refresh layer

After a deploy operation, Deploy can fire Umbraco cache refresher notifications so that:

- in-memory published data is refreshed
- other local caches can respond
- distributed environments can stay coherent
- Examine indexes can catch up

This is the main "uses cache" story.

2. It can suspend cache-related systems during bulk work

Deploy supports suspension options for operations such as import and restore.

The docs mention suspension targets including: [^05-suspensions]

- DocumentCache
- Examine
- Signatures

This means Deploy may deliberately reduce cache and index churn while it is mid-flight, then let the refresh happen at the operation boundary.

3. It can force a memory-cache reload after some disk-read operations

The `ReloadMemoryCacheFollowingDiskReadOperation` setting exists because some deployments can leave the CMS in a state where a reload is the safest fix.

That is not everyday behaviour, but it is still very important architecturally:

- Deploy does not only add data
- it sometimes has to repair cache state explicitly

Why Deploy does this

According to the Deploy docs, this behaviour is deliberate for:

- performance
- reliability
- avoiding inconsistent data

That makes sense because Deploy may process many items in one operation, not one editor action at a time.

So the question for extension authors becomes:

"If my normal publish handlers do not run, what can I safely hook into?"

The answer: cache refresher notifications

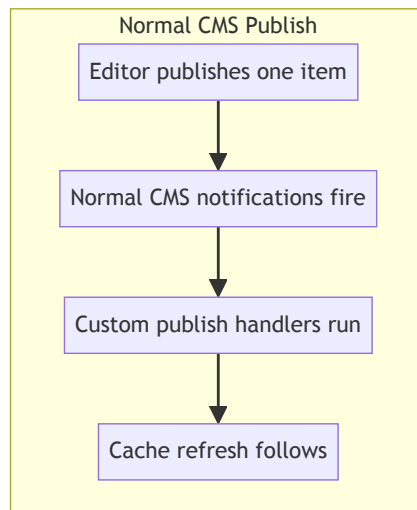
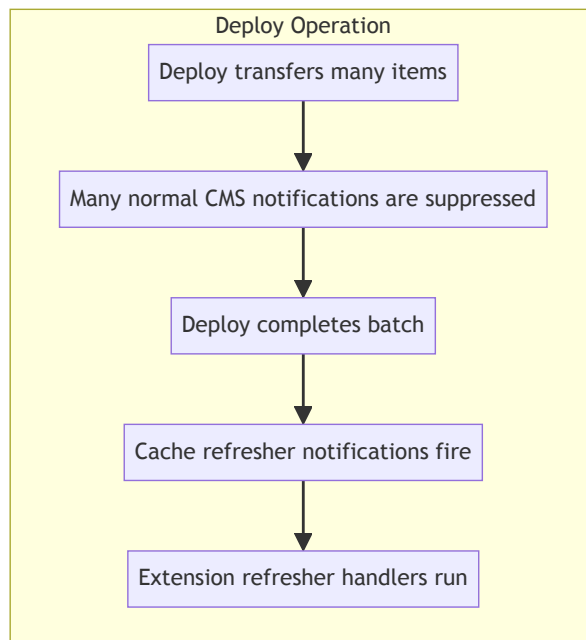
Deploy keeps cache refresher notifications available.

The docs say these are:

- batched
- fired after the deploy operation is complete
- suitable for work that must happen after deployed changes arrive

This is the extension-friendly cache-busting contract.

Normal publish vs Deploy



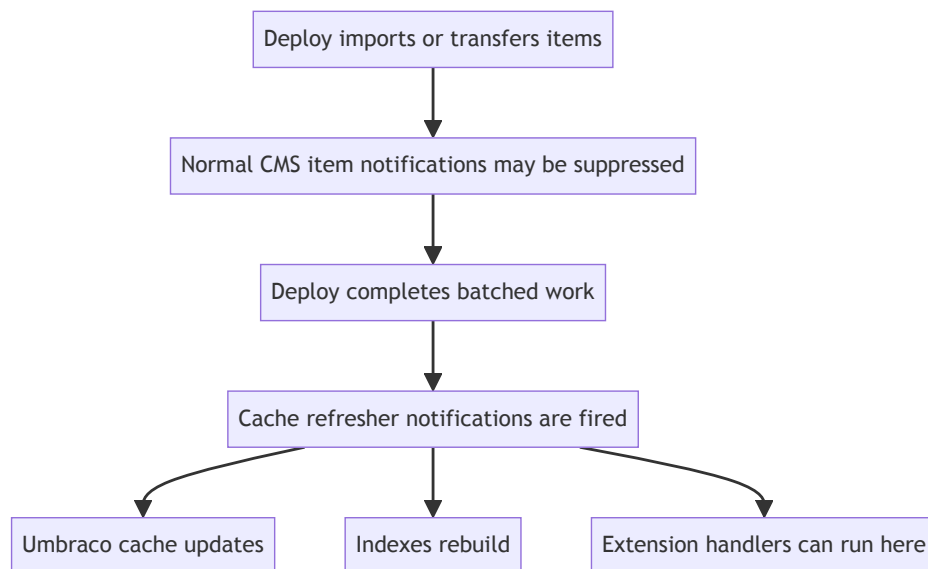
The practical rule

If your extension needs to react correctly after Deploy operations, prefer handling cache refresher notifications over assuming editor-style save/publish notifications will run.

Examples include:

- `ContentCacheRefresherNotification`
- `DictionaryCacheRefresherNotification`
- other CMS cache refresher notifications made available after Deploy

Mental model



What this means for custom extensions

If your package currently says:

- "when content is published, do X"

you should ask:

- "does X also need to happen after Deploy?"

If yes, relying only on `ContentPublishedNotification` is risky.

A safer design is often:

- keep the normal handler if needed for editor-time behaviour
- also handle the relevant cache refresher notification for Deploy-safe behaviour

Example from the Deploy docs

The docs show a `ContentCacheRefresherNotification` handler that:

- checks `MessageType`
- reads the `ContentCacheRefresher.JsonPayload[]`
- extracts content IDs
- reloads local content
- then runs custom logic

That pattern is exactly the right one for extension authors.

Why this is really about cache busting

This may look like an extension-events topic, but underneath it is a cache coherency topic.

Deploy is effectively saying:

- "do not assume editor notifications are the trusted replay model"
- "the trusted post-operation signal is the cache refresher layer"

That is a big architectural clue.

Important Deploy settings

`SuppressCacheRefresherNotifications`

This is the most important setting for our topic.[^05-suppress]

Default:

- `false`

Meaning:

- after Deploy completes, cache refresher notifications are fired
- Umbraco cache and search indexes are updated

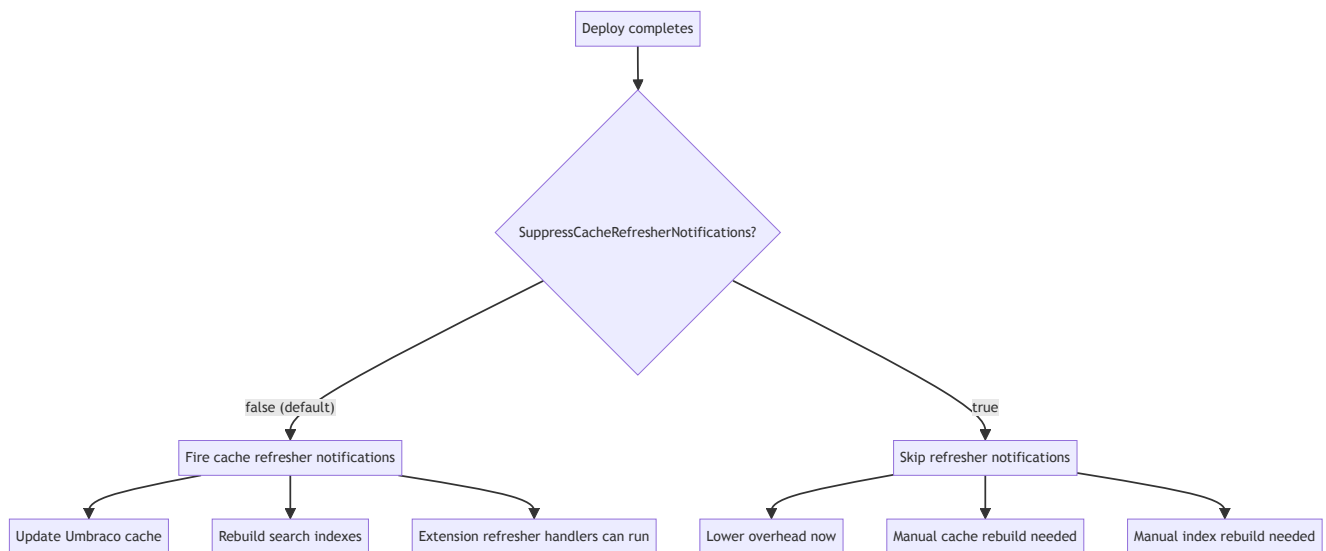
If set to `true` :

- Deploy skips firing and handling those notifications
- this removes performance overhead
- but you must rebuild cache and search index manually afterwards

The docs are very clear that this should not be changed in production.

So for extension authors, the meaning is:

- if someone turns this on, your cache-refresher-based extension hooks may not run either



Why this is the main cache-busting switch

If you want to know where Deploy busts cache, this setting points straight at it.

When `SuppressCacheRefresherNotifications` is `false`, Deploy busts cache by allowing the usual refresher notifications to run after the operation.

When it is `true`, Deploy does not do that automatic busting step, so freshness becomes your responsibility.

Where Deploy updates cache

Strictly speaking, Deploy does not seem to rebuild every cache directly inside the tiny OnPrem assembly we decompiled.

Instead, it updates cache indirectly by completing a Deploy operation and then triggering the CMS refresh pipeline.

So the update path is usually:

1. Deploy imports or restores items.
2. Deploy finishes the batch.
3. Cache refresher notifications are fired.
4. Umbraco and extensions react by refreshing local cache and indexes.

That distinction matters:

- Deploy is often the cause of cache updates
- the CMS refresher infrastructure is the mechanism

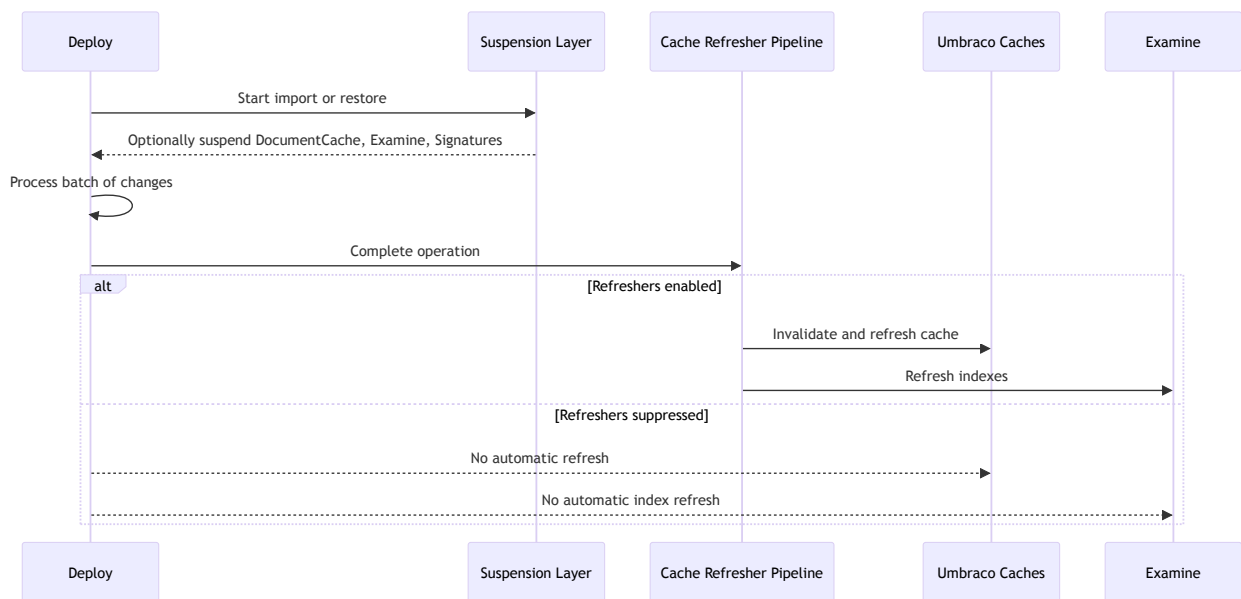
Where Deploy busts cache

For this book, the simplest truthful answer is:

- normal editor actions bust cache one item or one action at a time
- Deploy prefers batched busting after the bulk operation completes

So the most important busting mechanisms are:

- post-operation cache refresher notifications
- optional suspension of cache-related systems while work is in progress
- optional full memory-cache reload after disk-read operations



Suspensions

Deploy also supports suspension settings for operations like:

- `PartialRestore`
- `Restore`
- `Import`

These can suspend things such as:

- scheduled publishing
- Examine
- document cache
- signatures

This matters because it tells us Deploy does not always want live cache/index churn while it is mid-operation. It may suspend those systems and resume them around the operation boundary instead.

ReloadMemoryCacheFollowingDiskReadOperation

This is a workaround setting documented by Deploy for intermittent CMS memory-cache issues after deployments.

If enabled:

- a cache reload is triggered after disk-read operations

This is useful evidence for the book because it shows:

- extension and deployment workflows can expose cache edge cases
- sometimes a full cache reload is the pragmatic repair strategy

Deploy and search indexing

The docs note that cache refresher notifications after Deploy are also used to rebuild indexes.

So in practice, post-Deploy freshness is not just about published content and HTML output.

It also includes:

- search index coherence
- extension logic that depends on refresher notifications

That is another reason to keep cache and index language separate in the book:

- the published cache and app caches are not the same thing as Examine
- but Deploy and refresher workflows often have to keep all of them coherent together

See [13 - Examine, Indexes, and Cache-Adjacent Querying](#).

What the decompiled package did not show

The decompiled files you provided are still useful, but they do not expose the full Deploy internals.

What they showed clearly:

- the OnPrem package is mostly a composition layer
- it delegates the real setup to `AddUmbracoDeployCore(builder)`
- it adds licensing-specific services on top

What they did not show directly:

- the internal import pipeline
- the concrete code that suspends document cache or signatures
- the concrete code that emits cache refresher notifications

So the best evidence-based conclusion is:

- the local decompilation tells us where Deploy enters Umbraco
- the Deploy docs tell us how that deeper infrastructure affects caches

Best-practice guidance for extension authors

1. Separate editor-time hooks from deploy-time hooks

Do not assume they are the same lifecycle.

2. Prefer cache refresher notifications for post-Deploy work

These are the stable contract Deploy intentionally preserves.

3. Keep work idempotent

Deploy may batch many changed items, so your handler should be safe when processing multiple payload entries.

4. Be careful with performance

The reason Deploy suppresses many ordinary notifications is performance and consistency. Extension code should respect that and avoid heavy per-item work where possible.

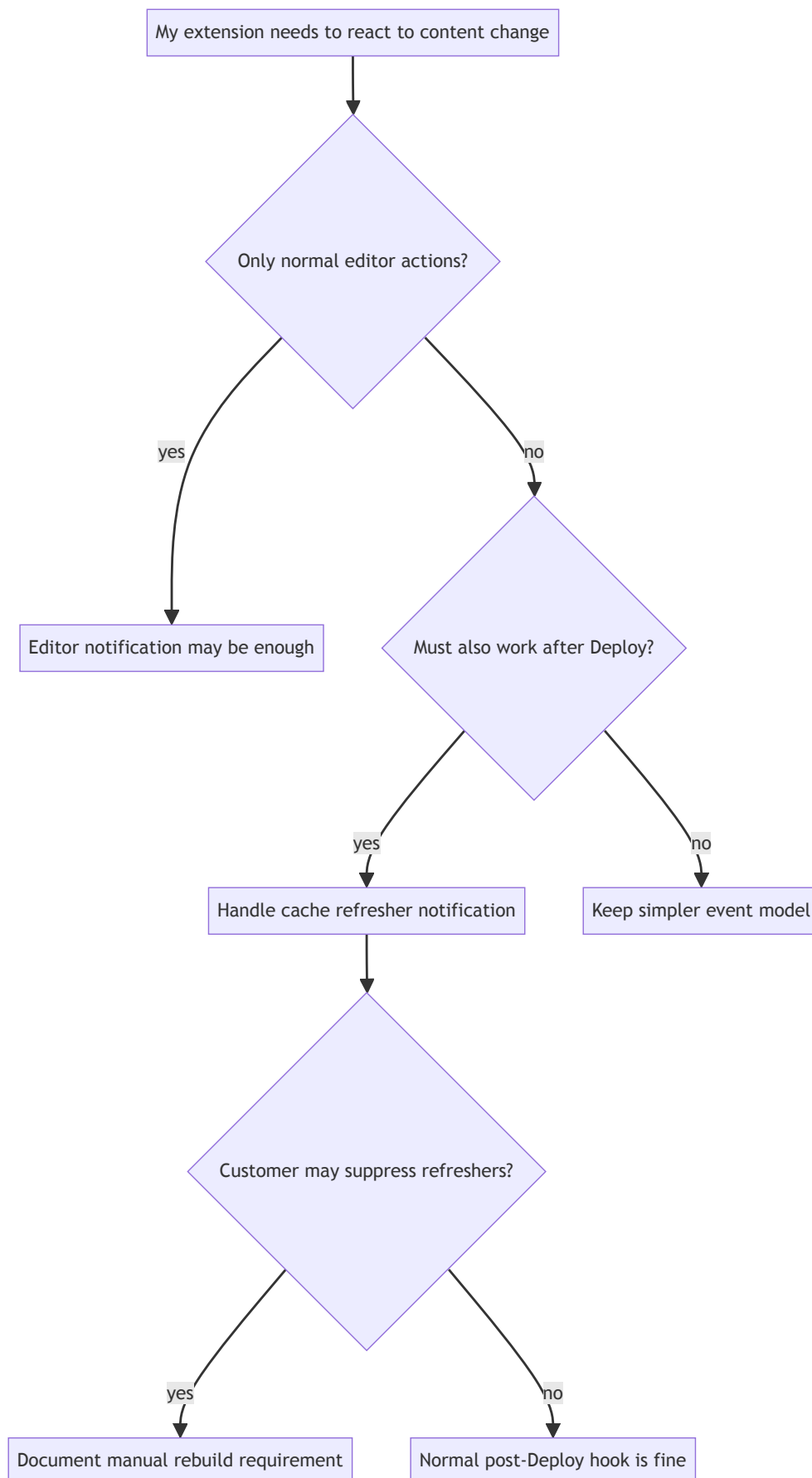
5. Know when your customer has disabled refresher notifications

If `SuppressCacheRefresherNotifications = true`, the normal post-Deploy update path changes dramatically.

Simple decision table

Situation	Better hook
Normal editor publish behaviour only	editor notification may be enough
Must also run after Deploy	cache refresher notification
Large one-off pre-live migration with refreshers suppressed	manual rebuild process required

Extension design map



One-sentence summary

For Umbraco extensions, Deploy-safe cache busting means treating cache refresher notifications as the reliable post-deployment signal, while remembering that Deploy may suspend cache-related systems during bulk work and can even require a forced memory-cache reload in edge cases.

Sources

- Docs:
 - [Handling cache refresher notifications \(Deploy\)](#)
 - [Deploy settings](#)
 - [Deploy troubleshooting](#)
- Decompiled local files:
 - `/Users/ted/Downloads/Umbraco.Deploy.OnPrem/UmbracoDeployOnPremComposer.cs`
 - `/Users/ted/Downloads/Umbraco.Extensions/UmbracoBuilderExtensions.cs`
 - `/tmp/umbraco-deploy-onprem-decompiled/Umbraco.Deploy.OnPrem.decompiled.cs`

Related chapter

For a focused walkthrough of the integration seam itself, see [08 - Umbraco Deploy OnPrem Integration](#).

[^05-deploy]: See [U11 in the appendix](#). [^05-decompiled]: See [D1](#), [D2](#), [D3](#), and [D4](#). [^05-suspensions]: See [U12](#). [^05-suppress]: See [U12](#) and [U13](#).

05. HQ Extensions and Cache

Umbraco HQ ships four main products that extend the CMS: Forms, Deploy, Engage, and Commerce.

Each one interacts with the Umbraco cache in a different way. This chapter only covers the differences from the standard CMS cache behaviour described in the earlier chapters. If a product simply rides the normal cache refresher pipeline without doing anything unusual, that is not worth repeating here.

At a glance

Product	The one cache thing to know
Forms	Form pages are excluded from the website output cache by default
Deploy	Batches work across many items, suppresses per-item notifications, fires refreshers once at the end
Engage	Personalisation and A/B testing require vary-by or bypass of the output cache
Commerce	Cart and checkout pages cannot be output-cached; catalogue pages can be

Umbraco Forms

Forms has a narrow but important interaction with the website output cache.

Why form pages are non-cacheable by default

The website output cache skips storing any response that sets a cookie or carries `Cache-Control: no-store`.

Form pages that render `@Html.AntiForgeryToken()` trigger a `Set-Cookie` response header, which the `DefaultWebsiteOutputCacheRequestFilter` treats as non-cacheable.

This is correct behaviour, not a limitation. Anti-forgery tokens must be unique per session, so a page that embeds one cannot safely be served from a shared cache entry.

Splitting content from forms

If a page mixes static editorial content with a form, you can keep the benefits of output caching for the content area by loading the form separately:

- render and cache the content page normally
- load the form via a separate request, for example an AJAX-loaded partial

The partial is not cached; the content page is.

Forms data is not published content

Form definitions, field configurations, entries, and workflow records are stored in their own database tables.

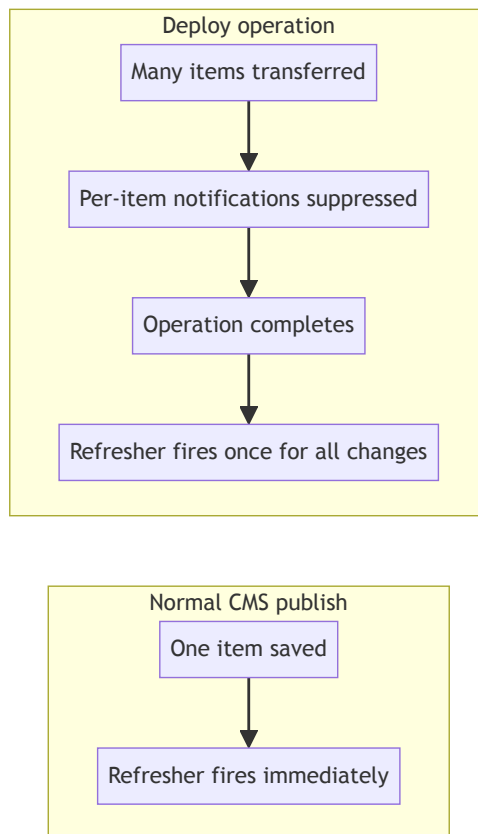
They do not go through the published content cache (`HybridCache`, seeding, refreshers). Standard published-cache behaviour does not apply to form data.

Umbraco Deploy

The core difference from a normal publish

A normal editor publish fires one cache refresher notification per changed item, immediately.

Deploy batches work across many items. While the operation is in progress, per-item notifications are suppressed. When the batch finishes, Deploy fires cache refresher notifications once, covering all the changes it made.



The key setting

`SuppressCacheRefresherNotifications`

- Default `false` : refreshers fire after the operation and cache is updated automatically.
- If `true` : the automatic cache refresh is skipped; you must rebuild cache and search indexes manually.

The Deploy docs advise leaving this at `false` in production.^[^05-suppress]

Advice for extension authors

If your extension needs to react after a Deploy operation, do not rely on `ContentPublishedNotification` or `ContentSavedNotification` . Those may not fire during a Deploy operation.

Hook into the cache refresher notifications instead:

- `ContentCacheRefresherNotification`
- `DictionaryCacheRefresherNotification`
- any other refresher notification that is relevant to your data

Umbraco Engage

Engage handles personalisation, A/B testing, and visitor analytics.

Personalisation creates a direct tension with output caching.

The tension

The website output cache and CDA output cache both assume the same URL returns the same response for every visitor.

Engage breaks that assumption deliberately: different visitor segments can see different content on the same URL.

What this means for output cache

For pages where Engage personalisation is active, the output cache typically needs to either:

- be bypassed for that page, or
- vary by a visitor segment identifier so each segment gets its own cache entry

Serving a cached response intended for one segment to a different segment would be a correctness error, not a performance trade-off.

The published content cache is unaffected

The tension is only at the output cache layer.

The underlying published content cache (`HybridCache`) is not affected. Engage works with the same published content as any other feature.

Practical guidance

Exact Engage integration details should be verified against the Engage documentation. The guidance below is based on the design constraints of personalisation.

- Disable website output caching on pages where Engage is delivering different content per segment, or configure a vary-by key based on the visitor's segment assignment.
- Pages that use only Engage analytics without content personalisation may still be safely output-cached.
- The CDA output cache has the same constraint for headless setups: personalised responses must not be shared across segments.

Umbraco Commerce

Commerce introduces per-user state — shopping carts, orders, personal pricing — which is fundamentally incompatible with shared output caching.

Pages that cannot be output-cached

These pages must bypass the output cache:

- cart and checkout pages (per-user state)
- order confirmation and account pages
- any page that renders member-specific prices or promotions

The existing `DefaultWebsiteOutputCacheRequestFilter` already excludes authenticated requests, so most logged-in customer journeys are excluded by default without extra configuration.

Pages that may be output-cached

Catalogue pages without per-user pricing may be safe to cache:

- product listing pages
- product detail pages, if price and stock are stable or rendered separately
- category and brand pages

The normal rules apply: tag entries on content key, wire eviction to product or price changes.

Commerce entities and the refresher pipeline

Exact Commerce cache internals should be verified against the Commerce documentation or source. The observations below are based on the general Umbraco cache refresher pattern.

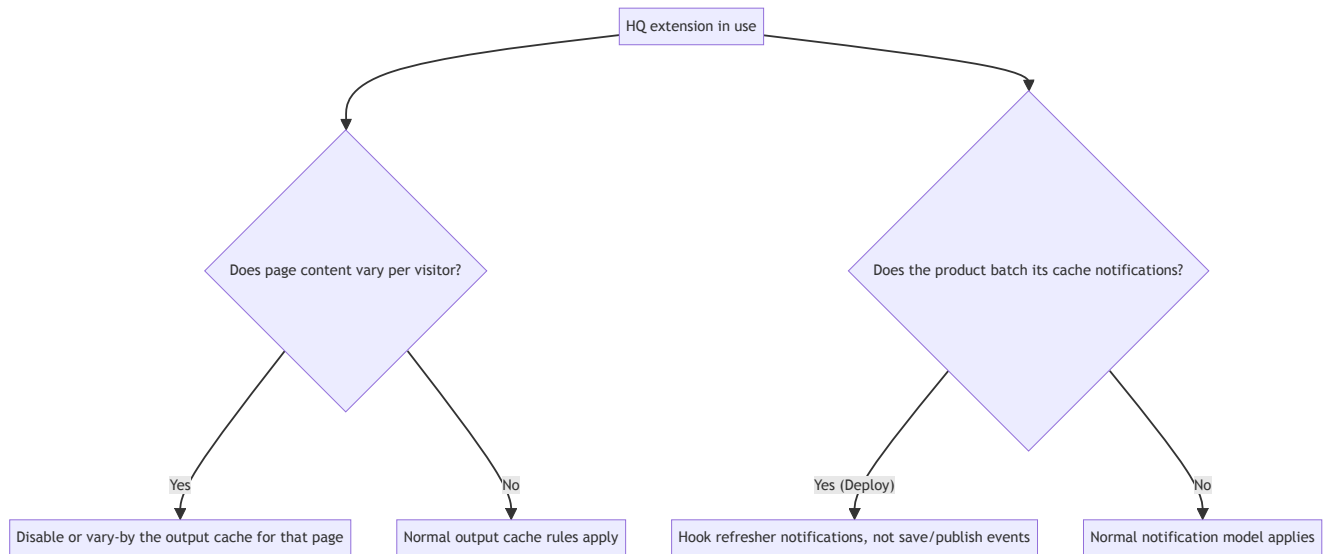
Commerce entities — products, variants, prices, orders — are stored in their own database tables, not in the published content cache.

Commerce registers its own cache refreshers for entity changes. Custom code that caches Commerce data should use `AppCaches` for small results, and combine that with refresher-driven invalidation for multi-server setups.

Common thread

Each product has one or more of these three cache concerns:

1. **Output cache exclusion** — the page cannot be served safely from a shared cache entry (Forms pages, Commerce cart/checkout, Engage personalised pages).
2. **Vary-by requirement** — the page can be cached, but differently per visitor or segment (Engage, member-specific Commerce pages).
3. **Notification batching** — the product uses the CMS refresher pipeline but fires notifications in a different pattern from a normal editor publish (Deploy).



Sources

- [Website output caching \(v17\)](#)
- [Handling cache refresher notifications \(Deploy\)](#)
- [Deploy settings](#)
- [Umbraco Forms docs](#)
- [Umbraco Engage docs](#)
- [Umbraco Commerce docs](#)

[^05-suppress]: See [U12](#) in the appendix.

06. Cache Settings, Talks, and Field Notes

This chapter gathers the settings and supporting material that help explain not just what Umbraco does, but why it does it that way.

Why this chapter exists

The code tells us how the cache behaves.

The docs tell us how to configure it.

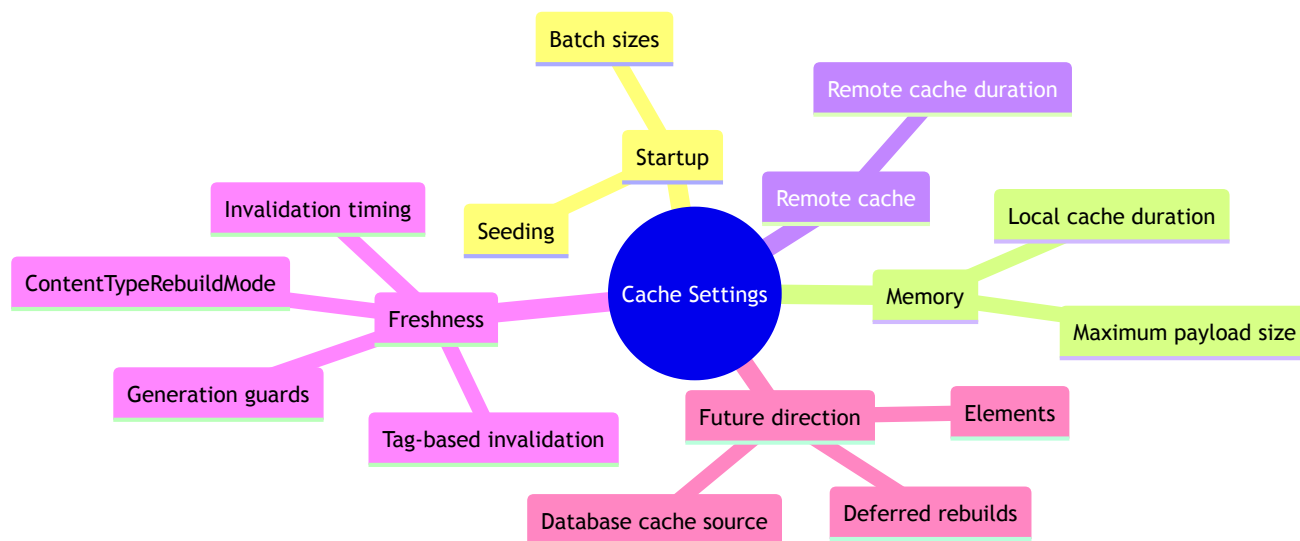
The talk, PDF, and product posts help explain the design pressure behind the new model:[^06-talks]

- too much startup memory
- too much warm-up cost
- too much assumption that all content should stay hot

The newer `main` branch code also reveals more of the actual mechanics behind those pressures:

- a local converted-object hot cache
- serialised `HybridCache` entries
- a database-backed cache source
- tag-based invalidation
- deferred rebuild workflows
- generation guards against stale write-back

One-picture summary



Two cache-settings pages matter

The versioned Umbraco 17 page

Use this as the safest reference for confirmed v17 behaviour:

- [Umbraco 17.latest cache settings](#)

The newer unversioned page

Use this as a signal for where the platform is moving:

- [Current cache settings page](#)

It includes newer element-related settings that go beyond the earlier v17-focused page.

The settings that matter most

`HybridCacheOptions.MaximumPayloadBytes`

This is one of the most important configuration details in the whole cache story.

- Microsoft default is 1 MB
- Umbraco overrides it to 100 MB by default[^06-payload]

Why:

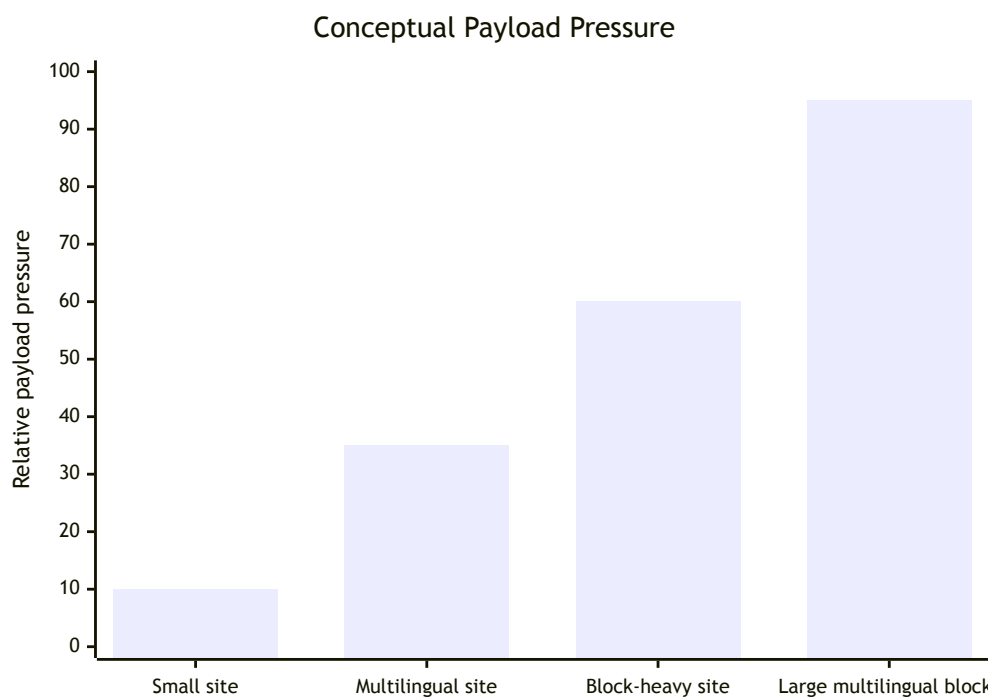
- multilingual payloads grow fast
- block editors grow fast
- a realistic Umbraco page tree can exceed generic defaults surprisingly quickly

This is also a nice example of Microsoft versus Umbraco responsibilities:

- Microsoft exposes `MaximumPayloadBytes` as a base `HybridCache` option
- Umbraco raises it because real published-content payloads are much larger than the generic default expects

And the `main` branch code makes the reason even clearer:

- entries are serialised `ContentCacheNode` payloads
- those payloads may contain a lot of culture, property, and nested data
- compression helps, but realistic sites still produce large objects



Seeding settings

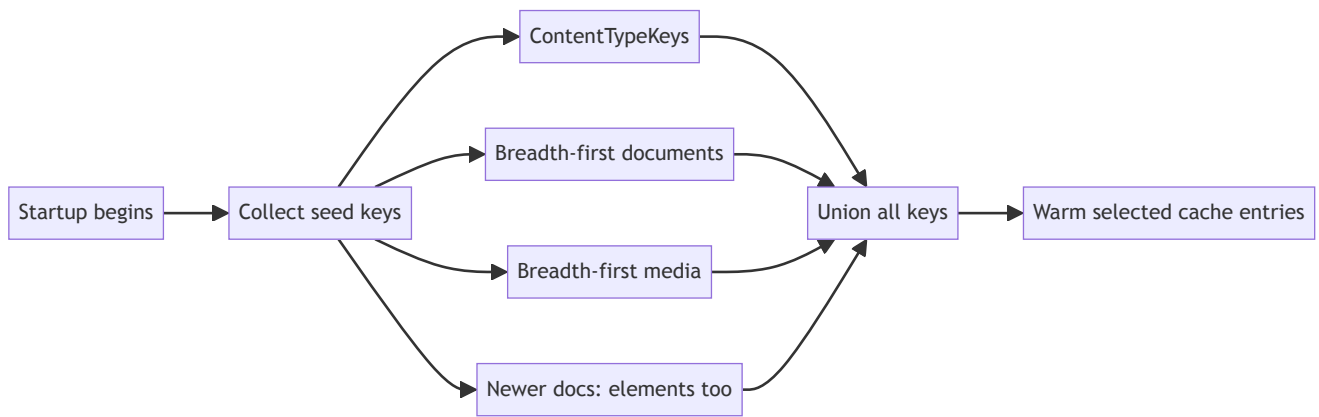
These decide what gets prewarmed on startup.

Important settings include:

- `ContentTypeKeys`
- `DocumentBreadthFirstSeedCount`
- `MediaBreadthFirstSeedCount`

And in newer docs:

- `ElementBreadthFirstSeedCount`



Seed batch sizes

These control how seed keys are processed during startup.

- `DocumentSeedBatchSize`
- `MediaSeedBatchSize`

And in newer docs:

- `ElementSeedBatchSize`

These are not just performance tuning knobs.

They also affect:

- startup shape
- database pressure during warm-up
- how quickly a new node becomes warm after restart

Cache entry durations

These settings control how long entries stay around.

`LocalCacheDuration`

How long entries stay in local memory.

`RemoteCacheDuration`

How long entries stay in a configured second-level cache.

This only matters if a remote cache exists.

`SeedCacheDuration`

How long seeded entries stay cached.

This is especially relevant for deliberately hot content such as:

- home pages
- key landing pages
- frequently reused media

The code also makes an architectural point here:

- seeded entries are not only "loaded first"
- they can also be given deliberately different lifetimes from ordinary entries

Newer element settings

The newer docs show the cache model becoming more explicit for elements:

- `ContentTypeKeys` now describes document types and element types
- entry durations are described for `Element`
- seeding counts and batch sizes are described for `Element`

This matches what we saw in the 18.0.2 code.

`ContentTypeRebuildMode`

This is one of the most revealing settings because it explains a real invalidation trade-off.

Values: `[^06-rebuildmode]`

- `Immediate`
- `Deferred`

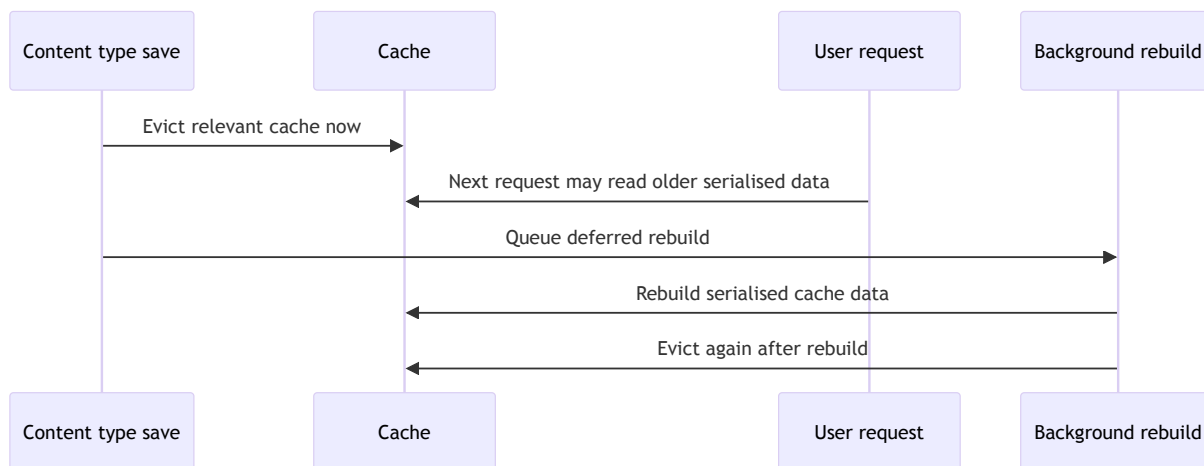
With `Deferred` :

- cache is still evicted immediately
- old serialised data may still be served temporarily
- background rebuild completes later
- cache is evicted again after rebuild completes

That is a beautiful example of how cache busting can be correct even when freshness is not perfectly immediate.

The `main` branch code also shows how this works operationally:

- structural content-type changes can be queued
- IDs are deduplicated
- a background worker rebuilds the database cache source
- matching `HybridCache` tags are removed
- matching converted-object caches are cleared



Old `NuCache` settings that still linger

Some settings still live under:

- `Umbraco:CMS:NuCache`

Examples:

- `UsePagedSqlQuery`
- `SqlPageSize`
- `NuCacheSerializerType`

These exist mostly for backward-compatibility reasons.

The important beginner lesson is:

- the modern implementation is HybridCache-based
- but not every old configuration name disappeared overnight

For a clearer separation between the old architecture and the newer one, see [12 - NuCache vs Hybrid Cache](#).

An even better lesson is:

- some old names still exist
- but the current architecture is no longer "just old NuCache with a new wrapper"
- it is a layered pipeline with explicit seeding, serialisation, rebuilding, and cache tagging

Microsoft-first takeaway

The future Hybrid Cache story is easiest to understand in two layers:

1. Microsoft defines the generic cache primitive.
2. Umbraco defines the published-content system built on top of it.

That framing helps a lot when reading the code, because some behaviours belong mostly to Microsoft:

- primary and secondary cache flow
- stampede protection
- serialiser registration
- tags
- entry options

while other behaviours belong mostly to Umbraco:

- content and element seed keys
- `ContentCacheNode`
- database cache rebuild
- content-type rebuild strategy
- preview and published separation

Video notes

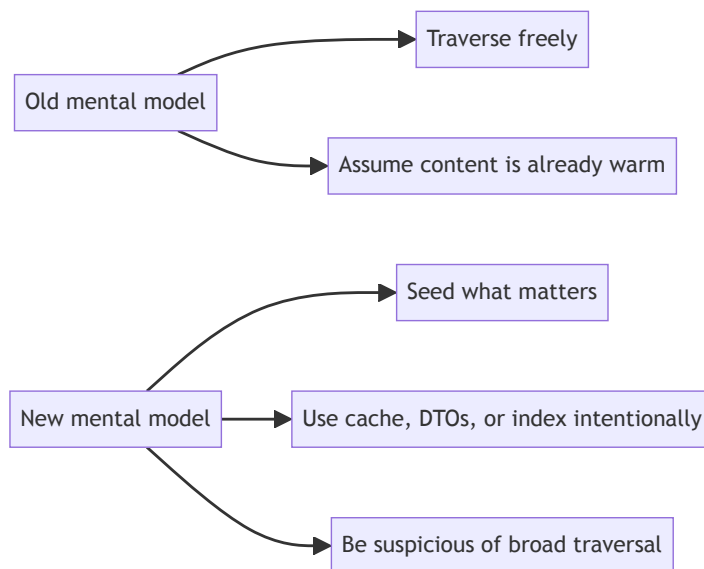
The talk:

- [Releasing HybridCache into the Wild with Umbraco](#)

is useful because it frames the migration in plain language.

Best high-level takeaways:

- the old model wanted too much in memory
- the new model warms what matters and lets colder entries miss
- startup and memory profile improve
- query and traversal habits matter more now



PDF notes

The PDF:

- [Hybrid Cache förändrar allt](#)

is useful because it states the trade-offs bluntly.

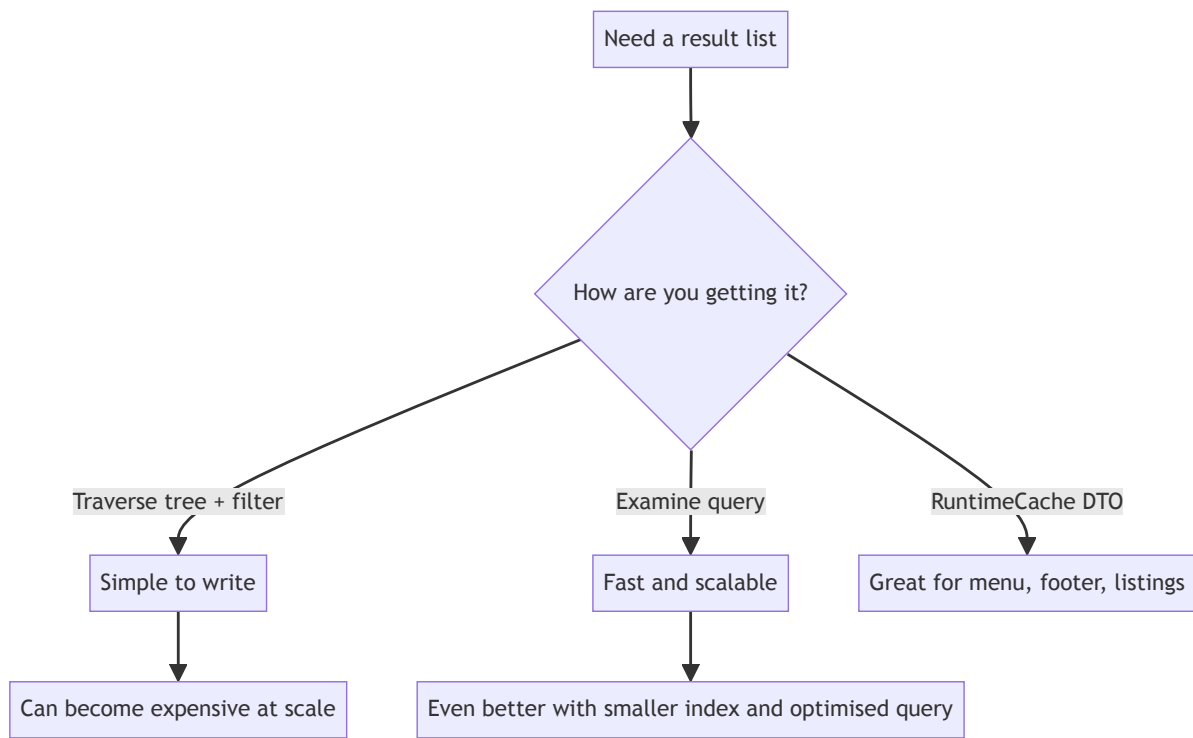
Big wins:

- lower memory usage
- faster startup
- better scaling for large sites

Big warnings:

- broad traversal becomes more expensive
- in-memory filtering habits become more visible
- seed strategy matters more than before

Query strategy graph



That is also the cleanest place to separate index and cache concerns:

- `Examine` is often the right answer when the hard part is discovery
- `RuntimeCache` is often the right answer when the hard part is recomputation

See [13 - Examine, Indexes, and Cache-Adjacent Querying](#) for the full comparison.

Best field note

The newer HybridCache world rewards:

- precise invalidation
- deliberate seeding
- better content-querying habits
- understanding which layer you are hitting
- knowing when a structural change needs rebuild rather than simple eviction

It punishes:

- lazy traversal
- assuming everything is already warm
- pretending startup and memory are free
- ignoring cache-busting complexity under concurrent refresh

Upgrade troubleshooting field note

One practical lesson from early Umbraco 16 upgrade reports is that severe post-upgrade "slowness" is not always a straightforward cache-tuning problem.

In one community report, an upgrade from Umbraco 13 to 16 led to 30 to 40 second delays in the backoffice during login and editing, while the frontend also became temporarily unresponsive.^[^06-upgrade]

That pattern matters.

If the backoffice hangs and the frontend stalls at the same time, the problem may be broader than "the editor UI feels slow". In Jacob Overgaard's reply, the likely explanation was database locking, including the long-running "Failed to acquire write lock" class of problems.^[^06-upgrade]

The same thread also suggested that more than one regression could overlap:

- a login-flow bug being addressed for 16.3
- a caching regression around missing dictionary-item null values in Umbraco 16

That is a useful operational reminder for this chapter.

When teams report that a site became slower after a major upgrade, treat that as a symptom description first, not a diagnosis. It may reflect lock contention, a version-specific regression, query-shape changes under HybridCache, or several of those at once. [^06-upgrade]

What the main branch adds to the story

The newer source shows several details that are easy to miss in talks alone:

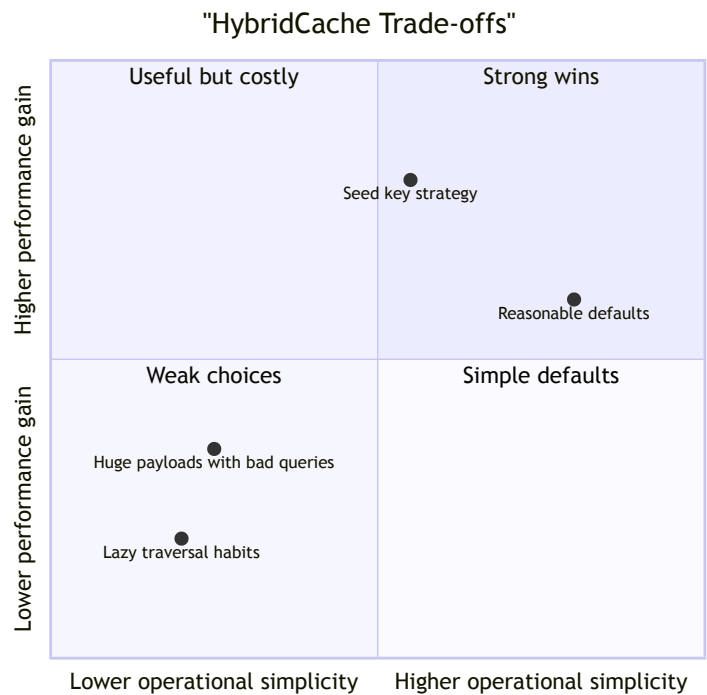
- the local converted-object cache is a first-class performance layer
- null entries are tagged so negative caching can also be invalidated correctly
- generation counters stop stale in-flight reads from clobbering fresher entries
- serialiser changes can trigger full database cache rebuilds at startup

Those are not small implementation details.

They are the practical engineering that makes the architecture reliable.

See also 10 - Future Hybrid Cache Architecture.

Trade-off chart



Sources

- Docs:
 - [Cache settings \(v17\)](#)
 - [Cache settings \(latest\)](#)
- Supporting material:
 - [Hybrid Cache förändrar allt — Umbraco Kalaset session \(YouTube\)](#)
 - [Hybrid Cache förändrar allt — Umbraco Kalaset slides \(PDF\)](#)

[^06-talks]: See [T1](#), [T2](#), [T4](#), [T5](#), [B1](#), [B5](#), and [B6](#). [^06-payload]: See [M6](#), [U4](#), [U5](#), and [C4](#). [^06-rebuildmode]: See [U5](#), [C4](#), and [C5](#).

[^06-upgrade]: See [F1](#), [F2](#), and [F3](#).

07. Small Local Cache Example with Tags

This chapter is the simple, friendly example.

It is not the full enterprise load-balanced story.

It is the small local pattern:

- cache something in `RuntimeCache`
- clear it by key when the content that affects it changes

Why this example is useful

The bigger cache chapters can feel abstract.

This one shows the smallest possible idea:

- pick a cache key
- decide how to fill it
- decide when to bust it

That is the basic muscle behind every bigger cache strategy.

The scenario from the docs

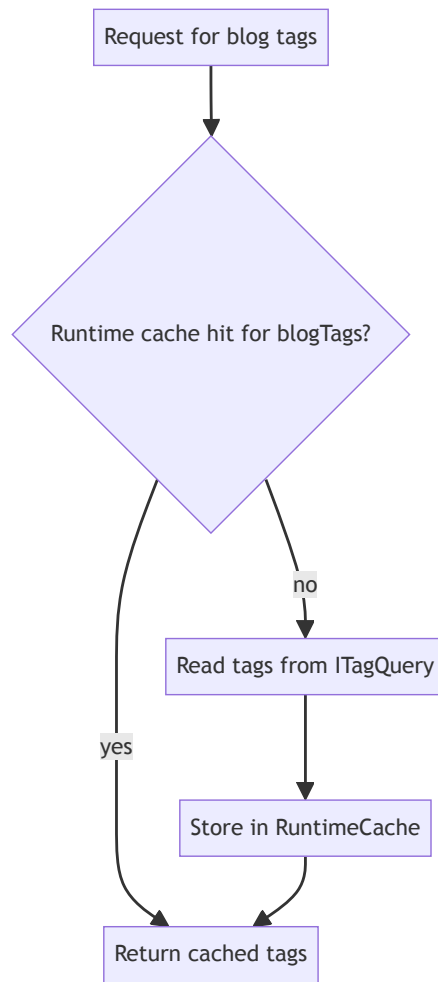
The docs use tag groups as the example:[^07-tags]

- one tag group called `default`
- one tag group called `blog`

The idea is:

- `default` tags are cached for one minute
- `blog` tags are cached until restart or manual removal
- publishing a `blogPost` clears the `blogTags` cache key

The simple architecture



The fill rule

The example uses:

- `AppCaches.RuntimeCache`
- `GetCacheItem(cacheKey, factory, timeout)`

So the cache fill logic becomes:

- if key exists, return cached value
- if key does not exist, fetch from `ITagQuery`
- store and return the result

The bust rule

The example then adds a notification handler for: `^07-publish`

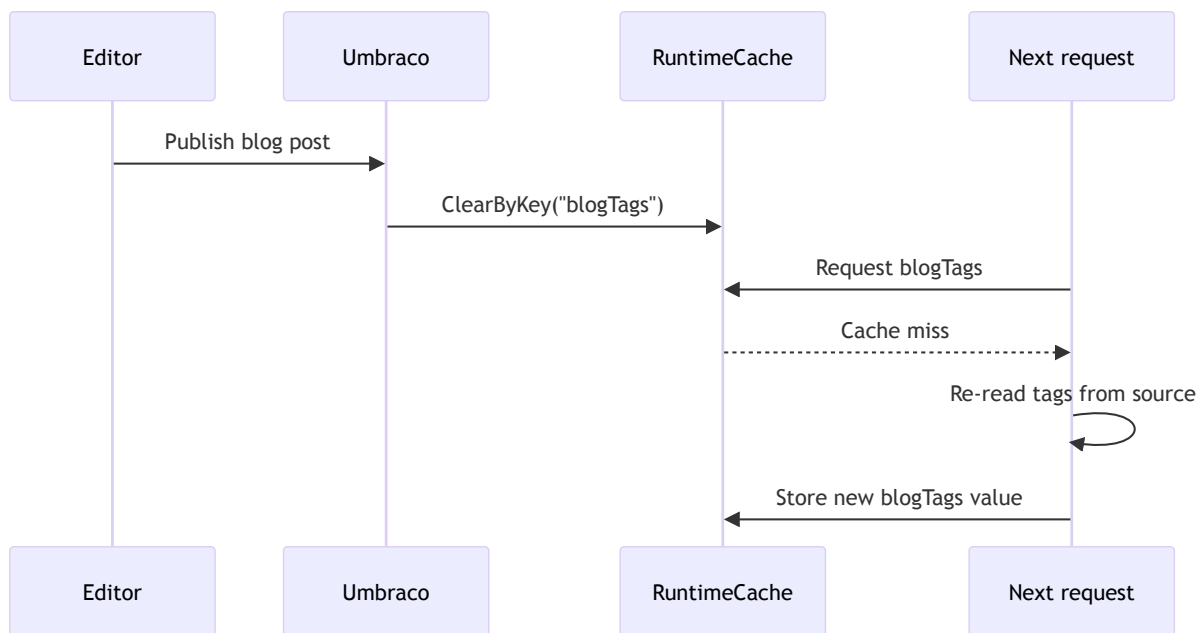
- `ContentPublishedNotification`

When a published entity has content type alias `blogPost`, it calls:

- `RuntimeCache.ClearByKey("blogTags")`

That means:

- publish blog post
- clear `blogTags`
- next request repopulates from source



Why this pattern is good

For simple local caching, this is easy to reason about:

- one cache key
- one source
- one clear event

It is also easy to debug.

Why this pattern is not enough on its own

This is the important warning.

By itself, this example is not the whole load-balanced story.

Why:

- it clears a local runtime cache key
- but it does not automatically describe how every other server should clear the same key

So this is a great pattern for understanding local cache behaviour, but it must be combined with the `ICacheRefresher` and `DistributedCache` model for multi-server-safe invalidation.

There is also a separate question hiding underneath it:

- if the problem is "remember this small computed result", `RuntimeCache` is a good fit
- if the problem is "find the right items across a large content set", `Examine` may be the better fit

See [13 - Examine, Indexes, and Cache-Adjacent Querying](#).

Good mental split

Small local app cache

Use:

- `RuntimeCache`
- `GetCacheItem(...)`
- `ClearByKey(...)`

Load-balanced-safe custom cache

Use:

- `RuntimeCache` for storage if appropriate
- plus `ICacheRefresher`
- plus `DistributedCache`

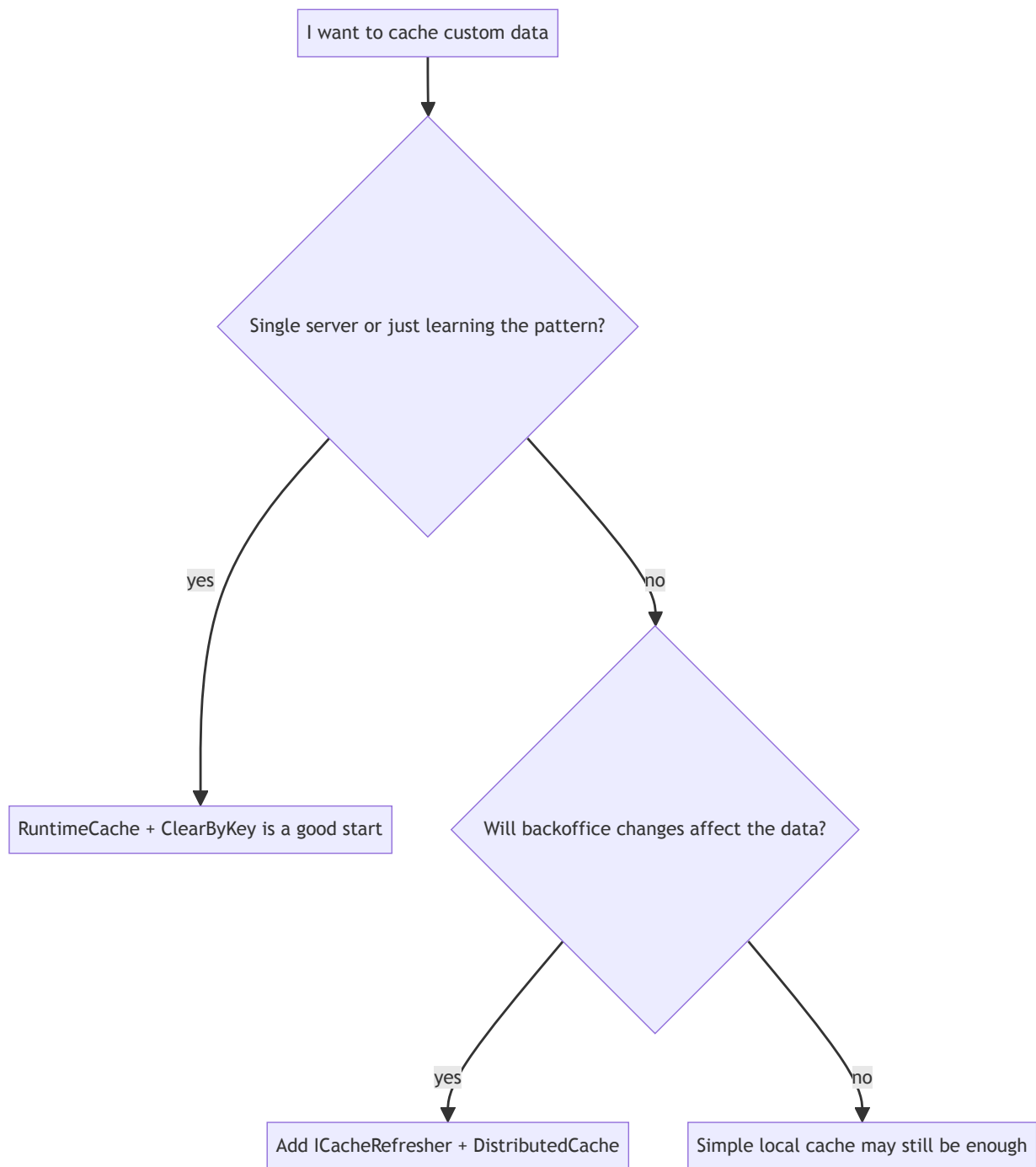
Relation to the bigger book

This chapter is the "hello world" for cache busting.

The bigger chapters explain:

- how Umbraco core does this at scale
- how output cache uses tags and eviction handlers
- how Deploy changes the notification model
- how load balancing requires distributed invalidation

Tiny decision chart



Another useful connection: seed providers

This chapter is about a tiny local cache example.

The custom seed key provider example is the opposite scale:

- instead of waiting for a request
- Umbraco decides at startup which keys should already be warm

That means both patterns answer the same question from different angles:

- tags example: "what should happen on miss?"
- seed-provider example: "what should not miss in the first place?"

Best one-line summary

This tags example teaches the smallest useful cache-busting loop in Umbraco: fill by key, clear by key, refill on next request.

Sources

- Docs:
 - [Cache tags example](#)
 - [Application cache](#)
 - [Server-side cache extensions](#)

[^07-tags]: See [U10 in the appendix](#) and [U7](#). [^07-publish]: See [U10](#) and [U6](#).

08. Umbraco Deploy OnPrem Integration

This chapter is about the integration boundary.

It does not try to pretend we have every internal Deploy class in front of us.

Instead, it documents what we can already prove from the decompiled `Umbraco.Deploy.OnPrem` package, and then connects that to the cache behaviour described in the official Deploy documentation.^[^08-scope]

Why this chapter matters

When people ask, "how does Deploy affect cache?", they often mix together three different things:

- the tiny OnPrem integration package
- the deeper Deploy infrastructure package
- the normal Umbraco cache refresher pipeline

Those are connected, but they are not the same thing.

This chapter is here to separate them cleanly.

The shortest true answer

`Umbraco.Deploy.OnPrem` is the bootstrapper.

It integrates Deploy into Umbraco, but it does not appear to contain the main cache mutation logic itself.

The real cache work seems to live in:^[^08-realwork]

- `Umbraco.Deploy.Infrastructure`
- Umbraco CMS cache refresher notifications
- Deploy settings that control refresh, suspension, and reload behaviour

What the decompiled files show

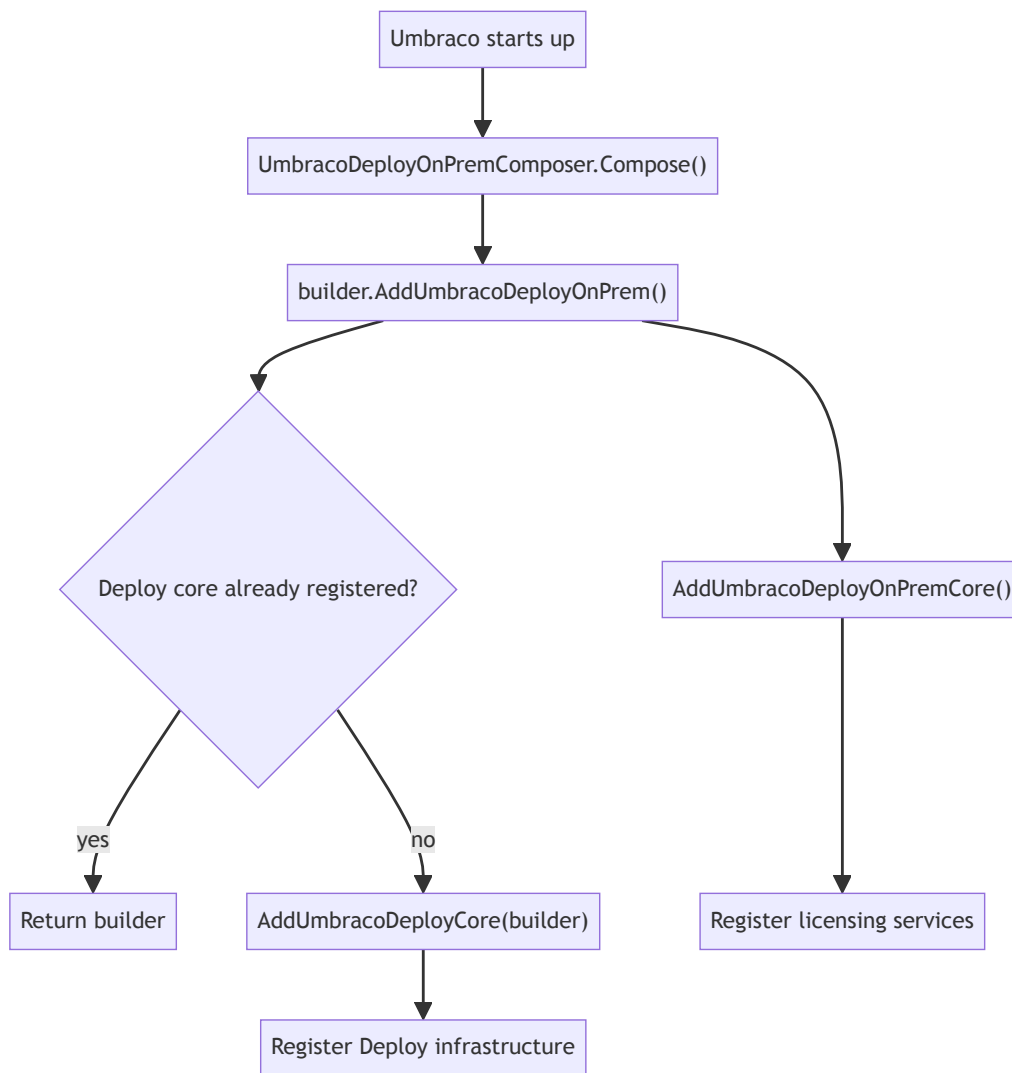
From the decompiled local files:

- `UmbracoDeployOnPremComposer.Compose()` calls `builder.AddUmbracoDeployOnPrem()`
- `AddUmbracoDeployOnPrem()` checks whether Deploy core is already registered
- if not, it calls `AddUmbracoDeployCore(builder)`
- after that, it adds OnPrem-specific licensing services

So the OnPrem package is mainly doing composition and registration.

It is not, by itself, the part that obviously clears content cache, refreshes indexes, or suspends document cache.

The integration path



What that means for caches

This integration path matters because it tells us where to look.

If we are trying to answer:

- where cache is updated
- where cache is used
- where cache is busted

the answer is not "inside the small OnPrem composer".

The composer only opens the door.

The cache-related behaviour happens later, inside the registered Deploy core services and the CMS refresh pipeline they use.

What we can prove from the DLL boundary

The DLL clearly references types in `Umbraco.Deploy.Infrastructure`, including: [^08-dll]

- `Umbraco.Deploy.Infrastructure.Extensions`
- `Umbraco.Deploy.Infrastructure.Disk`
- `Umbraco.Deploy.Infrastructure.Licensing`

That tells us:

- OnPrem depends on Deploy infrastructure
- OnPrem does not stand alone

- the interesting runtime behaviour is delegated

This is a very important architectural clue.

Cache impact: the best current model

With the evidence we have today, the cleanest model is this:

1. OnPrem integrates Deploy into Umbraco

This happens during composition and service registration.

2. Deploy core performs import, restore, and disk-based operations

This is where the heavy lifting happens.

3. Cache refresh happens through the Umbraco refresher layer

After operations complete, cache refresher notifications are the normal post-operation mechanism.

4. Some cache-related systems may be suspended during the operation

The Deploy settings docs mention suspension targets like:

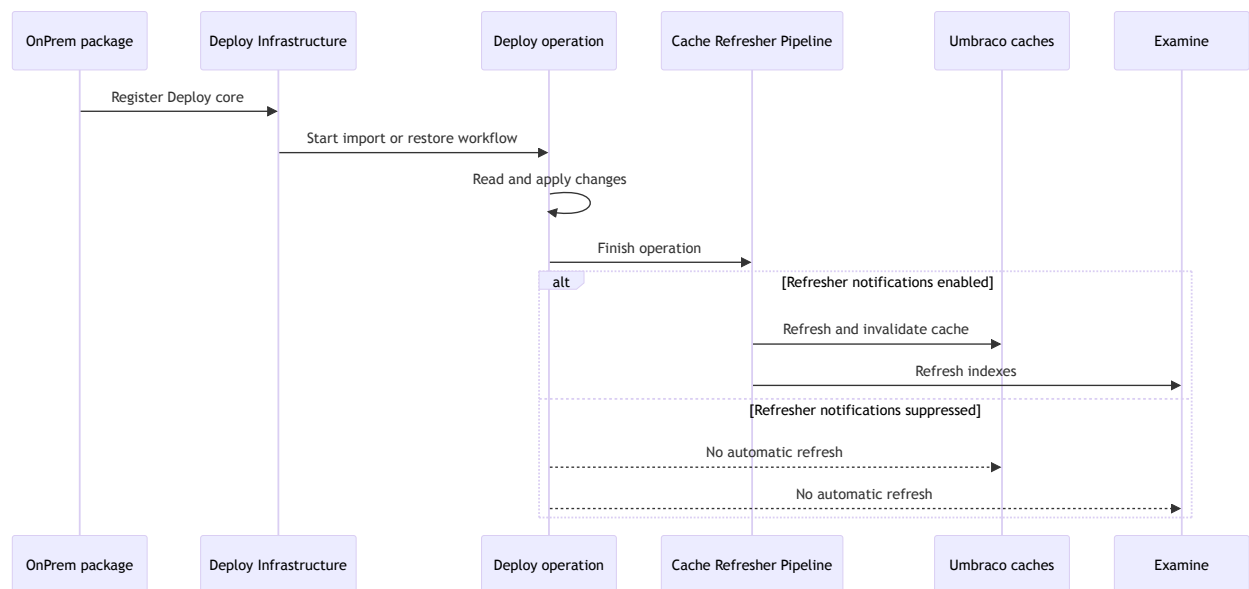
- `DocumentCache`
- `Examine`
- `Signatures`

So Deploy may avoid live cache and index churn while it is mid-operation.

5. In edge cases, a broader memory-cache reload may be needed

That is what `ReloadMemoryCacheFollowingDiskReadOperation` is about.

The likely runtime story



Where cache is used

Deploy uses cache indirectly through Umbraco's existing cache infrastructure.

The important examples are:

- published content cache updates after Deploy operations
- distributed refresher notifications in multi-server setups

- index refresh following deployed changes
- temporary suspension of cache and index-related systems during bulk work

So Deploy is not a separate cache engine.

It is a strong participant in Umbraco's normal freshness pipeline.

Where cache is updated

The clearest answer is:

- Deploy updates the system state
- Umbraco refreshers update the cache state

That distinction is important.

Deploy may be the trigger, but the cache refresher pipeline is the mechanism.

Where cache is busted

For this chapter, "cache busting" means invalidating stale state so fresh data can be served.

The main current busting path is:

1. Deploy finishes a batch of changes.
2. Cache refresher notifications are fired.
3. Umbraco caches and indexes refresh.

That is the big one.

The other important paths are:

- suspending cache-related systems while work is in progress
- reloading memory cache after certain disk-read operations
- manually rebuilding cache and indexes when refreshers are intentionally suppressed

The most important setting

`SuppressCacheRefresherNotifications`

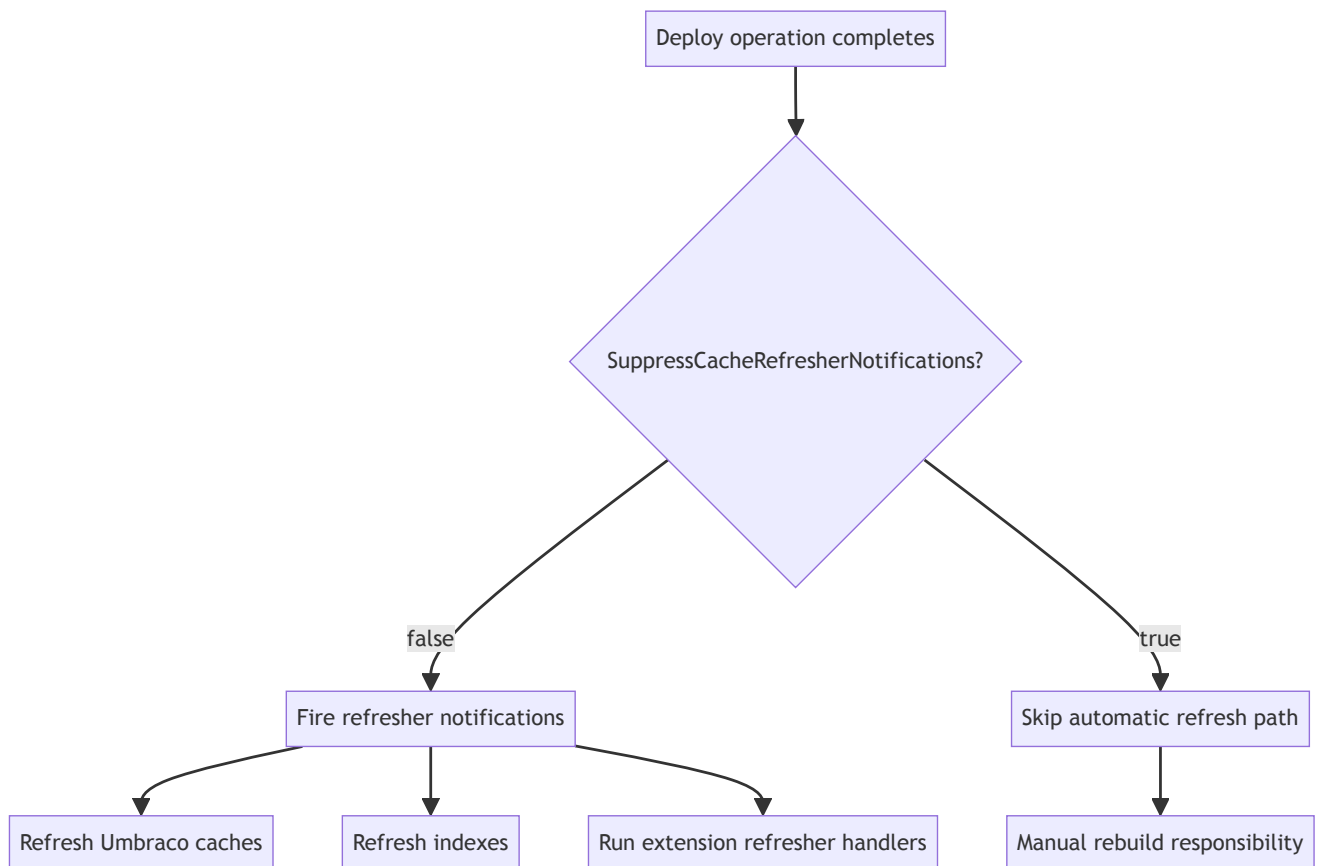
This is the most important Deploy setting for understanding cache busting.

If it is `false` :

- refresher notifications are fired after the Deploy operation
- Umbraco cache is updated
- indexes are rebuilt
- extension handlers listening to refresher notifications can run

If it is `true` :

- that automatic post-Deploy refresh path is skipped
- performance overhead is reduced
- manual cache and index rebuild becomes necessary



What extension authors should learn from this

If your extension needs to react after Deploy, the safe mental model is:

- do not rely only on editor-style save and publish notifications
- expect Deploy to batch work
- expect many normal notifications to be suppressed
- treat cache refresher notifications as the reliable post-Deploy signal

That is the integration lesson.

What this chapter does not claim

This chapter does not claim that we have already decompiled every internal Deploy cache class.

We have not.

Specifically, from the current artefacts we cannot yet point to:

- the exact internal class that suspends `DocumentCache`
- the exact internal class that emits refresher notifications
- the exact internal implementation of `ReloadMemoryCacheFollowingDiskReadOperation`

So this chapter is intentionally honest about the evidence boundary.

Practical conclusion

`Umbraco.Deploy.OnPrem` is not the part that directly looks like "cache code".

It is the integration seam that brings Deploy core into Umbraco.

Once that seam is in place, Deploy affects cache by participating in Umbraco's refresh pipeline, by optionally suspending cache-related systems during bulk work, and by relying on explicit refresh or reload strategies after the operation boundary.

Sources

- Docs:
 - [Handling cache refresher notifications \(Deploy\)](#)
 - [Deploy settings](#)
 - [Deploy troubleshooting](#)
- Decompiled local files:
 - `/Users/ted/Downloads/Umbraco.Deploy.OnPrem/UmbracoDeployOnPremComposer.cs`
 - `/Users/ted/Downloads/Umbraco.Extensions/UmbracoBuilderExtensions.cs`
 - `/Users/ted/Downloads/umbraco.deploy.onprem.18/lib/net10.0/Umbraco.Deploy.OnPrem.dll`
 - `/tmp/umbraco-deploy-onprem-decompiled/Umbraco.Deploy.OnPrem.decompiled.cs`

[^08-scope]: See [U11](#), [U12](#), [D1](#), and [D4](#). [^08-realwork]: See [D2](#), [D3](#), and [U12](#). [^08-dll]: See [D3](#) and [D4](#).

09. Storage Providers and Media Caching

This chapter is about a different side of caching.

Most of this book focuses on:

- published content cache
- cache refreshers
- load balancing
- Deploy-related cache busting

`Umbraco.StorageProviders` is not mainly about those things.

It is mainly about the caching and delivery story for media files.

The short version

Storage Providers affect cache in three important ways:[^09-three]

1. they can put media behind a CDN
2. they can cache Azure Blob metadata in `HybridCache`
3. they can move the ImageSharp image cache into Azure Blob Storage

So this is not "content cache" in the usual Umbraco sense.

It is "media delivery cache".

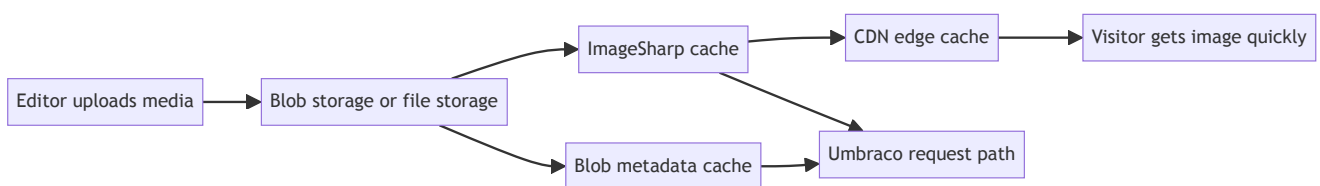
Why this matters

When a page renders an image, several different things may need to be fast:

- finding the media URL
- checking blob metadata like file size or last modified time
- generating an image variant
- serving that variant repeatedly without recomputing it
- serving the result from a CDN edge instead of the origin server

Storage Providers plug into that whole path.

The three cache layers



1. CDN media URL provider

The shared `Umbraco.StorageProviders` package includes a CDN media URL provider.

Its job is not to cache media inside Umbraco memory.

Its job is to generate media URLs that point at a CDN, so the CDN can cache the media response closer to visitors.

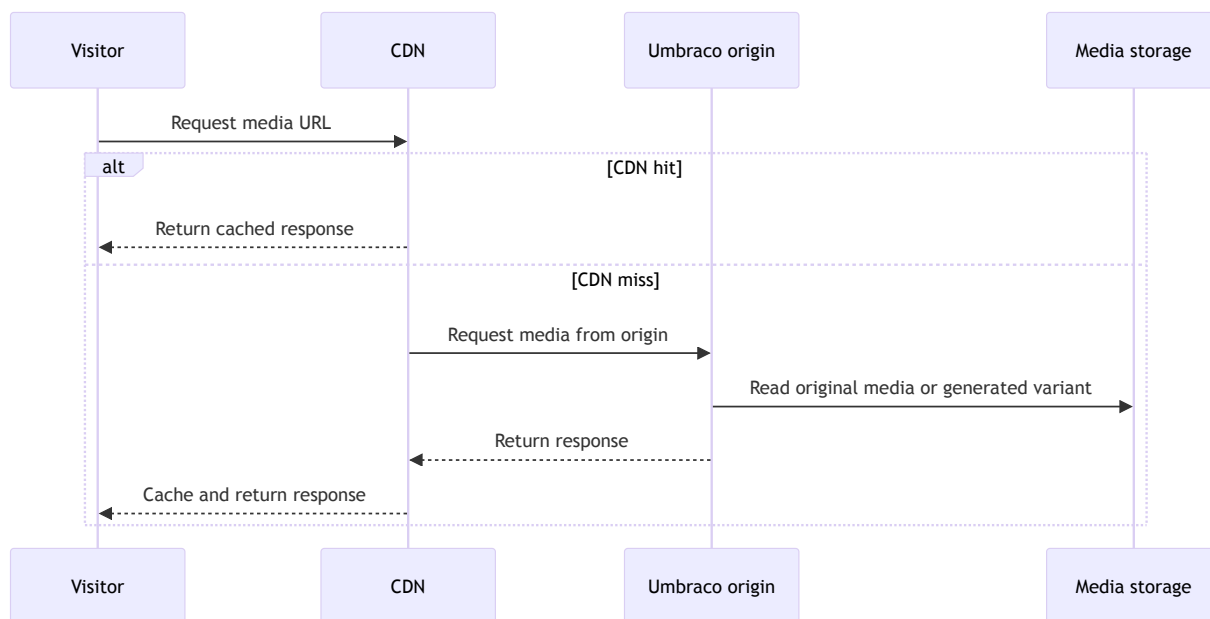
That means:

- the cache lives mainly at the CDN edge
- media requests do not always need to hit the Umbraco origin
- repeated requests for the same image become much cheaper

The package README also warns that the CDN origin should cache each unique URL, including the query string.

That is especially important for image processing URLs, because different query strings may mean different resized or cropped image variants.

CDN mental model



2. Azure Blob metadata caching

This is the most obviously cache-related feature in the package.

The Azure Blob provider caches blob metadata such as:

- file size
- last modified time

The README explains why this exists:

- the media file provider can otherwise call `Azure GetProperties()` on every request
- under load, retries and network delays can hold threads for a long time
- caching avoids those repeated round trips on the hot path

The package says this metadata caching uses Umbraco's shared `HybridCache`.

That is a very interesting connection to the wider book, because it means Storage Providers are already using the newer hybrid cache model for a practical workload.

Why this is a good fit for `HybridCache`

Blob metadata is a nice example of the sort of data `HybridCache` handles well:

- small
- read often
- expensive enough to fetch remotely
- safe to keep briefly
- useful to protect from stampedes

The README also says this gets stampede protection, so concurrent cold requests for the same blob path share one fetch instead of all hitting Azure separately.

Metadata cache behaviour

The package documents three especially important rules:

- metadata caching is enabled by default
- successful lookups have a hit duration
- not-found results have a shorter miss duration

That shorter miss duration is clever because it reduces the time newly uploaded blobs can remain invisible.

Metadata cache busting

This is the important bit for our book.

The provider says writes through the file system invalidate affected cache entries immediately.

That means operations like:

- `AddFile`
- `DeleteFile`
- `DeleteDirectory`

will bust the relevant metadata cache entries at once.

So the normal freshness story is:

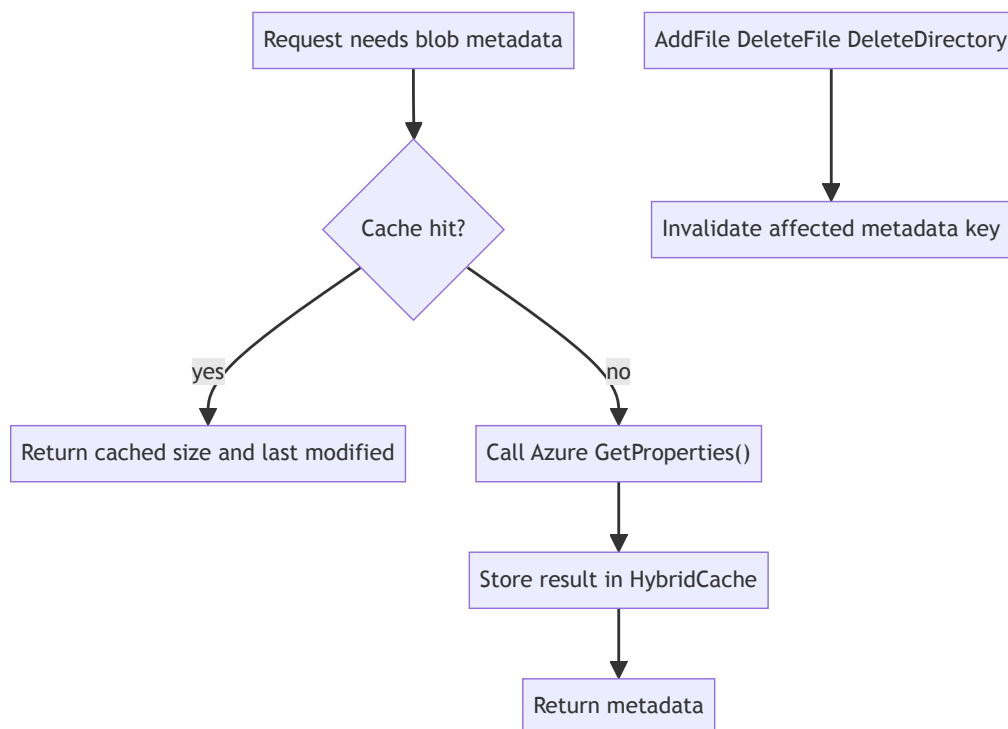
- read path uses `HybridCache`
- write path invalidates the affected key immediately

The stale-data risk mainly comes from writes that happen outside the current instance, such as:

- another process
- another instance
- direct Azure SDK access

In those cases, metadata may stay stale until the configured cache duration expires.

Metadata cache flow



3. ImageSharp cache in Azure Blob Storage

The `Umbraco.StorageProviders.AzureBlob.ImageSharp` package adds ImageSharp support for storing the image cache in Azure Blob Storage.

This changes where generated image variants live.

Instead of keeping the image cache only on local disk, you can store it centrally in blob storage.

That matters because:

- generated image variants can be shared more easily across instances
- local disk becomes less important
- cloud hosting becomes simpler
- CDN caching of generated variants becomes more predictable

Two different caches here

It is easy to confuse these:

- blob metadata cache
- ImageSharp file cache

They are not the same thing.

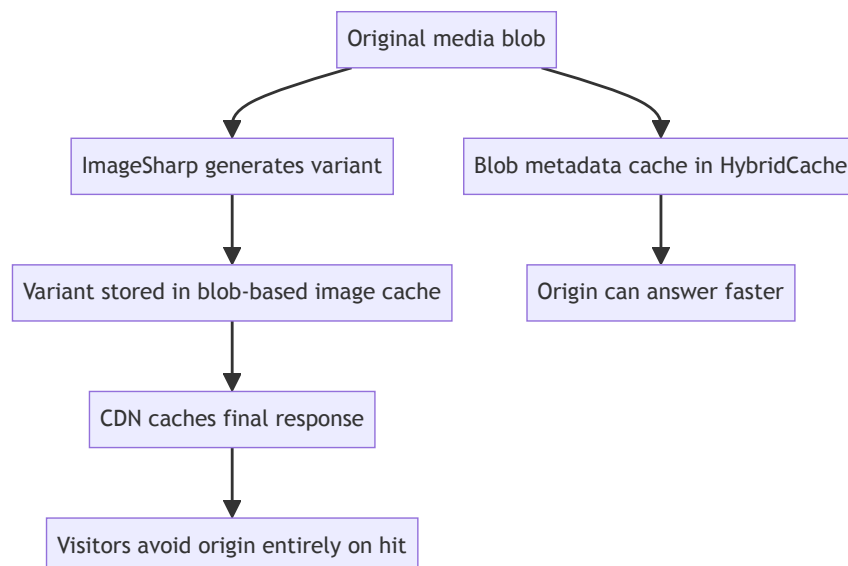
Blob metadata cache:

- small metadata values
- stored in `HybridCache`
- used to avoid repeated Azure property lookups

ImageSharp cache:

- actual generated image files
- stored in blob storage
- used to avoid regenerating image variants repeatedly

Media caching stack



How this differs from published content cache

This is worth stating plainly.

Published content cache is about things like:

- content nodes

- routes
- navigation
- culture variants
- publish state

Storage Providers media caching is about:

- media URL delivery
- file metadata lookup
- generated image variants
- CDN edge responses

So both are "cache", but they solve very different bottlenecks.

Where cache is created

For Storage Providers, cache is created in different places:

- at the CDN edge for media responses
- in `HybridCache` for Azure blob metadata
- in blob storage for generated ImageSharp cache files

This is a good reminder that not all Umbraco-related caching is in-memory inside the CMS process.

Where cache is busted

The most concrete busting rule we have from the package is this:

- writes through the Azure Blob file system invalidate the affected metadata cache entries immediately[^09-bust]

For the other layers:

- CDN cache busting depends on URL uniqueness, expiry, and CDN configuration
- ImageSharp cache freshness depends on how variants are addressed and stored

So the media side often relies more on:

- key design
- path design
- query-string design
- remote cache expiry

than on the classic Umbraco cache refresher pipeline.

Practical lesson

If content caching is about "which published data is safe to serve?",

media caching is more about:

- "where does the file live?"
- "can we avoid remote metadata calls?"
- "have we already generated this image variant?"
- "can the CDN answer this without touching Umbraco?"

That is why Storage Providers deserves its own chapter.

One-sentence summary

`Umbraco.StorageProviders` connects Umbraco to CDN caching, Azure blob metadata caching via `HybridCache`, and blob-backed ImageSharp cache storage, making it a media-delivery caching story rather than a published-content caching story.

Sources

- Docs:
 - [Umbraco Storage Providers](#)
- Source:
 - [Umbraco.StorageProviders on GitHub](#)

[^09-three]: See [U14 in the appendix](#) and [S1](#). [^09-hybrid]: See [S1](#) and [M2](#). [^09-bust]: See [S1](#).

10. Future Hybrid Cache Architecture

This chapter is one of the main pillars of the book.

It is based on the current `main` branch implementation in:

- `src/Umbraco.PublishedCache.HybridCache`

That matters because this is where we can see the cache model becoming more explicit and more ambitious than the earlier "HybridCache plus a few settings" story.

The most important idea

The future Hybrid Cache is not one cache.

It is a layered published-content system with at least four different responsibilities:

1. a tiny local hot cache of already converted published objects
2. serialised `HybridCache` entries
3. a database-backed source cache
4. notification-driven refresh, rebuild, and seeding workflows

That is why it is worth studying as a system, not just as a library call.

Start with Microsoft

Before we talk about Umbraco, we should be clear about the base layer.

Microsoft's official `HybridCache` is the cache primitive Umbraco is building on.^[^10-msbase]

From the broader Microsoft caching guidance, the relevant stack is:

- `IMemoryCache`
- `IDistributedCache`
- `HybridCache`

For Umbraco, the important point is not to memorise every Microsoft cache API.

It is to understand that `HybridCache` is Microsoft's higher-level cache layer that sits above plain memory caching and raw distributed-cache storage.

From Microsoft's official docs and blog, `HybridCache` provides:

- a unified API for in-memory and distributed caching
- primary local caching plus optional secondary distributed caching
- stampede protection
- configurable serialisation
- tag-based invalidation
- configurable entry options for overall and local cache duration

So for this book, Microsoft is not just background context.

Microsoft is a primary source for understanding the base engine.

What is relevant for Umbraco, and what is not

Not every Microsoft caching detail matters equally for this book.

The Microsoft topics most relevant to Umbraco are:^[^10-relevant]

- the difference between plain memory cache and hybrid cache
- the optional secondary distributed cache layer
- stampede protection

- tag-based invalidation
- entry durations for local and remote layers
- serialiser hooks

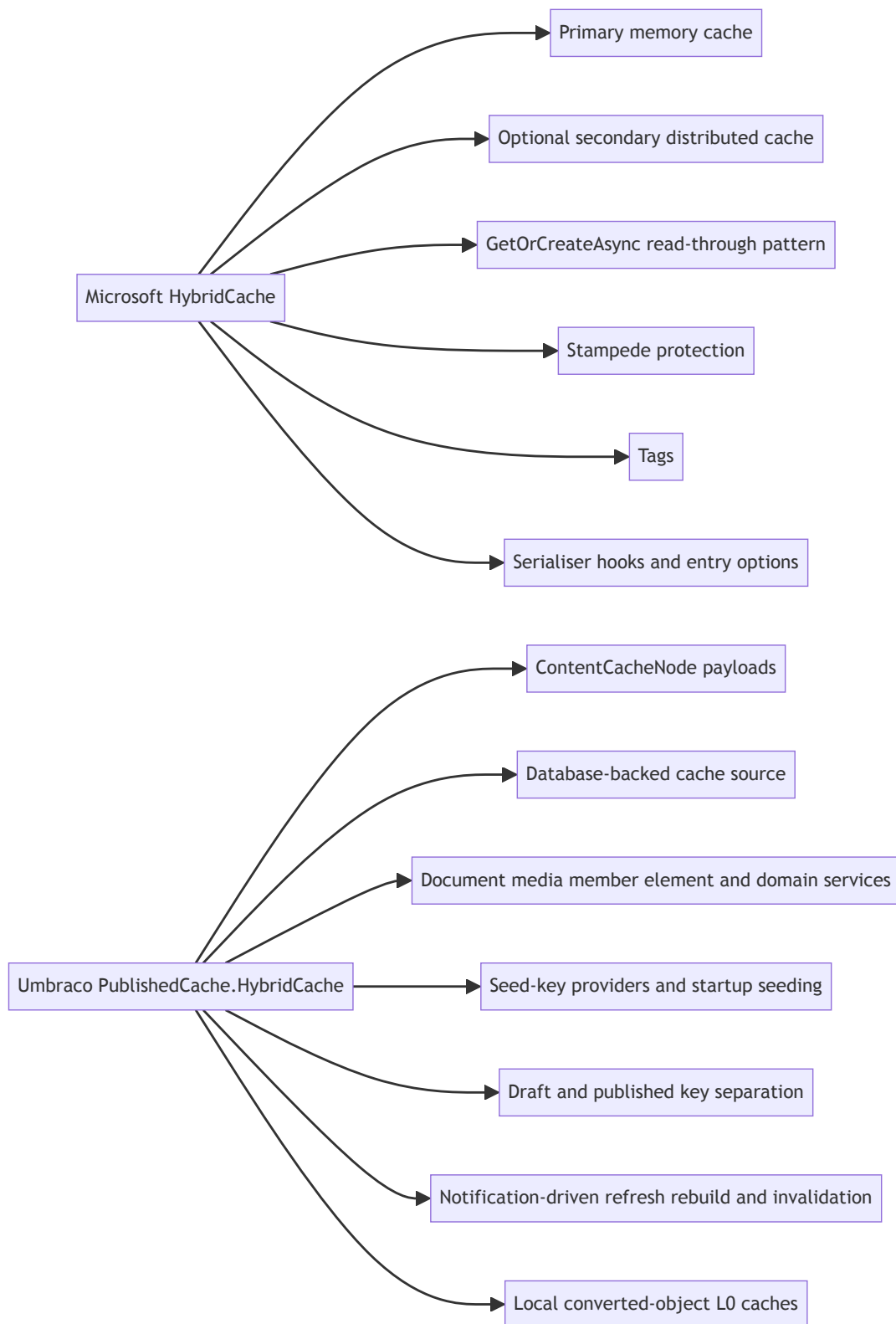
The less relevant topics for this book are:

- generic one-off examples unrelated to published content
- caching patterns that do not interact with Umbraco's published-content pipeline
- broad .NET caching guidance that never changes how Umbraco actually behaves

So when we use Microsoft sources, we should keep filtering them through one question:

"Does this help explain Umbraco's actual cache behaviour?"

What comes from Microsoft, and what Umbraco adds

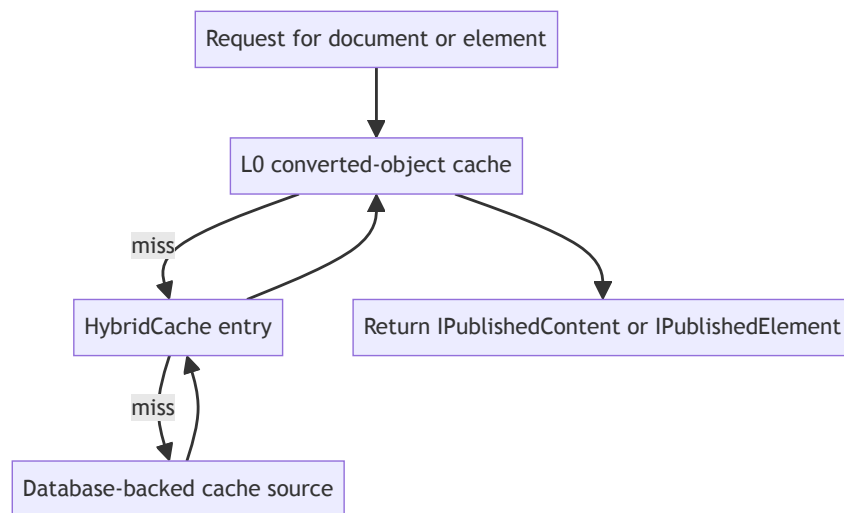


Microsoft HybridCache in one sentence

Microsoft `HybridCache` is the engine.

Umbraco's Hybrid Cache module is the published-content system built around that engine.^[10-umbracoblog]

The four layers



Why this is more than "memory plus Redis"

When people hear `HybridCache`, they often imagine:

- local memory
- remote cache

That is true, but incomplete.

In Umbraco's future model, Hybrid Cache also sits between:

- a high-speed cache of already materialised published objects
- and a database cache repository that stores serialised published data ready for read-through loading

So the architecture is really:

- L0 converted objects
- L1 and optional L2 inside `HybridCache`
- database-backed cache source

The Microsoft layer mainly gives Umbraco the middle line.

Umbraco adds the top and bottom lines.

That is also why Umbraco's own `DistributedCache` should not be confused with Microsoft's distributed cache layer.

- Microsoft distributed cache is about shared cache storage
- Umbraco distributed cache is about telling servers what to refresh or remove

Umbraco can use Microsoft's distributed storage layer underneath `HybridCache`, while still using its own distributed invalidation layer for correctness across servers.

The caches Umbraco registers

The `AddUmbracoHybridCache()` builder method registers first-class caches for: [^10-registers]

- documents
- media
- members
- elements
- domains

It also wires in:

- cache seeding
- notification handlers

- rebuild services
- serialiser selection
- database cache rebuild logic

That is a strong sign that this is the future centre of the published-cache subsystem, not a side experiment.

The local L0 hot cache

One of the most important implementation details is easy to miss.

`DocumentCacheService` and `ElementCacheService` both keep a local converted-object cache:[^10-10]

- `ConvertedPublishedContentCache<string, IPublishedContent>`
- `ConvertedPublishedContentCache<string, IPublishedElement>`

This is not just another generic cache layer.

It is a very deliberate optimisation for the render hot path.

The idea is:

- serialised cache nodes are good for broad cache storage
- but once an `IPublishedContent` or `IPublishedElement` has already been built, returning it from an in-process dictionary is even cheaper

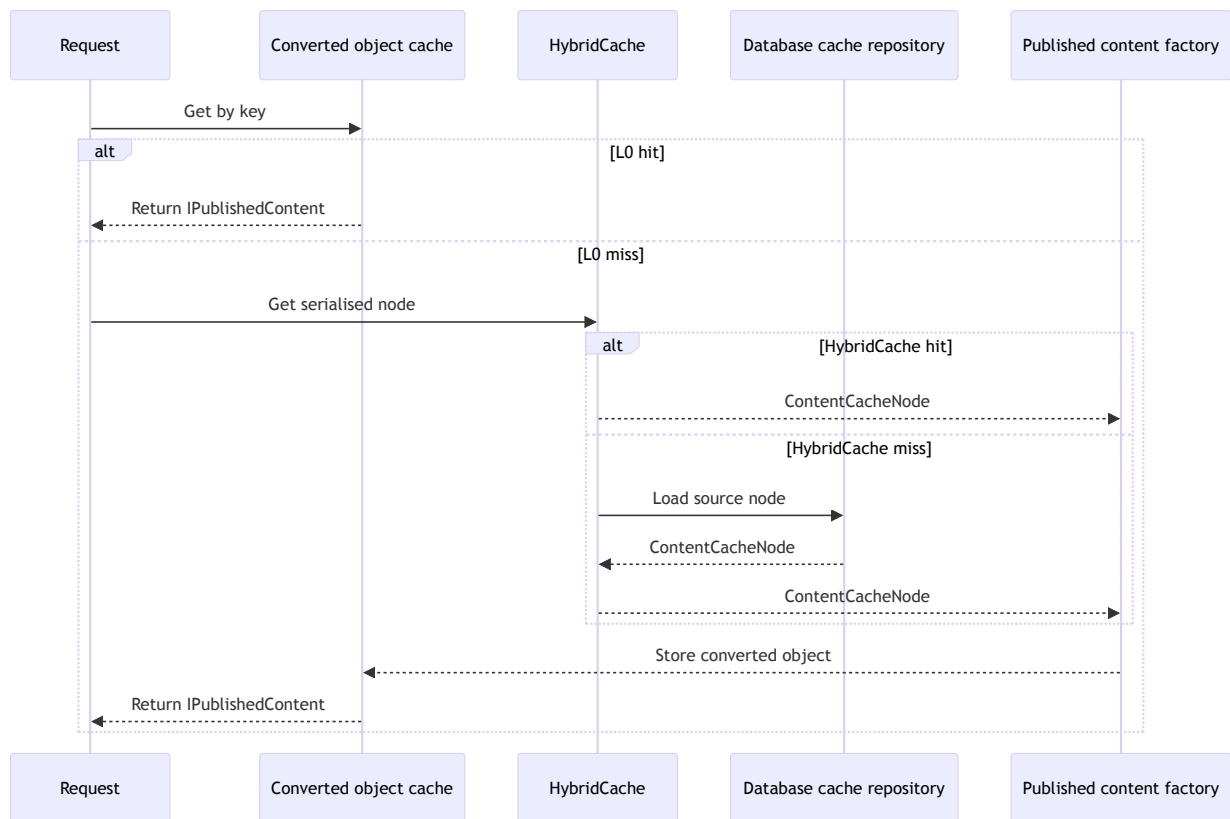
So the future model is not only about remote caching.

It is also about keeping already materialised objects in a tiny fast lane.

Request flow for a document

The document lookup path is roughly:

1. decide preview or published mode
2. try the L0 converted-object cache
3. try `HybridCache`
4. on miss, read from `DatabaseCacheRepository`
5. if safe, write the serialised node into `HybridCache`
6. convert it into `IPublishedContent`
7. store the converted object in the L0 cache



Preview and published are deliberately separate

The code keeps separate cache keys for preview and published entries.

The key format is effectively:

- published: `guid`
- preview: `guid+draft`

That matters because draft and published payloads are not the same thing.

This is a nice example of correctness beating simplistic cache-sharing.

The database-backed cache source

This is one of the most important pieces in the future design.

`DatabaseCacheRepository` stores and reads serialised content data from the database cache table.

So on a miss, Umbraco is usually not rebuilding everything from the full content graph in real time.

Instead, it reads from a prepared database-backed cache source and then repopulates the in-memory layers.

This is a huge clue for the book:

- Hybrid Cache is not "database or cache"
- it is "database cache source feeding runtime caches"

Why serialiser choice matters

The future Hybrid Cache is built around serialised cache nodes.

The module registers a custom `HybridCacheSerializer` using:

- `MessagePack`
- `LZ4` compression

That matters for two reasons:

1. payload size matters a lot in Umbraco
2. the database cache format can become incompatible when serialiser strategy changes

The startup handler even checks whether the configured serialiser changed and can rebuild the database cache when needed.[^10-serialiser]

So serialisation is not just an implementation detail.

It is part of cache correctness.

This also maps neatly onto the Microsoft design.

Microsoft provides the serialiser hook.

Umbraco chooses a payload format better suited to published content by using MessagePack and LZ4 for `ContentCacheNode` .

Seeding is first-class, not an afterthought

The future model treats seeding as a normal startup workflow.

At application start, Umbraco can seed:

- documents
- media
- elements

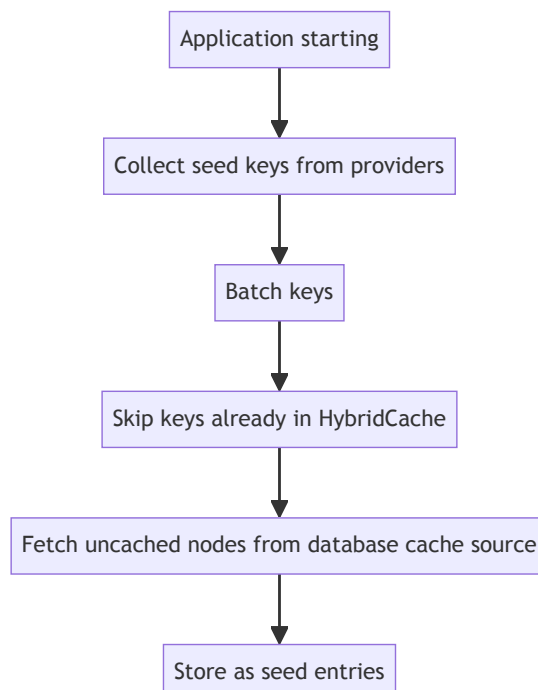
using seed-key providers such as:

- breadth-first providers
- content-type providers

The key idea is simple:

- not everything needs to start hot
- but some things should start hot on purpose

Seeding flow



Seed entries and normal entries have different durations

This is a subtle but important detail.

The code gives seeded entries their own cache durations.

That means Umbraco can say:

- "this key is intentionally hot"
- "keep it around longer"

without forcing the same lifetime onto every other entry.

That is much more deliberate than the older all-content-hot mental model.

Cache busting in the future model

This is the part your book especially cares about.

The future Hybrid Cache has several different busting paths.

1. Per-item refresh

When content, media, or elements refresh:

- the database cache source is updated
- draft and published entries are updated or removed as needed
- the converted-object cache is cleared for the affected key

2. Tag-based invalidation

Cache entries are tagged, for example by: `^10-tags`

- broad content or element tags
- content type or element type tags

That allows bulk invalidation such as:

- clear all document entries
- clear all element entries
- clear everything for a changed content type

3. Full memory clear and reseed

Some operations clear a whole area of cache and then seed again.

4. Rebuild after structural changes

Content type and media type structural changes can require rebuilding the database cache source and then clearing the matching runtime layers.

Tag-based invalidation is a big deal

The code uses cache tags to avoid overly blunt invalidation.

For example:

- documents always carry a broad content tag
- document entries can also carry `ct:{contentType}`
- elements always carry a broad element tag
- element entries can also carry `et:{elementType}`

That means cache busting can be:

- broad when needed

- selective when possible

This is one of the biggest reasons the future model deserves to be called smarter rather than merely newer.

It is also one of the clearest examples of the split between Microsoft and Umbraco.

Microsoft provides the tag mechanism.

Umbraco decides what those tags mean in content terms:

- all documents
- all elements
- specific content types
- specific element types

Negative-cache entries are treated carefully

There is a beautiful little correctness detail in the code.

Even null entries can get tags.

Why?

Because if Umbraco caches "not found" and then later clears by tag, those negative-cache entries must be evicted too.

Otherwise "not found" can become permanently stale.

That is the sort of edge-case engineering that makes the architecture trustworthy.

The stale-set clobber problem

This is one of the most interesting implementation details in the whole module.

`DocumentCacheService` and `ElementCacheService` keep a monotonic cache-generation counter.[^10-generation]

Why?

Because a request can start reading old data just before a concurrent publish or refresh updates the cache.

Without protection, that older in-flight read could finish later and write stale data back over the fresher entry.

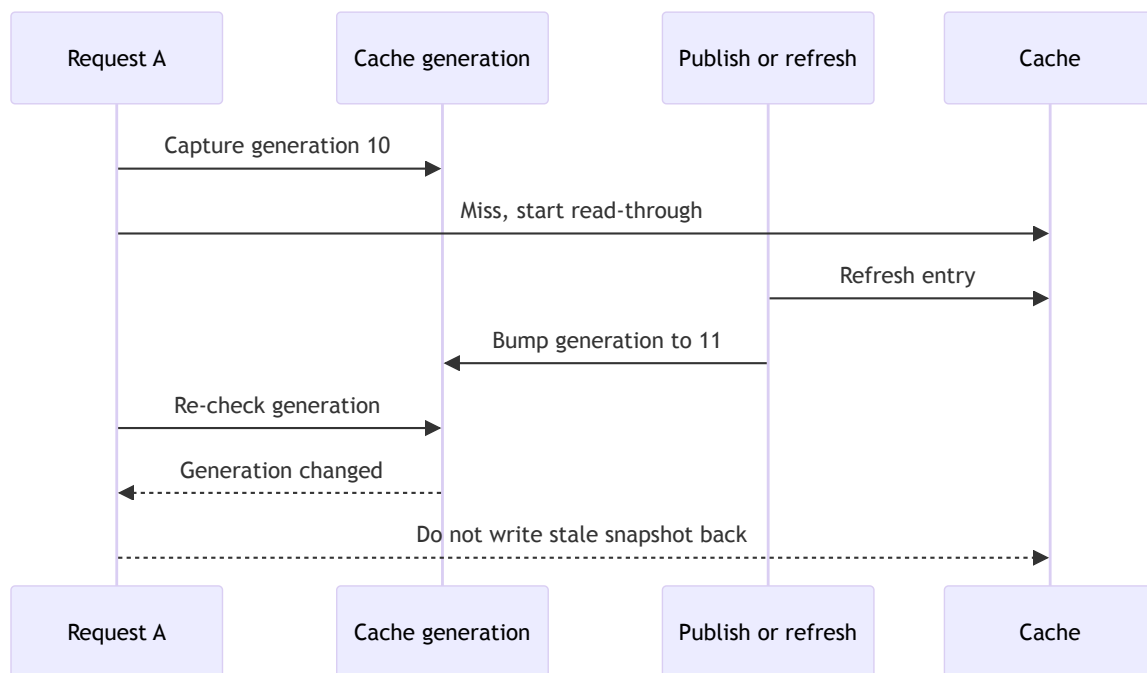
The generation counter prevents that.

If the generation changed while the request was doing its read-through work:

- the read result is still returned to that request
- but it is not written back into the cache

That is an excellent example of a subtle bug being solved at the architecture level.

Stale-set protection



Structural vs non-structural content-type changes

Another major improvement is that the code treats content-type changes more intelligently.

For non-structural changes such as:

- name
- icon
- description

Umbraco can often keep the serialised cache entries and just clear converted content.

For structural changes such as:

- removed properties
- alias changes
- variation changes

Umbraco rebuilds the deeper cache source.

That is a much more precise model than "content type changed, clear everything".

Immediate vs deferred rebuild

The future model supports deferred rebuild for structural type changes.

Why defer?

Because rebuilding inside the original save transaction can create lock contention.

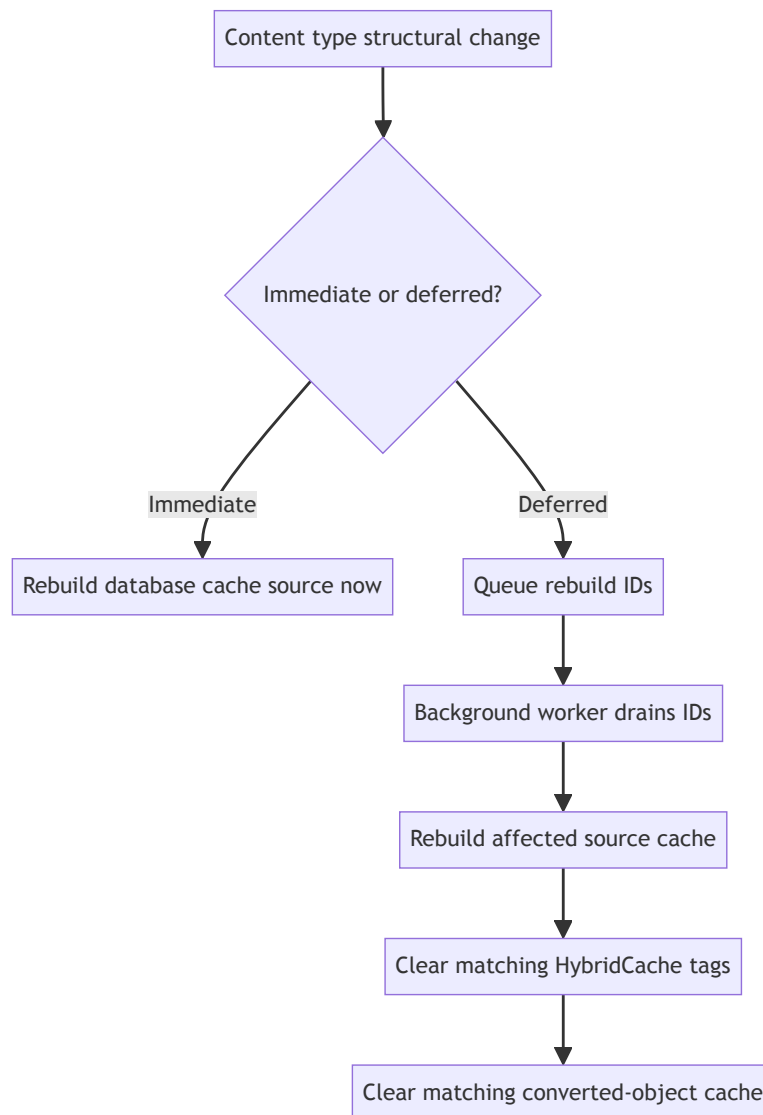
So the code can:

- queue affected IDs
- deduplicate them
- rebuild in the background
- then clear matching runtime caches

This is a very pragmatic trade-off:

- correctness still matters
- but availability and responsiveness matter too

Rebuild flow



Serializer change can trigger full database cache rebuild

One more future-facing detail matters a lot.

If the configured serialiser changes, Umbraco can rebuild the whole database cache source at startup.

That tells us something important:

- the database-backed cache source is a real cache artefact with a format contract
- changing that format is a rebuild event, not a tiny setting tweak

Why this changes performance advice

The old mental model encouraged people to assume lots of content was already hot.

The future Hybrid Cache still gives you hot paths, but it rewards more discipline:

- seed deliberately
- avoid broad traversal when a query would do
- avoid repeated lazy graph walking
- understand which layer you are actually hitting

In other words:

- startup gets lighter
- memory use gets lower
- but poor query habits become easier to feel

What Microsoft's docs explain well

Microsoft's Learn documentation is especially useful for the generic behaviour:[^10-msdocs]

- how primary and secondary caches interact
- how `GetOrCreateAsync` works
- how stampede protection combines concurrent misses for one key
- how serialisers and tags are configured

The wider Microsoft caching page is also useful for one framing idea:

- `HybridCache` is the modern higher-level cache option
- `IMemoryCache` and `IDistributedCache` are the lower-level building blocks beneath it

Umbraco source is then the best place to learn the content-specific behaviour:

- which keys are cached
- how seed keys are chosen
- how content-type rebuilds work
- how draft and published entries differ
- how cache busting is mapped onto content notifications

The best beginner mental model

If you want one simple sentence:

Umbraco's future Hybrid Cache is a staged delivery pipeline for published data, not a single bucket of cached objects.

And the sentence before that is:

Microsoft built the cache engine; Umbraco built a published-content pipeline around it.

One-sentence summary

The future Hybrid Cache combines a database-backed cache source, serialised `HybridCache` entries, local converted-object hot caches, seeding, tagging, and notification-driven rebuild logic into a much more deliberate published-data pipeline than the old all-hot mental model.

Sources

- Microsoft:
 - [Caching in .NET](#)
 - [HybridCache in ASP.NET Core](#)
 - [Caching overview in ASP.NET Core](#)
 - [HybridCache is now GA \(blog post\)](#)
 - [HybridCacheEntryOptions API](#)
 - [HybridCacheOptions API](#)
- Code:
 - `umbraco-main/src/Umbraco.PublishedCache.HybridCache/DependencyInjection/UmbracoBuilderExtensions.cs`
 - `umbraco-main/src/Umbraco.PublishedCache.HybridCache/Services/DocumentCacheService.cs`
 - `umbraco-main/src/Umbraco.PublishedCache.HybridCache/Services/ElementCacheService.cs`
 - `umbraco-main/src/Umbraco.PublishedCache.HybridCache/NotificationHandlers/CacheRefreshingNotificationHandler.cs`

- `umbraco-`
`main/src/Umbraco.PublishedCache.HybridCache/NotificationHandlers/DeferredCacheRebuildNotificationHandler.cs`
- `umbraco-main/src/Umbraco.PublishedCache.HybridCache/Services/DeferredCacheRebuildService.cs`
- `umbraco-main/src/Umbraco.PublishedCache.HybridCache/DatabaseCacheRebuilder.cs`
- `umbraco-main/src/Umbraco.PublishedCache.HybridCache/Serialization/HybridCacheSerializer.cs`
- `umbraco-main/src/Umbraco.PublishedCache.HybridCache/Persistence/DatabaseCacheRepository.cs`

[^10-msbase]: See [M1](#), [M2](#), and [M4](#). [^10-relevant]: See [M1](#), [M2](#), [M5](#), and [M6](#). [^10-registers]: See [C4](#) and [C5](#). [^10-l0]: See [C4](#). [^10-serialiser]: See [C4](#), [M2](#), and [M6](#). [^10-tags]: See [M2](#), [M4](#), and [C4](#). [^10-generation]: See [C4](#). [^10-msdocs]: See [M1](#), [M2](#), and [M3](#). [^10-umbracoblog]: See [B1](#), [B2](#), [B3](#), and [B6](#).

11. Appendix: Sources

This appendix is the book's source register.

The main chapters use footnotes to point here when a claim depends on a specific source or source family.

How to read this appendix

- Microsoft sources explain the base `HybridCache` platform.
- Umbraco docs explain supported behaviour and configuration.
- Umbraco source code shows how the platform is actually wired together.
- Supporting talks and articles help explain trade-offs and mental models.
- Local decompilation notes are used where the official package source was not directly available.

Microsoft sources

M1. Caching in .NET

- [Caching in .NET](#)

M2. ASP.NET Core HybridCache

- [HybridCache in ASP.NET Core](#)

M3. ASP.NET Core caching overview

- [Caching overview in ASP.NET Core](#)

M4. Hybrid Cache is now GA

- [HybridCache is now GA \(blog post\)](#)

M5. `HybridCacheEntryOptions`

- [HybridCacheEntryOptions API](#)

M6. `HybridCacheOptions`

- [HybridCacheOptions API](#)

Umbraco docs

U1. Caching overview

- [Caching overview \(v17\)](#)

U2. Response caching

- [Response caching \(v17\)](#)

U3. Website output caching

- [Website output caching \(v17\)](#)

U4. Cache settings for Umbraco 17

- [Cache settings \(v17\)](#)

U5. Current cache settings page

- [Cache settings \(latest\)](#)

U6. Server-side extensions cache docs

- [Server-side cache extensions \(v17\)](#)

U7. Application cache docs

- [Application cache \(v17\)](#)

U8. Cache seeding docs

- [Cache seeding \(v17\)](#)

U9. Custom seed-key provider example

- [Custom seed key provider example](#)

U10. Tags example

- [Cache tags example](#)

U11. Deploy: handling cache refresher notifications

- [Handling cache refresher notifications \(Deploy\)](#)

U12. Deploy settings

- [Deploy settings](#)

U13. Deploy troubleshooting

- [Deploy troubleshooting](#)

U14. Storage Providers docs

- [Umbraco Storage Providers](#)

U15. Examine overview

- [Examine](#)

U16. Examine management

- [Examine management](#)

U17. Examine `ISearcher` API

- [Examine `ISearcher` API](#)

Umbraco source code

C1. Umbraco 17 source checkout

- `umbraco-v17` cloned from `release/17.5.1`

C2. Umbraco 18 source checkout

- `umbraco-v18` cloned from `release/18.0.2`

C3. Umbraco main branch source checkout

- `umbraco-main`

C4. `Umbraco.PublishedCache.HybridCache` on main

- `umbraco-main/src/Umbraco.PublishedCache.HybridCache`

C5. `CLAUDE.md` for `Umbraco.PublishedCache.HybridCache`

- `umbraco-main/src/Umbraco.PublishedCache.HybridCache/CLAUDE.md`

C6. Website output-cache implementation

- [umbraco-v17/src/Umbraco.Web.Website/Caching](#)

C7. Core cache types and refreshers

- [umbraco-v17/src/Umbraco.Core/Cache](#)

Storage Providers source

S1. [Umbraco.StorageProviders](#) repository

- [Umbraco.StorageProviders on GitHub](#)

Umbraco blog posts

B1. Umbraco 15 Release

- [Umbraco 15 release](#)

B2. Umbraco 15 Release Candidate

- [Umbraco 15 release candidate](#)

B3. Umbraco 17 Beta is out

- [Umbraco 17 beta](#)

B4. Umbraco 18 Release Candidate

- [Umbraco 18 release candidate](#)

B5. Umbraco Product Update - August 2024

- [Umbraco product update — August 2024](#)

B6. Umbraco Product Update - Q1 2025

- [Umbraco product update — Q1 2025](#)

B7. Umbraco 12.3 release

- [Umbraco 12.3 release](#)

B8. Umbraco Product Update - November 2023

- [Umbraco product update — November 2023](#)

Supporting talks, PDFs, and explainer articles

T1. Releasing HybridCache into the Wild with Umbraco

- [Hybrid Cache förändrar allt — Umbraco Kalaset session \(YouTube\)](#)

T2. Hybrid Cache förändrar allt PDF

- [Hybrid Cache förändrar allt — Umbraco Kalaset slides \(PDF\)](#)

T3. Microsoft-side HybridCache explainer by Milan Jovanovic

- [HybridCache in ASP.NET Core — Milan Jovanović](#)

T4. Enkelmedia article

- [Hybrid Cache Changes Everything — Codegarden talk \(Enkelmedia blog\)](#)

T5. Enkelmedia PDF

- [Hybrid Cache Changes Everything — Codegarden slides \(PDF\)](#)

Community reports and issue threads

F1. Website significantly slower since upgrading from v13 to v16

- [Website significantly slower since upgrading from v13 to v16 \(Umbraco forum\)](#)
- Specific reply referenced in chapter 6:
 - [Website significantly slower — comment #11](#)

F2. Failed to acquire write lock for id: -333

- [Failed to acquire write lock for id: -333](#)

F3. `umbraco/oauth_complete?code=...` stuck after `umbraco/logout`

- [umbraco/oauth_complete?code=...](#) stuck after `umbraco/logout`

F4. Open cache issue survey (v17/v18)

Survey backing [chapter 14](#). All issues were read in full, including comments, on 30 June 2026. State is captured as of that date.

URL and routing cache (`DocumentUrlService`):

- Published documents losing their links
- `umbracoUrlAlias` saved values breaking published routing
- Cannot change a content item's `Key` because of the `umbracoDocumentUrl` foreign key
- `DocumentUrlService` `SqlBulkCopy` runs while a `DataReader` is open
- Preview does not refresh for a custom `UrlProvider`

Cold start, upgrade, and migration:

- Pages 404 after upgrade migrations
- Content blank after 13.13 to 17.2.2 upgrade
- Cold boot rebuilds `Examine` on every Azure Linux restart
- Always loading content from the database on Azure Linux App Service
- Content not displayed after 17.1 to 17.2 upgrade (closed)

Distributed invalidation and load balancing:

- Publisher stops writing cache instructions after failing to acquire `MainDom`
- Transient Delivery API 404s during Save and Publish in a split v17 setup
- Unpublished content can be reached / publish notifications race the cache
- Index rebuild does not propagate to front-end servers

HybridCache internals — locking, traversal, and config:

- v15+ sitemap approach warms the whole HybridCache
- `Failed to acquire read lock for id: -332` from HybridCache
- Rapid content-type updates deadlock the site
- Option to disable the members database cache

Historical NuCache `.db` pain:

- Process cannot access `NuCache.Content.db` during upgrade
- NuCache `.db` (de)serialisation allocates large amounts of memory at start-up

F5. Cache-related fixes and pull requests

Merge states verified against the GitHub API on 30 June 2026.

- Prevent changing the `Key` property on existing entities (merged into v18)
- Published Content Cache: defensive hardening against race conditions (merged)

- [Add a scope-level cache version tier to reduce database hits in bulk operations](#) (merged into v17)
- [Reduce NuCache memory allocations with `StringPool` / `ArrayPool`](#) (closed without merging)

Local Deploy package inspection

D1. Decompiled OnPrem composer

- `/Users/ted/Downloads/Umbraco.Deploy.OnPrem/UmbracoDeployOnPremComposer.cs`

D2. Decompiled Umbraco builder extension

- `/Users/ted/Downloads/Umbraco.Extensions/UmbracoBuilderExtensions.cs`

D3. OnPrem package DLL

- `/Users/ted/Downloads/umbraco.deploy.onprem.18/lib/net10.0/Umbraco.Deploy.OnPrem.dll`

D4. Full temporary decompilation output

- `/tmp/umbraco-deploy-onprem-decompiled/Umbraco.Deploy.OnPrem.decompiled.cs`

12. NuCache vs Hybrid Cache

This chapter exists to stop one of the easiest confusions in the whole Umbraco cache story.

In Umbraco 17, `NuCache` is both:

- an older architectural name
- and a set of names, settings, comments, and compatibility seams that still live inside the newer system

So if you read the code casually, it can look as if NuCache and Hybrid Cache are the same thing.

They are not.

The shortest true answer

Old NuCache is the earlier published-content cache architecture.

Hybrid Cache is the newer published-content cache architecture.

In Umbraco 17, the active implementation is Hybrid Cache, but NuCache terminology still survives in:

- configuration names
- serialiser names
- SQL template names
- comments
- some refresh APIs such as `RefreshAllPublishedSnapshot`

Why the confusion happens

The confusion is not your fault.

The v17 source itself still mixes the names in several places:

- the settings model is still `NuCacheSettings` [¹²-settings]
- the serialiser enum is still `NuCacheSerializerType` [¹²-serializer]
- `AddUmbracoHybridCache()` is the real registration method, but its comments still say "Adds Umbraco NuCache dependencies" [¹²-builder]
- SQL template names still include `NuCache` [¹²-sql]
- distributed refresh helpers still talk about `PublishedSnapshot` [¹²-snapshot]

That tells us Umbraco is in a migration period of vocabulary as well as architecture.

What NuCache used to mean

Historically, NuCache was the published-content cache implementation used in older Umbraco versions. [¹²-history]

The v17 serialiser enum says this very plainly: [¹²-legacy]

- the JSON option existed for backward compatibility with the cache implementation used from Umbraco 8 to 14
- the newer Hybrid Cache implementation has been in place from Umbraco 15 onward [¹²-legacy]

That is one of the clearest statements we have in the code.

What Hybrid Cache means in v17

In v17, the builder pipeline calls:

- `AddUmbracoHybridCache()` [¹²-active]

and the core runtime wiring clearly points at the `Umbraco.PublishedCache.HybridCache` project as the live implementation.

So for the book, the safest beginner phrasing is:

In Umbraco 17, Hybrid Cache is the active published-content cache implementation. NuCache is mostly the historical name that still survives in compatibility seams.

The settings seam

The best example is `NuCacheSettings`.

That class still configures things such as:

- `NuCacheSerializerType`
- `SqlPageSize`
- `UsePagedSqlQuery` [¹²-settings]

But these settings are now feeding the Hybrid Cache implementation.

So `NuCacheSettings` in v17 does **not** mean:

- "the old NuCache engine is still the main published-cache implementation"

It means:

- "the newer Hybrid Cache pipeline still uses a configuration section inherited from the old naming scheme"

The serialiser seam

`NuCacheSerializerType` is even more revealing.

It has:

- `MessagePack`
- `JSON`

But the remarks on `JSON` say:

- it is the legacy serialiser option
- it existed for backward compatibility with the Umbraco 8 to 14 cache implementation
- it is no longer supported with the cache implementation from Umbraco 15 based on .NET's Hybrid Cache
- it remains mainly for a readable format suitable for testing [¹²-legacy]

That gives us a very clean history:

1. older NuCache world
2. newer Hybrid Cache world
3. lingering compatibility vocabulary

The naming seam inside the Hybrid Cache module

One of the funniest details is that the Hybrid Cache builder file still contains comments like:

- "Extension methods ... for the Umbraco's NuCache"
- "Adds Umbraco NuCache dependencies" [¹²-builder]

But the method name is:

- `AddUmbracoHybridCache()`

That is a perfect illustration of the transition.

The code structure has moved first.

Some names are still catching up.

The SQL seam

The database-backed source cache in the Hybrid Cache project still uses SQL template names such as:

- `NuCacheDatabaseDataSource.ContentSourcesSelect`
- `NuCacheDatabaseDataSource.MediaSourcesSelect` [¹²-sql]

Again, that does not mean the old NuCache architecture is still the real engine.

It means the data-source naming has historical continuity.

The refresher seam

Another lingering old-world phrase is:

- `RefreshAllPublishedSnapshot()` [¹²-snapshot]

That method refreshes all published snapshots by triggering:

- all content cache refresh
- all media cache refresh
- all domain cache refresh

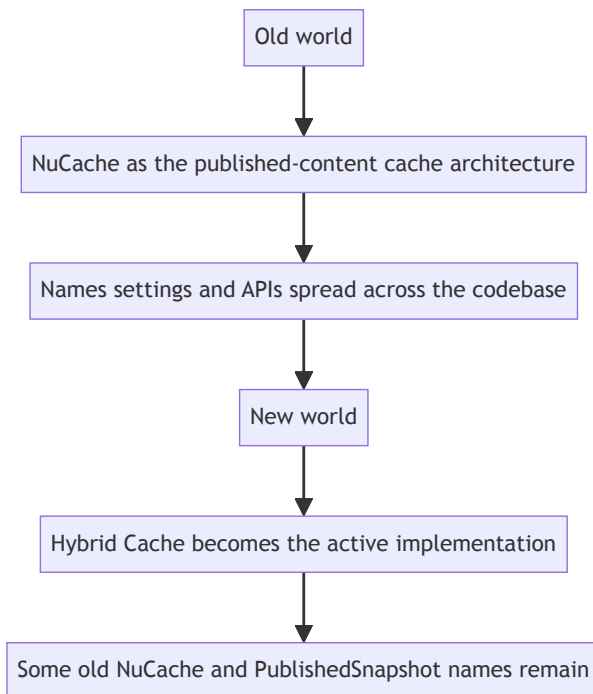
This shows that older published-snapshot language still survives in distributed invalidation APIs.

That is useful for the book because it explains why readers may see:

- NuCache
- PublishedSnapshot
- HybridCache

all in the same codebase and wonder what is going on.

The best mental model



What to teach beginners

If someone is learning Umbraco 17, tell them this:

1. Treat Hybrid Cache as the real current implementation

That is the architecture to understand first.

2. Treat NuCache as the historical predecessor

That is the older mental model.

3. Expect legacy names to remain in the code

Especially in:

- settings
- serialiser options
- SQL names
- comments
- refresh helpers

4. Do not assume every NuCache name means the old engine is still running

Often it means:

- the new engine is using an old config name
- or the old vocabulary was never fully renamed

NuCache and Hybrid Cache are not equal alternatives in v17

This point matters.

For the book, I would not present NuCache and Hybrid Cache as two equally current v17 implementations that you pick between.

That would be misleading.

A better framing is:

- NuCache is the older alternative
- Hybrid Cache is the newer active alternative
- v17 still contains NuCache-named compatibility seams

"But the NuCache settings are still there — isn't it still running?"

This is one of the most reasonable questions you can ask, so let's answer it properly.

It is tempting to picture Umbraco keeping two engines side by side: Hybrid Cache for new projects, and the old NuCache still quietly running for any project that carries NuCache settings. Plenty of frameworks do exactly that kind of thing, so it is a sensible guess.

But it is not what Umbraco does.

In Umbraco 15 and later, the NuCache **engine** was removed. What survives is its **vocabulary**. There is only one published-content cache engine in v17 — Hybrid Cache — and it is the engine for every project, freshly created or upgraded alike.

We can be confident about this straight from the source:[^12-oneengine]

- There is only one published-cache project to find: `Umbraco.PublishedCache.HybridCache`. The old NuCache engine types — the published-snapshot service, the in-memory content store, the BTree-backed local cache file — are simply gone.
- There is only one registration method: `AddUmbracoHybridCache()`. There is no `AddUmbracoNuCache()` to opt back into, so there is no switch to flip.
- Every consumer of `NuCacheSettings` lives *inside* the Hybrid Cache project. Those settings tune Hybrid Cache; they do not choose a different engine.
- Even selecting the `JSON` serialiser does not bring NuCache back. It only picks a legacy, test-only serialisation format inside the Hybrid Cache pipeline.

The version boundary that matters

This is the part worth keeping straight:

- **Umbraco 8 to 14:** NuCache really was the engine.
- **Umbraco 15 onward:** the NuCache engine was retired and Hybrid Cache took over. The names stayed behind.

So "NuCache is gone" is a statement about v15 and later — not about those older majors, where it was very much alive and doing the work.

What this means in practice

- You cannot "stay on NuCache" in v17, because there is nothing to stay on.
- There is no per-project engine choice. An upgraded site moves onto Hybrid Cache exactly like a brand-new one.
- The NuCache-named settings still do something real — they tune Hybrid Cache (serialiser, SQL paging) — they just do not keep an old engine alive.

A friendly way to remember it:

The NuCache engine retired in v15. Hybrid Cache does the work now — the NuCache name simply kept its old job title on a few doors.

Quick comparison table

Aspect	NuCache (old)	Hybrid Cache (new, active)
Role	Published-content cache architecture	Published-content cache architecture
Era	Umbraco 8 to 14	Umbraco 15 onward (active in 17 and 18)
Built on	Umbraco's own implementation	Microsoft's HybridCache (L1 memory + optional L2 distributed)
Default serialiser	JSON	MessagePack (with LZ4 compression)
Registration	legacy wiring	AddUmbracoHybridCache()
Status in v17	historical predecessor; name survives in seams	the real, active implementation

NuCache-named leftovers in v17 at a glance

This is the "which ones are just names, which ones still affect behaviour" view.

Leftover name	What it really is	Just a name, or real behaviour?
NuCacheSettings	Configuration section now feeding Hybrid Cache	Real settings, old name
NuCacheSerializerType	Serialiser enum (MessagePack / JSON)	Real behaviour, old name
NuCacheSerializerType.JSON	Legacy serialiser for the Umbraco 8 to 14 cache	Real but legacy; kept mainly for readable test output
NuCacheDatabaseDataSource.* SQL templates	Queries for the database-backed cache source	Just historical naming
RefreshAllPublishedSnapshot()	Distributed refresh helper	Real behaviour, old "snapshot" vocabulary
"Adds Umbraco NuCache dependencies" comments	Comments inside AddUmbracoHybridCache()	Just stale comments

One-sentence summary

In Umbraco 17, NuCache is mainly the old published-cache architecture whose names and settings still survive, while Hybrid Cache is the newer active implementation built on Microsoft's HybridCache .

Sources

- Microsoft:
 - HybridCache in ASP.NET Core
- Code:
 - umbraco-v17/src/Umbraco.Core/Configuration/Models/NuCacheSettings.cs
 - umbraco-v17/src/Umbraco.Core/Configuration/Models/NuCacheSerializerType.cs

- `umbraco-v17/src/Umbraco.Web.Common/DependencyInjection/UmbracoBuilderExtensions.cs`
- `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/DependencyInjection/UmbracoBuilderExtensions.cs`
- `umbraco-v17/src/Umbraco.Core/Cache/DistributedCacheExtensions.cs`
- `umbraco-v17/src/Umbraco.Core/Constants-SqlTemplates.cs`

[^12-settings]: See [C1](#) and specifically [NuCacheSettings.cs](#) . [^12-serializer]: See [C1](#) and `NuCacheSerializerType.cs` . [^12-builder]: See [C1](#) and the v17 `Umbraco.PublishedCache.HybridCache/DependencyInjection/UmbracoBuilderExtensions.cs` . [^12-sql]: See [C1](#) and `Constants-SqlTemplates.cs` . [^12-snapshot]: See [C7](#) and `DistributedCacheExtensions.cs` . [^12-legacy]: See [C1](#) and `NuCacheSerializerType.cs` . [^12-active]: See [C1](#), [C4](#), and [M2](#). [^12-history]: See [B7](#), [B8](#), [B1](#), and [B2](#). [^12-oneengine]: See [C1](#) and [C4](#). The v17 source contains a single published-cache project, `Umbraco.PublishedCache.HybridCache` , registered through `AddUmbracoHybridCache()` ; there is no NuCache engine project or alternative registration to fall back to, and the old engine types (published-snapshot service, in-memory content store, BTree local cache) are absent.

13. Examine, Indexes, and Cache-Adjacent Querying

Examine is one of the easiest Umbraco topics to mislabel.

People often ask:

"Is Examine a cache?"

The best beginner-safe answer is:

No, not primarily. Examine is a search index. But it can play a cache-adjacent role because it stores a derived representation of content so expensive lookups do not have to start from the published tree every time.[^13-examine]

That distinction matters more in the HybridCache era.

Why this chapter belongs in a caching book

This book is about caching, but HybridCache changes the query trade-offs enough that we also need one chapter about when not to solve a problem with a cache.

Markus Johansson's HybridCache talk and PDF make that point clearly:

- broad traversal is more expensive than many teams expect
- filtering after loading content can become much more visible
- some workloads are better served by `Examine` than by repeated published-content traversal[^13-talk]

So this chapter is really about choosing the right tool:

- published-content cache for published-content retrieval
- `RuntimeCache` for your own small computed results
- `Examine` for index-shaped query problems

The shortest possible distinction

Published-content cache

This is Umbraco's internal content retrieval layer.

Its job is:

- load published content correctly
- keep repeated content access fast
- stay coherent after invalidation

`RuntimeCache`

This is your small app-level cache.

Its job is:

- store a computed result
- avoid recomputing it for every request
- clear it when the underlying data changes

Examine

This is a search/indexing system built on Lucene.[^13-examine]

Its job is:

- store a derived searchable representation
- answer search/filter/query requests efficiently
- avoid walking large parts of the content tree just to find matches

Why Examine can feel cache-like

Even though Examine is not Umbraco's general-purpose cache, it does share some cache-like properties:

- it stores derived data rather than the original source object
- it is built so repeated queries become cheap
- it must be refreshed or rebuilt when underlying content changes

That is why it is fair to call it cache-adjacent.

But "index" is still the better primary word.

If you call Examine a cache without qualification, beginners may assume it behaves like:

- `RuntimeCache`
- `HybridCache`
- `IMemoryCache`

and that will lead them toward the wrong mental model.

Searchers are the clue

The Umbraco docs describe Examine management in terms of `indexes` and `searchers`.^[13-searchers]

That wording is revealing.

A searcher is not "the cache reader".

It is the component that queries one or more indexes.

That makes Examine much closer to:

- a search engine
- a read model
- a precomputed query structure

than to a simple in-memory cache entry store.

When Markus recommends Examine instead of cache

The `HybridCache` material is helpful because it does not say "cache more".

Instead, it often says:

- stop broad tree traversal
- stop loading lots of content just to filter it afterwards
- query more intentionally

That is where Examine comes in.^[13-talk]

For some workloads, the best answer is not:

- "put the result in `RuntimeCache` "

but:

- "query an index that already knows how to find the right subset"

Examples Markus calls out include cases like sitemap generation and content selection patterns where a query can be expressed more naturally as indexed retrieval than as full traversal and filtering.^[13-talk]

A practical decision rule

Ask this question first:

Am I trying to remember a small result, or am I trying to find the right items across a larger set?

If the answer is "remember a small result", a cache is often right.

If the answer is "find the right items", an index is often right.

Quick comparison

Problem	Best fit	Why
Reuse a small computed object	RuntimeCache	You already know the shape of the answer and just want to avoid recomputing it.
Load published content by route or id	Published-content APIs	This is the platform's native retrieval path.
Find many matching items across a large set	Examine	The hard part is discovery, so an index is better than broad traversal.
Cache rendered HTML	Output cache	The expensive step is rendering the page, not looking up content.
Keep one custom app projection warm	RuntimeCache plus invalidation	Good when the source is small and change events are well defined.

The important shape of the decision is this:

- caches remember answers
- indexes help you find answers
- published-content APIs retrieve the source of truth

Troubleshooting checklist

When a feature feels slow, do not jump straight to "add a cache".

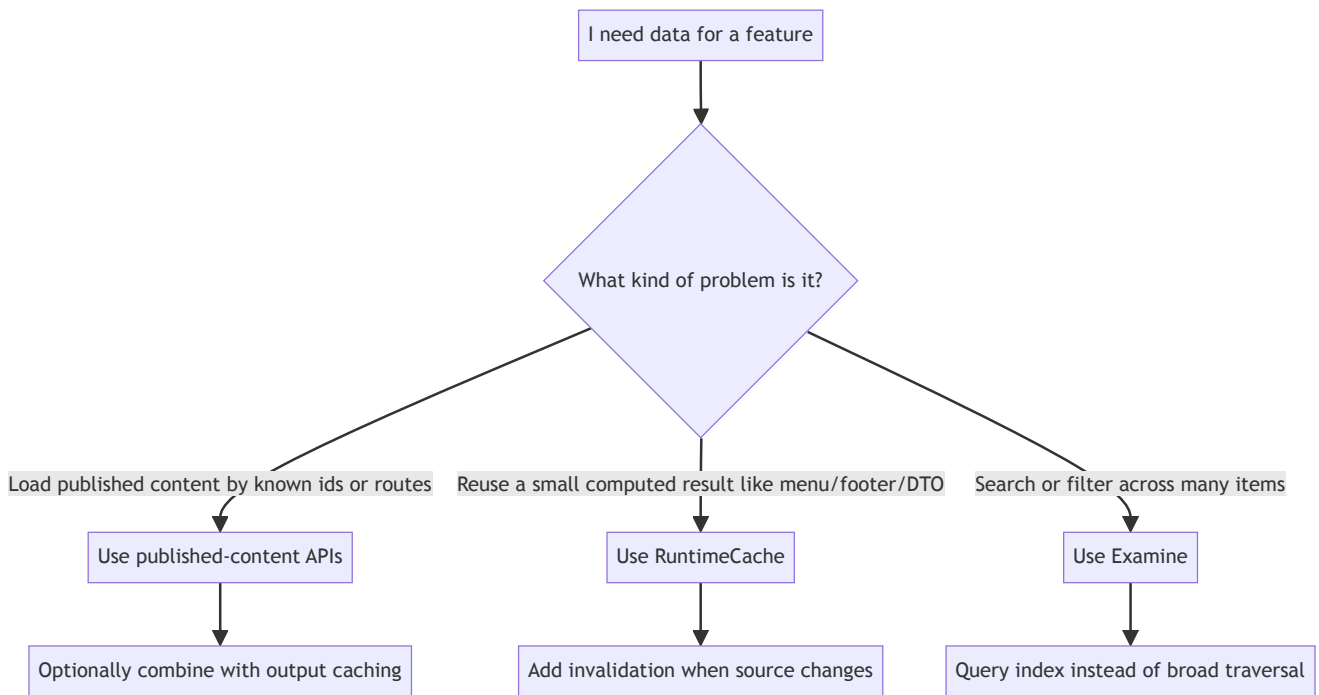
Check these in order:

1. Are you traversing a large part of the tree and filtering afterwards?
2. Are you reusing a computed result that belongs in RuntimeCache ?
3. Are you really trying to search or filter across many items, which is where Examine fits better?
4. Is the problem actually lock contention, rebuild lag, or another operational issue?

That checklist is the practical version of the chapter's main message:

- not every slow query is a cache miss
- not every repeated lookup should become a cache
- not every search problem should be solved by walking content

Decision chart



Good examples for `RuntimeCache`

- menu DTOs
- footer link groups
- a small settings projection
- a compact lookup table built from a known source

These are all cases where your code already knows what data shape it wants and just wants to avoid rebuilding it repeatedly.

Good examples for Examine

- search pages
- filtered listings
- sitemap-like discovery workloads
- "find all matching content" queries across a large set

These are cases where the main cost is not recomputing one tiny DTO.

The main cost is discovering the right set of content in the first place.

Bad pattern to watch for

This is the trap the HybridCache material warns about:

1. load a broad part of the tree
2. materialise many content items
3. filter in memory
4. call that "fine because it is cached"

That may have felt acceptable in older mental models.

It is much less safe in the newer world.

A better sentence for the book

If we want one clean sentence to reuse elsewhere, this is probably it:

Examine is not Umbraco's general-purpose cache, but it is a derived index that can play a cache-adjacent role by making expensive discovery queries cheap enough to avoid broad published-content traversal.[^13-examine]

Relationship to the other chapters

- For the internal published-content cache story, see [03 - Published Content Cache, AppCaches, and Load Balancing](#).
- For the practical query-strategy warning from the HybridCache talk, see [06 - Cache Settings, Talks, and Field Notes](#).
- For the tiny custom cache example using `RuntimeCache`, see [07 - Small Local Cache Example with Tags](#).
- For index refresh behaviour during deployment workflows, see [05 - Extensions and Deploy Cache Busting](#).

Sources

- Docs:
 - [Examine](#)
 - [Examine management](#)
- Supporting material:
 - [Hybrid Cache förändrar allt — Umbraco Kalaset slides \(PDF\)](#)
 - [Examine ISearcher API](#)

[^13-examine]: See [U15](#), [U16](#), and [U17](#). [^13-searchers]: See [U16](#) and [U17](#). [^13-talk]: See [T2](#).

14. Lessons from the Issue Tracker

The earlier chapters explain how Umbraco caching is *meant* to work.

This chapter does the opposite. It looks at what actually breaks.

It is a survey of the open, genuinely cache-related issues on the Umbraco CMS tracker, read in full (body plus comments) and grouped into the patterns that matter for Umbraco 17 and 18.[^14-survey]

How to read this chapter

A bug report is not a source of truth in the same way source code is. So this chapter is careful about confidence:

- **Confirmed** means an Umbraco HQ maintainer acknowledged the cause or behaviour in the thread.
- **Reported** means a community reporter diagnosed it convincingly, but HQ has not yet confirmed it.
- **Unknown** means nobody has identified the cause yet.

Two more caveats:

- This is a snapshot taken on **30 June 2026**. Issue state changes over time, so treat status as "as captured", not "forever".
- Many of these issues are labelled `needs-reproduction`. That is itself a finding: cache bugs are timing- and environment-sensitive, so they are genuinely hard to reproduce on demand.

The headline finding

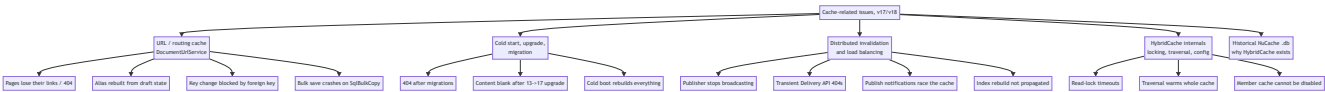
The single most important pattern is this:

Most of the *new* cache problems in v17 are not in `HybridCache` itself. They live in the **routing/URL cache** introduced alongside it: `DocumentUrlService` and the SQL-backed `umbracoDocumentUrl` table.

This layer did not exist in the NuCache era. It is keyed by document GUID, it is refreshed off `ContentTreeChangeNotification` (which fires on **save**, not only on publish), and it has weak draft-versus-published guards in places. That combination explains a whole family of "my page suddenly 404s" reports.

The old NuCache `.db` -file complaints, by contrast, are now historical. They are useful mainly because they explain *why* the architecture moved to `HybridCache` in the first place.

A map of where the problems cluster



Theme 1: the URL and routing cache is the new failure surface

These problems are specific to the v15+/v17 routing layer. They are the ones most likely to surprise someone coming from v13.

Issue	What breaks	Cause	Where it stands
#23234	Published pages randomly lose their backoffice "Links", so the front end 404s. Tends to happen after deploys or overnight restarts.	Unknown. Symptoms point to the <code>DocumentUrl</code> route store going stale or empty across restarts.	Open, needs investigation. Workaround: re-save and re-publish the page.
#23206	Editing a <code>umbracoUrlAlias</code> and only saving (not publishing) can hide the still-published route.	Reported. <code>DocumentUrlAliasService</code> rebuilds aliases on any tree change, guards only <code>Trashed / Blueprint</code> (no <code>Published</code> check), and reads the alias with <code>published: false</code> , so the draft alias overwrites the published one.	Open, needs reproduction.
#21131	Changing a content item's <code>Key</code> throws a SQL foreign-key error (<code>FK_umbracoDocumentUrl_umbracoNode_uniqueId</code>). Breaks tools like uSync that realign keys.	Confirmed. <code>umbracoDocumentUrl</code> deliberately uses the document GUID as its foreign key for performance, and there was no public API to clear those rows.	v17 workaround: delete the rows via the repository first. v18 changes the rules: entity keys are now immutable.
#22782	Bulk save/publish of many items crashes with "There is already an open DataReader".	Reported. During cache refresh, <code>DocumentUrlRepository.Save</code> runs <code>SqlBulkCopy</code> on the shared parent-scope connection while a streaming <code>SqlDataReader</code> is still open.	Open, needs reproduction. Workaround: drain readers (use <code>Fetch / ToList</code> , not lazy <code>Query</code>); suppress <code>ContentTreeChangeNotification</code> during the batch.
#21820	A custom/headless <code>UrlProvider</code> preview does not refresh on a second "Save and Preview".	Reported. The backoffice assumes SignalR will trigger the refresh; headless setups have none, and the preview URL is cached client-side.	Open, reproduced. <i>(This is a backoffice client-side cache, not a server one — a useful contrast case.)</i>

The v18 answer to [#21131](#) is worth singling out, because it is a real architectural decision rather than a patch. Pull request [#21374](#), "Entities: Prevent changing Key property on existing entities", was merged into v18.[^14-fixes] In other words, rather than make the URL cache tolerate key changes, Umbraco made entity keys **immutable**. That tells you how central the GUID-as-foreign-key choice is to the routing cache.

Theme 2: cold start, upgrade, and migration

Here the published cache is left in a bad state at start-up, and a restart or a republish is the only cure.

Issue	What breaks	Cause	Where it stands
#22581	After an upgrade that runs database migrations, every page 404s; publishing warns "the document does not have a URL".	Unknown. HQ could not reproduce.	Open. Workaround: restart the site. Candidate mitigation: the cache-hardening work in #22393 .
#22322	After a 13.13 to 17.2.2 upgrade, content exists in the database but will not render. Only republishing each node fixes it; 17.1.0 is fine.	Reported. A key observation in-thread: programmatic <code>Publish / PublishBranch</code> only push <i>dirty</i> properties, so unchanged properties never reach the published cache.	Open, needs reproduction (regression versus 17.1). Related closed issue: #21863 .
#22947	On Azure Linux App Service, all Examine indexes rebuild on every restart.	Reported. Each container restart gets a new <code>Environment.MachineName</code> , so the "last synced" lookup returns nothing, the boot is treated as <code>SyncBootState.ColdBoot</code> , and everything rebuilds.	Open, reproduced, sprint candidate. Shares its root cause with the published-cache version of the same problem, #13909 .

The "only dirty properties get published" observation in [#22322](#) is the most useful detail in this group. It is a cache-coherence subtlety, not a UI bug: a UI "Save and publish" forces a full overwrite, but a programmatic branch publish may not, so the published cache can carry forward stale values after a major upgrade.

Theme 3: distributed invalidation and load balancing

This is the book's central thesis, restated by real bugs: cache **busting** across servers is the hard part.

Issue	What breaks	Cause	Where it stands
#23219	A publisher that loses the MainDom race at boot applies changes locally but silently stops writing umbracoCacheInstruction rows for the rest of the process. The cluster diverges permanently, with no error logged.	Reported. The broadcast gate depends on a <code>Lazy<SyncBootState?></code> that is evaluated once and caches the failed result forever.	Open, needs reproduction. Only recovery is an app-pool restart. Not load-balancer-exclusive: an overlapped IIS recycle can trigger it on one server.
#22328	In a split v17 deployment (separate Backoffice and Delivery API apps), Save and Publish causes transient Delivery API 404s (about 1 in 1000).	Unknown. There is a brief window during publish where already-published content is not resolvable.	Open. Workaround suggested by HQ: enable Delivery API output caching to mask the gap.
#17393	Publish/unpublish notification handlers read stale content, because the notification fires before the cache reflects the change.	Reported. No notification cleanly signals "the cache is now consistent"; even <code>ContentCacheRefresherNotification</code> can fire early.	Open, reproduced, sprint candidate. Workaround: call <code>DistributedCache.RefreshContentCache(...)</code> explicitly, or read inside a fresh scope.
#8060	The "Rebuild index" button only rebuilds the admin server's Examine index, not the replicas'.	Confirmed. By design: there is no cluster-wide rebuild instruction.	Open, stale, up-for-grabs. Workaround: use a search-as-a-service backend, or schedule rebuilds on every node.

[#23219](#) is the clearest cautionary tale in the whole survey. A node quietly stops emitting cache-busting instructions, so editors see their change on the publishing server only, and nothing in the logs says anything is wrong. It is the precise failure the `DistributedCache` and `IServerMessenger` machinery from [chapter 4](#) exists to prevent — and a reminder that the machinery only helps while it is actually running.

Theme 4: HybridCache internals — locking, traversal, and config gaps

These touch the published cache engine itself.

Issue	What breaks	Cause	Where it stands
#20552	The documented sitemap approach traverses the whole tree, which warms the entire content cache on large sites.	Reported. Full traversal hydrates every node; the lightweight navigation services do not carry enough data (name, dates) to build a sitemap without hydration.	Open. Direction: extend the navigation query services, or build the sitemap at publish time.
#21164	After a deploy, rendering a block throws <code>Failed to acquire read lock for id: -332</code> (ContentTypes).	Unknown. The stack shows <code>HybridCache</code> lazily materialising content during a request, taking the <code>ContentTypes</code> read lock, which then times out under contention (likely a concurrent post-deploy rebuild).	Open, needs reproduction.
#17550	Rapid <code>IContentTypeService.UpdateAsync</code> calls deadlock the whole site.	Reported (HQ hypothesis). A deadlock between Examine indexing and the cache rebuild, both needing the content-type read lock. Database-specific: reproduces on SQLite (where the read lock is nearly a no-op), not on SQL Server.	Open, reproduced. Workaround: throttle the updates.
#23070	On sites with 300k+ members, the database cache rebuild is slow and blocks all member changes, and it cannot be disabled.	Reported. v13's public <code>NuCacheContentRepository</code> (subclassable to skip members) was replaced by an <code>internal sealed DatabaseCacheRepository</code> , removing the extension point.	Open. Requested: a settings flag, or a public repository.

[#20552](#) is the canonical illustration of a recurring theme in this book: **broad traversal has a cost in the HybridCache world**. Walking the whole content tree no longer reads from a fully in-memory structure; it hydrates and caches each node you touch. The lightweight `I*NavigationQueryService` exists precisely so you can walk structure without paying that price — see [chapter 10](#) and [chapter 13](#).

Theme 5: historical NuCache pain, and why HybridCache exists

These two issues predate HybridCache. They are "before" evidence.

Issue	What breaks	Cause	Where it stands
#15634	"The process cannot access NuCache.Content.db " aborts an upgrade. On Azure the temp directory is not accessible to delete the locked file.	Reported. The local BPlusTree .db files are locked by another process during the upgrade.	Open. Workaround: restart the web app to clear the .db files.
#15809	NuCache .db (de)serialisation allocates tens of gigabytes at start-up on large sites (versus roughly 200 MB of files), causing long GC pauses.	Reported. Neither the NuCache serialiser nor the underlying CSharpTest.Net.Collections library pooled their buffers.	Open. A community pooling pull request, #15808 , was closed without being merged , so the underlying cost remains in the NuCache stack.

Read together, these explain the move to HybridCache : local .db files that lock during upgrades and balloon memory at start-up are exactly the fragilities a database-backed, IMemoryCache / IDistributedCache -layered cache was designed to remove. See [chapter 12](#).

The solutions picture

Pulling the fixes and workarounds together gives a clearer view than any single issue.

Shipped and merged

- **v18 — immutable entity keys.** [#21374](#) closes off the key-change foreign-key problem by disallowing key changes outright.[^14-fixes]
- **v17 — published cache race hardening.** [#22393](#), "Published Content Cache: Defensive hardening against race conditions", was merged and is the realistic candidate mitigation for the migration-and-cold-start 404 family.
- **v17 — fewer database hits in bulk operations.** [#22563](#) adds a scope-level cache-version tier to cut repeated database reads during bulk content operations.

Standing workarounds people actually use

- Re-save and re-publish nodes to regenerate lost links or stale content.
- Restart the site after a migration-bearing upgrade.
- Delete umbracoDocumentUrl rows via the repository before changing a key (v17 only).
- Enable Delivery API output caching to mask transient publish-window 404s.
- Use the navigation query services instead of full-tree traversal for sitemaps and similar sweeps.
- Drain data readers (prefer Fetch / ToList over lazy Query) before other database calls in bulk pipelines.
- Use a search-as-a-service backend so load-balanced servers share one index, sidestepping per-server rebuilds.

Not solved

- The NuCache start-up memory cost: the pooling pull request [#15808](#) was closed unmerged.
- Disabling or configuring the member cache at scale ([#23070](#)).
- A cluster-wide Examine rebuild instruction ([#8060](#)).

Cross-cutting patterns

Five lessons recur across the whole survey:

1. **The routing/URL cache is the riskiest new surface.** If a v17 site mysteriously 404s, suspect DocumentUrlService before suspecting HybridCache .
2. **Save is not publish, and the cache knows the difference — sometimes too well.** Several bugs come from draft state leaking into a published cache, or from publish pushing only dirty properties.
3. **Silent divergence is the worst failure mode.** [#23219](#) shows a cluster going wrong with nothing in the logs. Invalidation that fails quietly is worse than invalidation that throws.
4. **Traversal has a cost.** Warming the whole cache by accident is a real, reported performance problem now, not a theoretical one.
5. **Environment shapes behaviour.** Azure Linux container names, SQLite versus SQL Server lock semantics, and split Backoffice/Delivery API deployments each surface bugs that do not appear on a single Windows box with SQL Server.

Honest caveats

- Several causes above are marked **Reported**: convincing community diagnoses that Umbraco HQ has not confirmed. They are flagged deliberately. Do not quote them as settled fact.
- This is a point-in-time read of open issues. Some will be fixed, reclassified, or closed as not-reproducible after this snapshot.
- The survey deliberately excludes issues where "cache" appears only incidentally (for example a NuGet restore failure, or a backoffice asset-loading complaint). The aim was a true cache picture, not a keyword dump.

Best one-sentence summary

The v17/v18 tracker says the published-content engine (`HybridCache`) is largely behaving — and that the hard, live problems are in routing-cache coherence, cold-start population, and keeping invalidation honest across servers.

Sources

- Issue tracker survey captured 30 June 2026; see [F4 in the appendix](#) for the full issue list and [F5](#) for the referenced pull requests.
- Related chapters: [04 - Cache Busting and Invalidation](#), [10 - Future Hybrid Cache Architecture](#), [12 - NuCache vs Hybrid Cache](#), [13 - Examine, Indexes, and Cache-Adjacent Querying](#).

[^14-survey]: See [F4 in the appendix](#). All issues were read in full, including comments, on 30 June 2026. [^14-fixes]: See [F5 in the appendix](#). Pull-request merge states were verified against the GitHub API on 30 June 2026.

15. Reading the Cache Code

Opening the Umbraco source and searching for "cache" returns hundreds of files.

This chapter gives you a map so you can read those files without getting lost.

The two questions that matter

Before diving into types, ask yourself which question you are actually trying to answer:

1. **"Where does Umbraco store cached things?"** — that leads you to `IAppCache` and `HybridCache`.
2. **"How does Umbraco tell all servers a thing changed?"** — that leads you to `ICacheRefresher` and `DistributedCache`.

Most confusing moments in the source code happen because these two concerns live side by side and share the word "cache".

The `IAppCache` family

`IAppCache` ^{15-appcache} is the base interface for all Umbraco application-level caches.

It lives in `Umbraco.Core/Cache/IAppCache.cs` and declares three essential operations:

- **Get** — retrieve a value by key, optionally creating it if absent
- **Clear** — remove one or all entries
- **Search** — find entries matching a key prefix or pattern

Two sub-interfaces extend it:

- `IAppPolicyCache` adds timeout and sliding-expiry support
- `IRequestCache` adds enumeration of current entries (one HTTP request only)

```
IAppCache
├─ IAppPolicyCache      (timeout-aware, used for RuntimeCache)
└─ IRequestCache        (one HTTP request lifetime)
```

Implementations you will see in the source

Class	What it is
<code>ObjectCacheAppCache</code>	Wraps .NET <code>MemoryCache</code> ; the default <code>RuntimeCache</code>
<code>DictionaryAppCache</code>	Simple dictionary; used for <code>RequestCache</code>
<code>FastDictionaryAppCache</code>	Thread-safe dictionary without lock contention
<code>DeepCloneAppCache</code>	Wraps another cache; deep-clones all values on read to prevent mutation bugs
<code>NoAppCache</code>	No-op; useful for tests or cases where caching should be skipped

`AppCaches`

`AppCaches` ^{15-appcaches} is the container object you will see passed around:

```
public class AppCaches
{
    public IAppPolicyCache RuntimeCache { get; }
    public IRequestCache RequestCache { get; }
    public IsolatedCaches IsolatedCaches { get; }
}
```

`IsolatedCaches` gives each repository or service type its own named cache bucket so they cannot accidentally clear each other's entries.

How to use it in your own code

```
public class MyService
{
    private readonly IAppPolicyCache _cache;

    public MyService(AppCaches appCaches)
    {
        _cache = appCaches.RuntimeCache;
    }

    public MyModel GetThing(int id)
    {
        return _cache.GetCacheItem($"my-thing-{id}", () => LoadFromDb(id));
    }
}
```

The factory lambda runs only on a cache miss.

The `ICacheRefresher` pattern

`ICacheRefresher` ^15-refresher is how Umbraco invalidates caches.

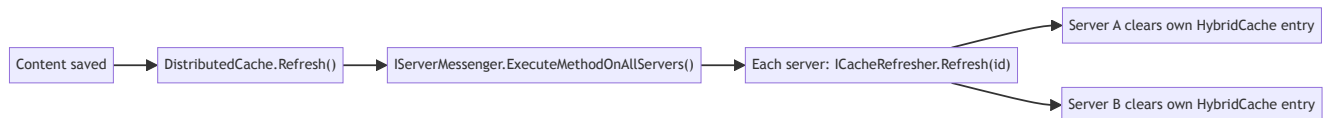
It lives in `Umbraco.Core/Cache/Refreshers/ICacheRefresher.cs`.

Each refresher has a unique GUID that identifies it across servers.

```
public interface ICacheRefresher
{
    Guid RefresherUniqueId { get; }
    string Name { get; }

    void RefreshAll();
    void Refresh(int id);
    void Refresh(Guid id);
    void Remove(int id);
}
```

When something changes on one server, `DistributedCache` ^15-distributed picks up the instruction and passes it to `IServerMessenger`, which broadcasts the same instruction to every server. Each server then runs the refresher locally.



The refresher inheritance chain

```
ICacheRefresher
├─ ICacheRefresher<T> (adds Refresh(T instance))
│   └─ IJsonCacheRefresher (adds Refresh(string jsonPayload))
│       └─ IPayloadCacheRefresher<TPayload> (typed payload)
└─ Base classes (in Umbraco.Core/Cache/Refreshers/):
    └─ CacheRefresherBase<TNotification>
        └─ JsonCacheRefresherBase<TNotification>
            └─ PayloadCacheRefresherBase<TNotification, TPayload>
```

The 17 (v17) / 19 (v18) concrete refreshers

Every significant entity type in Umbraco has a matching refresher:

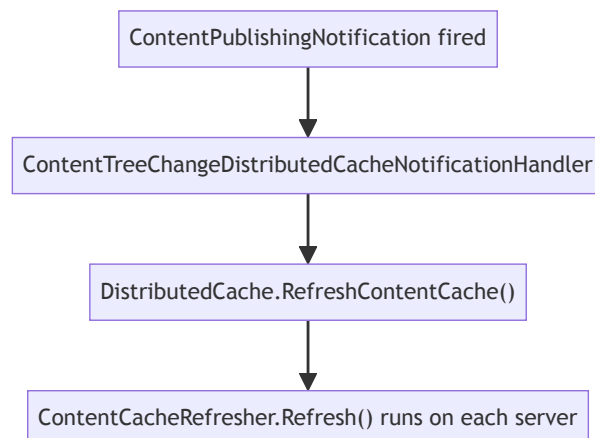
Refresher	Handles
ContentCacheRefresher	Published documents
MediaCacheRefresher	Published media
MemberCacheRefresher	Members
ContentTypeCacheRefresher	Content type definitions
DataTypeCacheRefresher	Data type configurations
TemplateCacheRefresher	Razor templates
LanguageCacheRefresher	Language and culture settings
DomainCacheRefresher	Domain assignments
DictionaryCacheRefresher	Dictionary items
UserCacheRefresher	Users
UserGroupCacheRefresher	User groups
MemberGroupCacheRefresher	Member groups
RelationTypeCacheRefresher	Relation types
PublicAccessCacheRefresher	Public access rules
ExternalMemberCacheRefresher	External identity members
ApplicationCacheRefresher	Backoffice sections
ValueEditorCacheRefresher	Property value editors
ElementCacheRefresher (v18)	Block/element types
ElementContainerCacheRefresher (v18)	Element container types

The notification handler pattern

Cache refreshers do not observe entity saves directly.

Instead, Umbraco uses a two-step chain:

1. An entity is saved → Umbraco raises a **notification** (e.g. `ContentSavingNotification`)
2. A **distributed cache notification handler** receives that notification and calls `DistributedCache` to schedule the refresher



The handler base classes live in `Umbraco.Core/Cache/NotificationHandlers/` :

Base class	Handles
<code>SavedDistributedCacheNotificationHandlerBase<TEntity, TNotification></code>	Saved entity notifications
<code>DeletedDistributedCacheNotificationHandlerBase<TEntity, TNotification></code>	Deleted entity notifications
<code>TreeChangeDistributedCacheNotificationHandlerBase<TEntity, TNotification></code>	Tree structure changes
<code>ContentTypeChangedDistributedCacheNotificationHandlerBase<TEntity, TNotification></code>	Type definition changes

For example, `DataTypeSavedDistributedCacheNotificationHandler` extends `SavedDistributedCacheNotificationHandlerBase` and simply calls `DistributedCache.RefreshDataTypeCache(savedEntities)`.

There are over 30 concrete implementations in the `Implement/` subfolder — one pair (saved + deleted) for each entity type that matters for caching.

The published cache type hierarchy

The "published content cache" is the cache that serves `IPublishedContent` objects for pages, media, and members.

At the top of this hierarchy are these interfaces in `Umbraco.Core/PublishedCache/`:

```
IPublishedCache          (base: GetById, GetAtRoot, HasContent)
├─ IPublishedContentCache (documents / pages)
├─ IPublishedMediaCache   (media)
├─ IPublishedElementCache (block elements – v18 only)
└─ IDomainCache           (domain assignments)

IPublishedMemberCache    (members – separate hierarchy)
```

`ICacheManager` aggregates all of these:

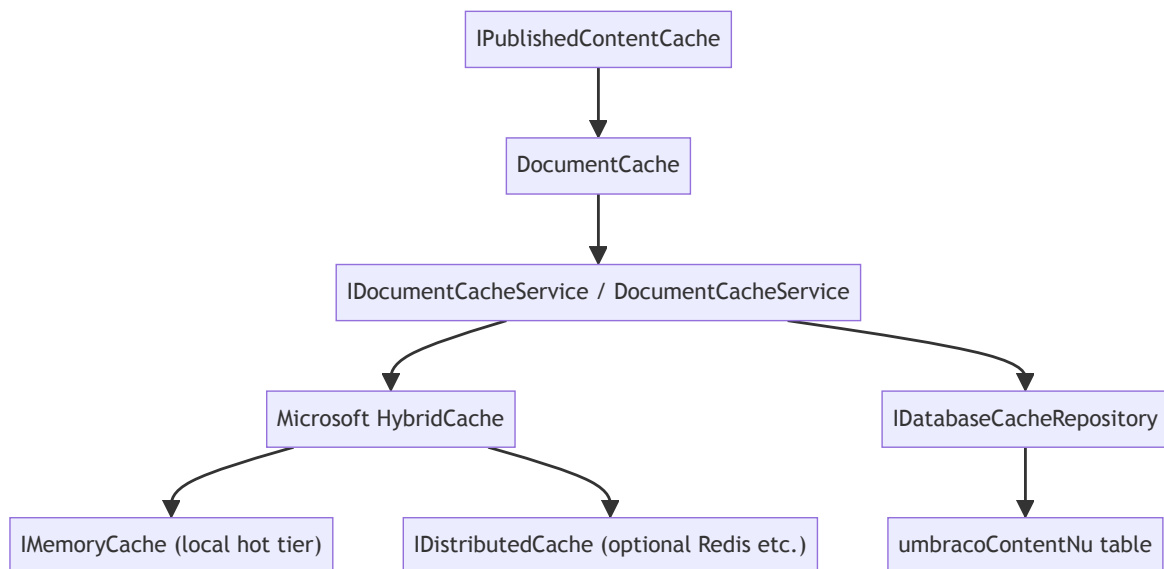
```
public interface ICacheManager
{
    IPublishedContentCache Content { get; }
    IPublishedMediaCache Media { get; }
    IPublishedMemberCache Members { get; }
    IDomainCache Domains { get; }
    // v18 adds: IPublishedElementCache Elements
}
```

If you want to understand what Umbraco does when you call `umbracoHelper.Content(id)`, the trail starts here.

The HybridCache implementation

The concrete published cache lives in the `Umbraco.PublishedCache.HybridCache` project.

Here is the layered structure:



DocumentCacheService — the key class

DocumentCacheService ^{^15-docsvc} is where a cache miss becomes a database read.

Its `GetByKeyAsync` method:

1. Calls `HybridCache.GetOrCreateAsync(key, ...)`
2. On cache miss, falls back to `IDatabaseCacheRepository.GetContentSourceAsync(key)`
3. Deserialises the result using `IContentCacheDataSerializer`
4. Returns the entry to `HybridCache` for storage

Serialisation options

The cache can serialise content using either:

- **MessagePack** (default) — compact binary format with optional LZ4 compression via `LazyCompressedString`
- **JSON** — available for debugging or migration from older versions

`NuCacheSerializerType` in the settings controls this, despite the NuCache name.

The seeding system

On startup, Umbraco does not wait for individual cache misses.

`SeedingNotificationHandler` triggers seed key providers to pre-warm the cache for common content.

The seed key provider types:

Provider	Strategy
<code>DocumentBreadthFirstKeyProvider</code>	Top-level pages first, then deeper levels
<code>MediaBreadthFirstKeyProvider</code>	Same pattern for media
<code>ContentTypeSeedKeyProvider</code>	All items of specific content types
<code>ElementBreadthFirstKeyProvider</code> (v18)	Block element seeding

Repository cache policies

Repositories that deal with entities (content types, templates, languages, etc.) use a separate caching layer called **repository cache policies**.

The interface is `IRepositoryCachePolicy<TEntity, TId>`.

Available policies in `Umbraco.Infrastructure/Cache/`:

Policy	Behaviour
DefaultRepositoryCachePolicy	Cache by ID and by GUID key
FullDataSetRepositoryCachePolicy	Cache entire result set (good for small, infrequently changed sets)
SingleItemsOnlyRepositoryCachePolicy	Cache individual fetches but not collection queries
GuidReadRepositoryCachePolicy	Cache GUID-based lookups only
MemberRepositoryUsernameCachePolicy	Adds username-keyed lookups on top
NoCacheRepositoryCachePolicy	No caching (test or admin-only repositories)

Each repository picks its own policy when constructed.

Output cache types

Output caching is a separate concern from published content caching.

The interfaces for website output caching live in `Umbraco.Core/Cache/` :

Interface	Purpose
<code>IWebsiteOutputCacheManager</code>	Evict entries programmatically
<code>IWebsiteOutputCacheDurationProvider</code>	Control how long pages are cached
<code>IWebsiteOutputCacheTagProvider</code>	Assign tags to responses so they can be evicted by group
<code>IWebsiteOutputCacheEvictionProvider</code>	Define extra eviction logic per content node
<code>IWebsiteOutputCacheRequestFilter</code>	Decide whether a request is eligible for caching
<code>IWebsiteOutputCacheVaryByProvider</code>	Define Vary-By dimensions (e.g. culture, cookie)

The Delivery API has its own mirror set:

- `IDeliveryApiOutputCacheManager`
- `IDeliveryApiOutputCacheTagProvider`
- `IDeliveryApiOutputCacheEvictionProvider`
- `IDeliveryApiOutputCacheRequestFilter`
- `IDeliveryApiOutputCacheVaryByProvider`

The eviction handlers in `Umbraco.Web.Website/Caching/` are what connect entity changes to output-cache evictions:

- `WebsiteDocumentOutputCacheEvictionHandler` — evicts pages by content key tag or ancestor tag
- `WebsiteMediaOutputCacheEvictionHandler` — evicts pages referencing changed media
- `WebsiteMemberOutputCacheEvictionHandler` — evicts pages referencing changed members

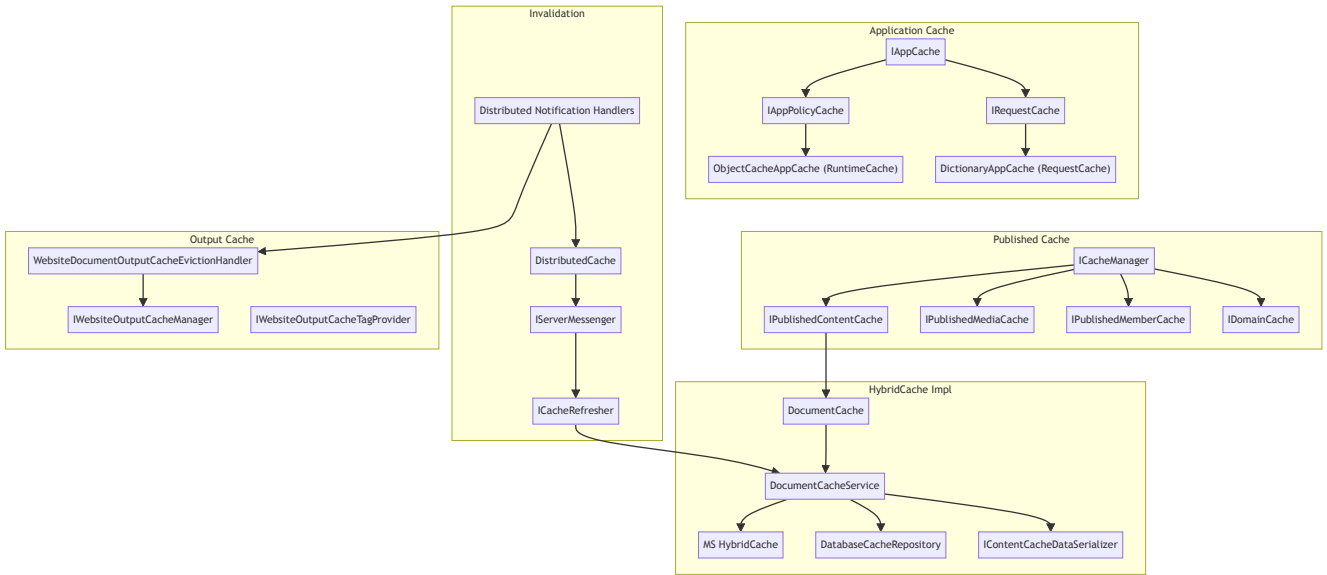
In v18, `DeliveryApiElementOutputCacheEvictionHandler` adds the same pattern for block element changes.

Where to look for what

When you need to understand a specific cache behaviour, start here:

Question	Start here
"What is cached and how?"	<code>Umbraco.Core/Cache/IAppCache.cs</code> , <code>AppCaches.cs</code>
"How does content become cached?"	<code>Umbraco.PublishedCache.HybridCache/Services/DocumentCacheService.cs</code>
"How is a publish turned into a cache bust?"	<code>Umbraco.Core/Cache/Refreshers/Implement/ContentCacheRefresher.cs</code>
"How is the bust broadcast to other servers?"	<code>Umbraco.Core/Cache/DistributedCache.cs</code>
"How does output cache know to evict a page?"	<code>Umbraco.Web.Website/Caching/WebsiteDocumentOutputCacheEvictionHandler.cs</code>
"How are repository entities cached?"	<code>Umbraco.Infrastructure/Cache/DefaultRepositoryCachePolicy.cs</code>
"What are all the refreshers?"	<code>Umbraco.Core/Cache/Refreshers/CacheRefresherCollection.cs</code>
"How is cache pre-warmed?"	<code>Umbraco.PublishedCache.HybridCache/NotificationHandlers/SeedingNotificationHandler.cs</code>

The full type map (v17)



What v18 adds to the type map

The v18 additions are almost entirely about giving block/element content its own first-class slot in every layer:

v17 Published Cache:	v18 adds:
<code>IPublishedContentCache</code>	<code>IPublishedElementCache</code>
<code>IDocumentCacheService</code>	<code>IElementCacheService</code>
<code>ContentCacheRefresher</code>	<code>ElementCacheRefresher</code>
	<code>ElementContainerCacheRefresher</code>
<code>(no element handler)</code>	<code>ElementTreeChangeDistributedCacheNotificationHandler</code>
<code>(no element eviction)</code>	<code>DeliveryApiElementOutputCacheEvictionHandler</code>
<code>(no element seeding)</code>	<code>ElementBreadthFirstKeyProvider</code>
	<code>IElementSeedKeyProvider</code>

The pattern is identical to documents and media — element types just did not have their own lane before v18.

Beginner rules for reading the code

- When you see `AppCaches` , think "read/write cached data for this server".
 - When you see `DistributedCache` , think "tell other servers something changed".
 - When you see `ICacheRefresher` , think "the actual invalidation logic that runs everywhere".
 - When you see `INotificationHandler<XxxNotification>` in the cache namespace, think "the bridge between entity events and cache invalidation".
 - When you see `HybridCache` being injected, think "the published content storage and retrieval layer".
 - The word "distributed" in Umbraco code usually means "reaches multiple servers", not "stored in Redis".
-

Sources

- Code:
 - `umbraco-v17/src/Umbraco.Core/Cache/IAppCache.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/AppCaches.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/DistributedCache.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/Refreshers/ICacheRefresher.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/Refreshers/CacheRefresherBase.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/Refreshers/Implement/ContentCacheRefresher.cs`
 - `umbraco-v17/src/Umbraco.Core/Cache/NotificationHandlers/`
 - `umbraco-v17/src/Umbraco.Core/PublishedCache/ICacheManager.cs`
 - `umbraco-v17/src/Umbraco.Core/PublishedCache/IPublishedContentCache.cs`
 - `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/Services/DocumentCacheService.cs`
 - `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/Persistence/DatabaseCacheRepository.cs`
 - `umbraco-v17/src/Umbraco.PublishedCache.HybridCache/NotificationHandlers/SeedingNotificationHandler.cs`
 - `umbraco-v17/src/Umbraco.Infrastructure/Cache/DefaultRepositoryCachePolicy.cs`
 - `umbraco-v17/src/Umbraco.Web.Website/Caching/WebsiteDocumentOutputCacheEvictionHandler.cs`
 - `umbraco-v18/src/Umbraco.Core/PublishedCache/IPublishedElementCache.cs`
 - `umbraco-v18/src/Umbraco.Core/Cache/Refreshers/Implement/ElementCacheRefresher.cs`

99. Notes Not Yet Added To The Book

This file is a working backlog.

It tracks where source-backed observations have landed in the main chapters, and what (if anything) is still genuinely missing.

Already woven into chapters

Most of the original notes are now covered. Quick index of where they live:

- General cache families and the "not one cache" framing → [Chapter 1](#)
- Microsoft `HybridCache` stack (`IMemoryCache` , `IDistributedCache` , `HybridCache`) → [Chapter 1](#) and [Chapter 10](#)
- NuCache vs Hybrid Cache, plus the lingering v17 names → [Chapter 12](#) (now with comparison and leftovers tables)
- Future Hybrid Cache architecture: L0 hot lane, serialised entries, database-backed source, generation counter, negative-cache tagging, seeding, deferred rebuilds, serialiser-change rebuilds → [Chapter 10](#)
- Cache busting and invalidation, refresh types, broad refreshers → [Chapter 4](#)
- Publish vs Deploy vs output-cache eviction (the three busting paths) → [Chapter 4](#)
- Deploy specifics (`SuppressCacheRefresherNotifications` , suspend during import/restore, broader memory reload) → [Chapter 5](#) and [Chapter 8](#)
- Website output caching → [Chapter 2](#)
- Storage providers and media caching (CDN URLs, blob metadata via `HybridCache` , `ImageSharp` cache) → [Chapter 9](#)
- Talks, PDFs, and blog framing → [Chapter 6](#) and the [appendix](#)

Diagrams now added

- Cache architecture timeline, v7 to v18 → [Chapter 1](#) (`cache-timeline.svg`)
- How the cache families relate: engine vs Umbraco layers vs invalidation → [Chapter 1](#) (`cache-layers.svg`)

Still worth expanding later

The big-ticket backlog items are done. What remains is lower-priority polish:

- A focused audit of every NuCache-named leftover in v18 `main` , confirming which are still only names versus which now drive behaviour (the v17 view is captured in [Chapter 12](#)).
- A worked, end-to-end example tracing a single publish through every layer it touches, with real key values.
- A configuration reference: every cache-related setting and its effect, gathered in one place.

Nice quotes or concepts to maybe reuse later

- "Microsoft built the cache engine; Umbraco built a published-content pipeline around it."
- "The cache busting path is the real architecture."
- "NuCache is the older alternative; Hybrid Cache is the active one."
- "The middleware/output-cache layer and the published-content layer solve different problems."

Source families now in use

- Microsoft docs and blog
- Umbraco docs
- Umbraco blog
- Umbraco source code
- Umbraco.StorageProviders repository
- talks and PDFs
- local Deploy decompilation notes